

Hardware data sheet section III

ET1810 / ET1811 / ET1812

Ether**CAT**[®]  slave controller
IP core for Altera[®] FPGAs
Release 4.0.0

Section I – Technology
(Online at <http://www.beckhoff.com>)

Section II – Register description
(Online at <http://www.beckhoff.com>)

Section III – Hardware description
Installation, configuration, resource
consumption, interface specification

Version 1.0
Date: 2025-09-15

BECKHOFF

DOCUMENT ORGANIZATION

The Beckhoff EtherCAT slave controller (ESC) documentation covers the following Beckhoff ESCs:

- ET1200
- ET1100, ET1150
- EtherCAT IP core for FPGAs
- ESC20

The documentation is organized in three sections. Section I and section II are common for all Beckhoff ESCs, section III is specific for each ESC variant.

The latest documentation is available at the Beckhoff homepage (<http://www.beckhoff.com>).

Section I – technology (all ESCs)

Section I deals with the basic EtherCAT technology. Starting with the EtherCAT protocol itself, the frame processing inside EtherCAT slaves is described. The features and interfaces of the physical layer with its two alternatives Ethernet and EBUS are explained afterwards. Finally, the details of the functional units of an ESC like FMMU, SyncManager, distributed clocks, slave information interface, interrupts, watchdogs, and so on, are described.

Since section I is common for all Beckhoff ESCs, it might describe features which are not available in a specific ESC. Refer to the feature details overview in section III of a specific ESC to find out which features are available.

Section II – register description (all ESCs)

Section II contains detailed information about all ESC registers. This section is also common for all Beckhoff ESCs, thus registers, register bits, or features are described which might not be available in a specific ESC. Refer to the register overview and to the feature details overview in section III of a specific ESC to find out which registers and features are available.

Section III – hardware description (specific ESC)

Section III is ESC specific and contains detailed information about the ESC features, implemented registers, configuration, interfaces, pinout, usage, electrical and mechanical specification, and so on. Especially the process data interfaces (PDI) supported by the ESC are part of this section.

Additional documentation

Application notes and utilities can also be found at the Beckhoff homepage. Pinout configuration tools for EtherCAT ASICs are available. Additional information on EtherCAT IP cores with latest updates regarding design flow compatibility, FPGA device support and known issues are also available.

Trademarks

Beckhoff®, ATRO®, EtherCAT®, EtherCAT G®, EtherCAT G10®, EtherCAT P®, MX-System®, Safety over EtherCAT®, TC/BSD®, TwinCAT®, TwinCAT/BSD®, TwinSAFE®, XFC®, XPlanar®, and XTS® are registered trademarks of and licensed by Beckhoff Automation GmbH. Other designations used in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owners.

Disclaimer

The documentation has been prepared with care. The products described are, however, constantly under development. For that reason, the documentation is not in every case checked for consistency with performance data, standards or other characteristics. In the event that it contains technical or editorial errors, we retain the right to make alterations at any time and without warning. No claims for the modification of products that have already been supplied may be made on the basis of the data, diagrams and descriptions in this documentation.

Copyright

© Beckhoff Automation GmbH & Co. KG 09/2025.

The reproduction, distribution and utilization of this document as well as the communication of its contents to others without express authorization are prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design.

DOCUMENT HISTORY

Version	Comment
1.0	Initial release EtherCAT IP core for Altera FPGAs 4.0.0

CONTENTS

1	Overview	1
1.1	Frame processing order	2
1.2	Scope of this document	3
1.3	Scope of delivery	3
1.4	Target FPGAs and design flow requirements	4
1.4.1	Altera FPGAs and design flow versions supported at the time of release	4
1.5	Tested FPGA/design flow combinations	5
1.6	Release notes V4.0.0 (09/2025)	6
1.6.1	Reading IP core version from device	7
1.7	Design flow	8
1.8	Intel FPGA IP evaluation mode	9
1.9	Simulation	10
2	Features and registers	11
2.1	Features	11
2.2	Registers	15
2.3	ESC features (power-on values of user RAM)	20
2.4	ESC port features (power-on values of user RAM)	26
3	IP core installation	28
3.1	System requirements	28
3.2	EtherCAT IP core installation	28
3.2.1	EtherCAT IP core ZIP file	28
3.2.2	Vendor ID package	28
3.2.3	License file	28
4	IP core usage	31
4.1	IP core configuration	32
4.1.1	Overview	32
4.1.2	Identification tab	33
4.1.3	Physical layer tab	34
4.1.4	Core basics tab	36
4.1.5	Core details tab	38
4.1.6	PDI basics tab	40
4.1.7	PDI0/PDI1 configuration tab	42
4.1.8	Integration tab	51
4.1.9	Information tab	52
5	FPGA resource consumption	54
6	IP core signals	57
6.1	General signals	57
6.1.1	Clock source example schematics	58
6.2	SII EEPROM interface signals	59
6.3	LED signals	59

6.4	Distributed clocks SyncSignals/LatchSignals	60
6.5	Physical layer interface	61
6.5.1	MII interface	63
6.5.2	RMII interface	65
6.5.3	RGMII interface	66
6.6	PDI signals	70
6.6.1	General PDI signals	70
6.6.2	Digital I/O interface	70
6.6.3	SPI slave interface	72
6.6.4	Asynchronous 8/16/32 bit μ Controller interface	74
6.6.5	Avalon on-chip bus	75
6.6.6	AXI4 / AXI4-Lite on-chip bus	77
7	Ethernet interface	80
7.1	PHY management interface	80
7.1.1	PHY management interface signals	80
7.1.2	PHY address configuration	81
7.1.3	MI link detection and configuration	82
7.1.4	MII management timing specifications	83
7.2	PHY reset	84
7.2.1	PHY reset timing specifications	84
7.3	MII interface	85
7.3.1	MII interface signals	85
7.3.2	TX shift compensation	86
7.3.3	MII timing specifications	87
7.3.4	MII example schematic	88
7.4	RMII interface	89
7.4.1	RMII interface signals	89
7.4.2	RMII example schematic	90
7.5	RGMII interface	91
7.5.1	RGMII interface signals	92
7.5.2	RGMII example schematic	93
7.5.3	RGMII RX timing options	94
7.5.4	RGMII TX timing options	94
7.5.5	RGMII timing specifications	94
8	PDI description	96
8.1	PDI deactivated	97
8.2	Digital I/O interface	98
8.2.1	Interface	98
8.2.2	Configuration	99
8.2.3	Digital inputs	99
8.2.4	Digital outputs	99

8.2.5	Output enable	100
8.2.6	SyncManager watchdog	100
8.2.7	SOF	101
8.2.8	OUTVALID	101
8.2.9	Timing specifications	101
8.3	SPI slave interface	104
8.3.1	Interface	104
8.3.2	Configuration	105
8.3.3	Access types	105
8.3.4	SPI_SEL random access	105
8.3.5	SPI_SEL_MISO/SPI_SEL_MOSI configured SyncManager access	109
8.3.6	MISO/MOSI data width x1, x2, x4, x8	110
8.3.7	Access errors and SPI status flag	110
8.3.8	Timing specifications	112
8.4	Asynchronous 8/16/32 bit μ Controller interface	123
8.4.1	Interface	123
8.4.2	Configuration	124
8.4.3	μ Controller access	124
8.4.4	Write access	125
8.4.5	Read access	125
8.4.6	μ Controller access errors	126
8.4.7	Default BUSY state	126
8.4.8	Connection with 16 bit μ Controllers without byte addressing	126
8.4.9	Connection with 8 bit μ Controllers	127
8.4.10	Timing specification	128
8.5	Avalon on-chip bus	132
8.5.1	Interface	132
8.5.2	Configuration	133
8.5.3	Interrupts	133
8.5.4	On-chip bus access and data bus width	133
8.5.5	Timing specifications	134
8.6	AXI4/AXI4-Lite on-chip bus	136
8.6.1	Interface	136
8.6.2	Configuration	138
8.6.3	Interrupts	138
8.6.4	On-chip bus access and data bus width	138
8.6.5	Timing specifications	139
9	Distributed clocks DC_SYNC/DC_LATCH signals	141
9.1	Signals	141
9.2	Timing specifications	141
10	SII EEPROM interface (I ² C)	142

	10.1 Signals	142
	10.2 Timing specifications	142
11	Electrical specifications	143
12	Synthesis constraints	144
	12.1 Timing requirements	144
	12.2 Automatic TX shift compensation	146
	12.3 Manual TX shift compensation	148
13	Appendix	149
	13.1 Third party trademarks	149
	13.2 Support and service	149
	13.2.1 Beckhoff's branch offices and representatives	149
	13.3 Beckhoff headquarters	149

TABLES

Table 1: IP core main features	1
Table 2: Frame processing order	2
Table 3: Tested FPGA/design flow combinations	5
Table 4: Register revision (0x0001)	7
Table 5: Register build (0x0002:0x0003)	7
Table 6: IP core feature details	11
Table 7: ESC feature legend	14
Table 8: Register availability	15
Table 9: ESC register legend	19
Table 10: ESC features (reset values of user RAM – 0x0F80:0x0FFF)	20
Table 11: EtherCAT IP core ZIP content	28
Table 12: Approximate resource requirements for main configurable functions	55
Table 13: EtherCAT IP core configuration for typical EtherCAT devices	56
Table 14: General signals	57
Table 15: SII EEPROM signals	59
Table 16: LED signals	59
Table 17: DC SyncSignals/LatchSignals	60
Table 18: Physical layer general	61
Table 19: PHY interface MII	63
Table 20: PHY interface RMII	65
Table 21: PHY interface RGMII	66
Table 22: General PDI signals	70
Table 23: Digital I/O PDI0	70
Table 24: Digital I/O PDI1	71
Table 25: SPI slave PDI0	72
Table 26: SPI slave PDI1	73
Table 27: 8/16/32 Bit μ C PDI0	74
Table 28: Avalon PDI0	75
Table 29: Avalon PDI1	76
Table 30: AXI4 / AXI4-Lite PDI0	77
Table 31: AXI4 / AXI4-Lite PDI1	78
Table 32: PHY management interface signals	80
Table 33: MI link detection for 100Base-TX	82
Table 34: MI link detection for 100Base-FX	82
Table 35: PHY initialization ROM interface signals	83
Table 36: PHY initialization ROM layout	83
Table 37: MII management timing characteristics	83
Table 38: PHY reset signals	84
Table 39: PHY reset timing characteristics	84
Table 40: MII interface signals for port n	85
Table 41: MII TX timing characteristics	87
Table 42: MII timing characteristics	87
Table 43: RMII interface signals for port n	90
Table 44: RGMII interface signals for port n	92
Table 45: RGMII timing characteristics IP core	94
Table 46: Available PDIs for EtherCAT IP core	96
Table 47: IP core digital I/O signals	98
Table 48: Input/output register reference	99
Table 49: Digital I/O timing characteristics IP core	101
Table 50: SPI signals	105
Table 51: SPI commands CMD0 and CMD1	106
Table 52: Address modes without wait state byte (read access without wait state byte)	106
Table 53: Address modes for read access with wait state byte	106
Table 54: Interrupt request register transmission	107
Table 55: Write access for 2 and 4 byte SPI masters	108
Table 56: SyncManager used by SPI slave for SPI_SEL_MISO/SPI_SEL_MOSI	109
Table 57: SPI slave timing characteristics IP core	112
Table 58: SPI slave max. SPI_CLK frequency $f_{CLK}=1/t_{CLK}$	112
Table 59: Read/write timing diagram symbols	113
Table 60: μ Controller signals	123

Table 61: 8 bit high/low byte and 16 bit access distinction	124
Table 62: 32 bit access distinction	124
Table 63: μ Controller timing characteristics IP core.....	128
Table 64: Avalon signals	132
Table 65: Avalon timing characteristics	134
Table 66: AXI4-Lite signals.....	136
Table 67: Additional AXI4 signals.....	137
Table 68: AXI timing characteristics	139
Table 69: Distributed clocks signals	141
Table 70: DC SYNC/LATCH timing characteristics IP core	141
Table 71: I ² C EEPROM signals	142
Table 72: EEPROM timing characteristics IP core.....	142
Table 73: AC Characteristics	143
Table 74: Forwarding delays	143
Table 75: EtherCAT IP core constraints	144

FIGURES

Figure 1: EtherCAT IP core block diagram.....	1
Figure 2: Frame processing.....	2
Figure 3: Design flow.....	8
Figure 4: License setup.....	29
Figure 5: EtherCAT IP core configuration interface.....	32
Figure 6: Identification tab.....	33
Figure 7: Physical layer tab.....	34
Figure 8: Core basics tab.....	36
Figure 9: Core details tab.....	38
Figure 10: PDI basics tab.....	40
Figure 11: Available PDI interfaces.....	42
Figure 12: PDI configuration tab – digital I/O configuration.....	43
Figure 13: PDI configuration tab – μ C configuration.....	45
Figure 14: PDI configuration tab – SPI configuration.....	47
Figure 15: Register PDI – Avalon interface configuration.....	49
Figure 16: PDI configuration tab – AXI interface configuration.....	50
Figure 17: Integration tab.....	51
Figure 18: Information tab.....	52
Figure 19: EtherCAT IP core clock source (MII).....	58
Figure 20: EtherCAT IP core clock source (RMII).....	58
Figure 21: EtherCAT IP core clock source (RGMII).....	58
Figure 22: PHY management interface signals.....	80
Figure 23: PHY initialization ROM interface signals.....	82
Figure 24: MII interface signals for port n	85
Figure 25: MII TX timing diagram.....	86
Figure 26: MII timing RX signals.....	87
Figure 27: MII example schematic.....	88
Figure 28: RMII interface signals for port n	89
Figure 29: RMII example schematic.....	90
Figure 30: RGMII interface signals for port n	92
Figure 31: RGMII example schematic.....	93
Figure 32: RGMII receive timing.....	95
Figure 33: RGMII transmit timing.....	95
Figure 34: IP core digital I/O signals.....	98
Figure 35: Digital output principle schematic.....	100
Figure 36: Digital input: input data sampled at SOF, input data can be read in the same frame.....	102
Figure 37: Digital input: input data sampled with LATCH_IN.....	102
Figure 38: Digital input: input data sampled with DC_SYNC.....	102
Figure 39: Digital output timing.....	103
Figure 40: OUT_ENA timing.....	103
Figure 41: SPI master and slave interconnection (unidirectional).....	104
Figure 42: SPI master and slave interconnection (bidirectional).....	104
Figure 43: SPI slave bidirectional mode with simultaneous SEL_MISO/SEL_MOSI access.....	110
Figure 44: Basic SPI_MOSI/SPI_MISO timing (*refer to access diagrams for relevant edges of SPI_CLK).....	113
Figure 45: SPI_SEL random read access (2 byte addressing, 1 byte read data) with wait state byte.....	114
Figure 46: SPI_SEL random read access (2 byte addressing, 2 byte read data) with wait state byte.....	115
Figure 47: SPI_SEL random write access (2 byte addressing, 1 byte write data).....	116
Figure 48: SPI_SEL random write access (3 byte addressing, 1 byte write data).....	117
Figure 49: SPI_SEL random read access (2 bit data width, with wait state byte).....	118
Figure 50: SPI_SEL random read access (4 bit data width, with wait state byte).....	119
Figure 51: SPI_SEL random read access (8 bit data width, with wait state byte).....	120
Figure 52: SPI_SEL_MISO access (wait state byte).....	121
Figure 53: SPI_SEL_MISO access (no wait state byte).....	122
Figure 54: μ Controller interface interconnection.....	123
Figure 55: Connection with 8 bit μ Controllers.....	127
Figure 56: Read access (without preceding write access).....	129
Figure 57: Write access (write after rising edge nWR, without preceding write access).....	130
Figure 58: Sequence of two write accesses and a read access.....	130
Figure 59: Write access (write after falling edge nWR).....	131

Figure 60: Avalon signals	132
Figure 61: Avalon read access (single access).....	135
Figure 62: Avalon read access (multiple access).....	135
Figure 63: Avalon write access (multiple access)	135
Figure 64: AXI4 signals/channels	136
Figure 65: AXI read access	140
Figure 66: AXI write access	140
Figure 67: Distributed clocks signals	141
Figure 68: LatchSignal timing	141
Figure 69: SyncSignal timing.....	141
Figure 70: I ² C EEPROM signals.....	142
Figure 71: MII auto TX shift constraining (principle).....	146

ABBREVIATIONS

μC	Microcontroller
ADR	Address
AL	Application layer
AMBA®	Advanced microcontroller bus architecture from ARM®
AXI™	Advanced eXtensible interface bus, an AMBA interconnect. Used as on-chip-bus
BHE	Bus high enable
CMD	Command
CS	Chip select
DC	Distributed clock
DL	Data link layer
ECAT	EtherCAT
EOF	End of frame
ESC	EtherCAT slave controller
ESI	EtherCAT slave information
FMMU	Fieldbus memory management unit
FPGA	Field programmable gate array
GPI	General purpose input
GPO	General purpose output
HDL	Hardware description language
IP	Intellectual property
IRQ	Interrupt request
LC	Logic cell
LE	Logic element
MAC	Media access controller
MDIO	Management data input / output
MI	(PHY) management interface
MII	Media independent interface
MISO	Master in – slave out
MOSI	Master out – slave in
PDI	Process data interface
PLD	Programmable Logic Device
PLL	Phase Locked loop
RD	Read
RMII	Reduced media independent interface
RGMII	Reduced gigabit media independent interface
SM	SyncManager
SoC	System on a chip
SOF	Start of frame
SPI	Serial peripheral interface
VHDL	Very high speed integrated circuit hardware description language
WR	Write

1 Overview

The EtherCAT IP core is a configurable EtherCAT slave controller (ESC). It takes care of the EtherCAT communication as an interface between the EtherCAT fieldbus and the slave application. The EtherCAT IP core is delivered as a configurable system so that the feature set fits the requirements perfectly and brings costs down to an optimum.

Table 1: IP core main features

Feature	IP core configurable features
Ports	1-4 MII/RMII/RGMII ports
FMMUs	0-16
SyncManagers	0-16
RAM	0-60 KB
Distributed clocks	Yes, 32 bit or 64 bit
Process data interfaces	Up to two interfaces: PDI0: 32 Bit digital I/O (unidirectional) - SPI slave (1/2/4/8 MISO/MOSI signals) 8/16/32 bit asynchronous μController interface - Avalon® on-chip bus - AMBA® AXI4/AXI4-Lite on-chip bus PDI1: 32 Bit digital I/O (unidirectional) - SPI slave (1/2/4/8 MISO/MOSI signals) - Avalon on-chip bus - AMBA AXI4/AXI4-Lite on-chip bus
Other features	- Example designs for easy start up available online - Slave applications can run on-chip if the appropriate FPGAs with sufficient resources are used

The general functionality of the EtherCAT IP core is shown in Figure 1:

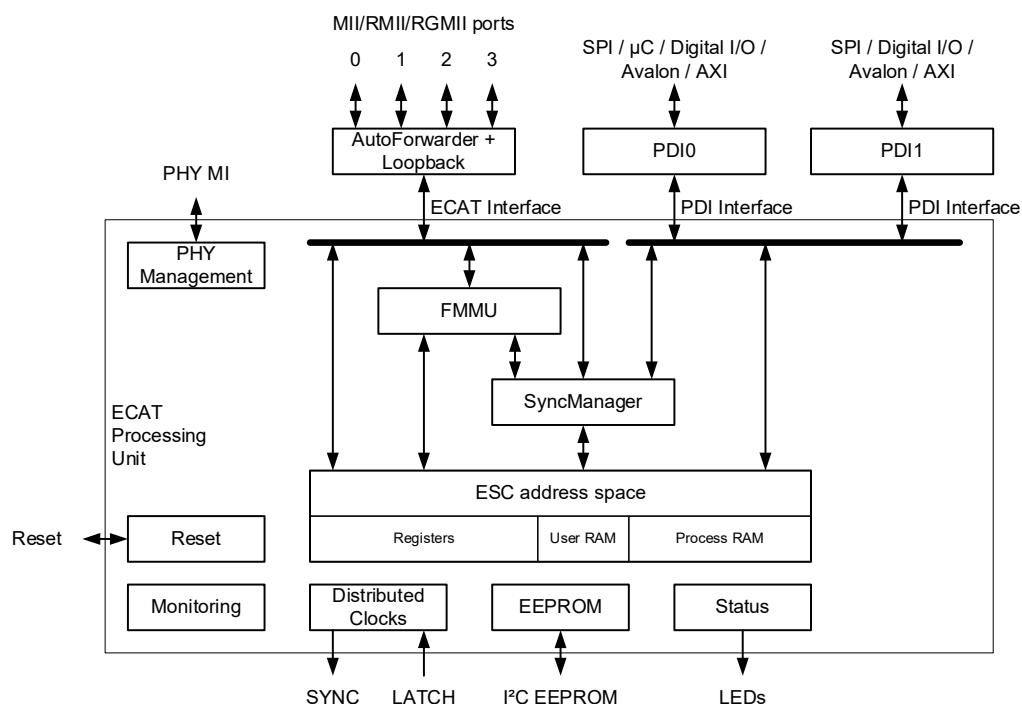


Figure 1: EtherCAT IP core block diagram

1.1 Frame processing order

The frame processing order of the EtherCAT IP core is as follows:

Table 2: Frame processing order

Number of ports	Frame processing order
1	0→EtherCAT processing unit→0
2	0→EtherCAT processing unit→1 / 1→0
3	0→EtherCAT processing unit→1 / 1→2 / 2→0
4	0→EtherCAT processing unit→3 / 3→1 / 1→2 / 2→0

Figure 2 shows the frame processing in general:

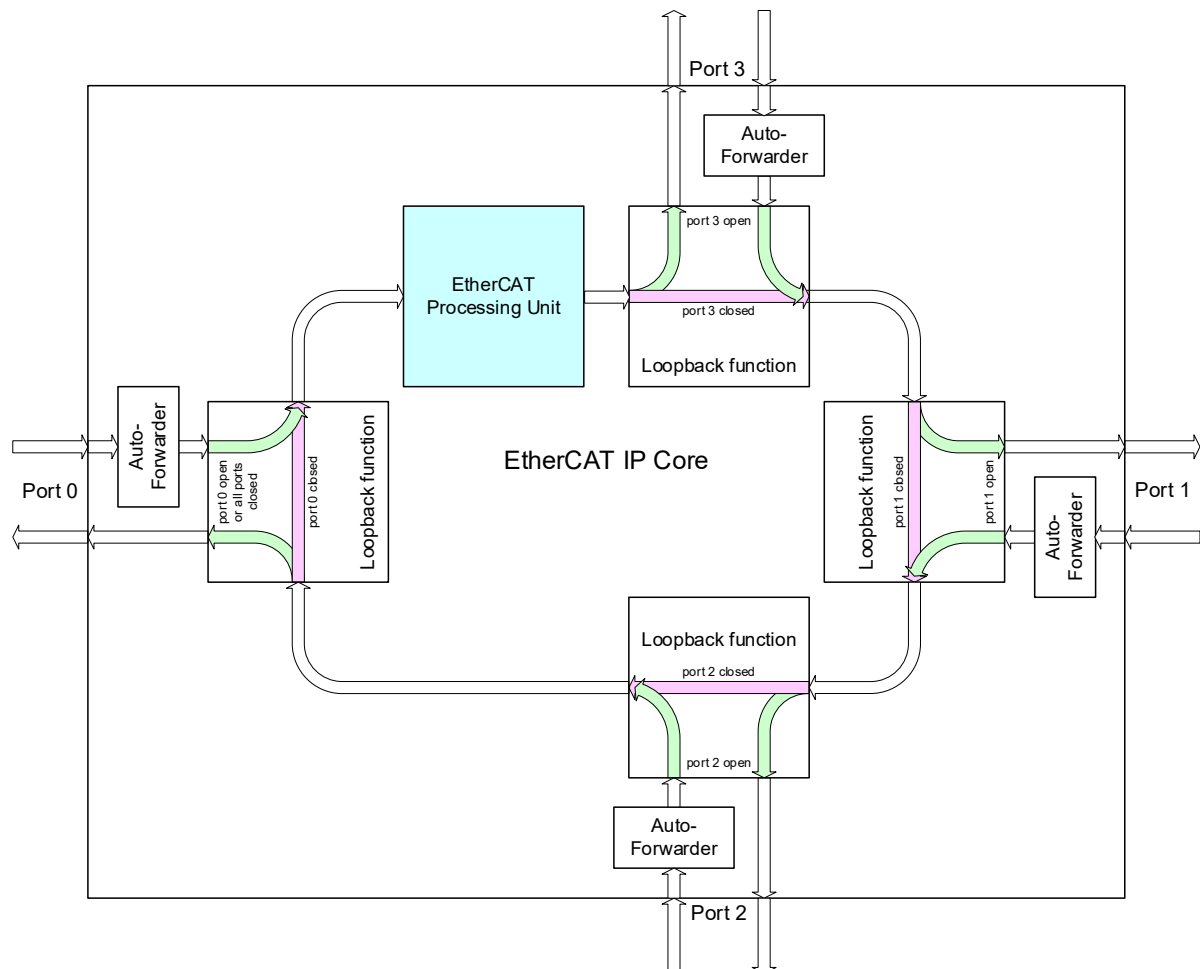


Figure 2: Frame processing

Frame processing Example with ports 0 and 1

A frame received at port 0 goes via the auto-forwarder and the loopback function to the EtherCAT processing unit which processes it. Then, the frame is sent to port 1. If port 1 is open, the frame is sent out at port 1. If it is closed, the frame is forwarded by the loopback function to port 2. Since port 2 is not configured, the loopback function of port 2 forwards the frame to the loopback function of port 0, and then it is sent out at port 0 – back to the master.

1.2 Scope of this document

Purpose of this document is to describe the installation, configuration, and usage of the EtherCAT IP core for Altera FPGAs. Furthermore, the signals and registers of the IP core depending on the chosen configuration are described.

This documentation was made with the assumption that the user is familiar with the usage of the FPGA development environment.

1.3 Scope of delivery

The EtherCAT IP core is delivered including:

- EtherCAT IP core (encrypted VHDL library)
- IP core configuration in Quartus
- Documentation
- Example timing constraints
- Example implementation of a PHY initialization ROM

The following additional items, which contain customer specific information, are required to synthesize the IP core.

- License file to use the EtherCAT IP core
- Encrypted vendor ID package: pk_ECAT_VENDORID_<company>_Altera.vhd

Example designs showing how to use the EtherCAT IP core are available separately.

1.4 Target FPGAs and design flow requirements

The EtherCAT IP core is designed to support a wide range of FPGAs without modifications, because it does not instantiate dedicated FPGA resources, or rely on device specific features. Thus, the IP core is easily portable to new FPGA families.

The complexity of the IP core is highly configurable, so its demands for logic resources, memory blocks, and FPGA speed cover a wide range. Thus, it is not possible to run any IP core configuration on any target FPGA with any speed grade. I.e., there are IP core configurations requiring a faster speed grade, or a larger FPGA, or even a more powerful FPGA family.

It is necessary to run through the whole synthesis process – including timing checks –, to evaluate if the selected FPGA is suitable for a certain IP core configuration before making the decision for the FPGA. Please consider a security margin for the logic resources to allow for minor enhancements and bug fixes of the IP core and the user logic.

Most likely, newer FPGA families and newer design flow versions are supported by the EtherCAT IP. Installation of the latest service pack is recommended.

Refer to the *Hardware data sheet section III addendum* available at the Beckhoff homepage (<http://www.beckhoff.com>) for latest updates regarding device support, design flow compatibility, and known issues. This addendum is regularly updated.

1.4.1 Altera FPGAs and design flow versions supported at the time of release

The EtherCAT IP core for Altera® FPGAs is targeted at these FPGA families:

- Agilex™ 3, Agilex 5, Agilex 7
- Arria® V, Arria V GZ, Arria 10
- Cyclone® IV E+GX, Cyclone V, Cyclone V SoC, Cyclone 10 LP+GX
- MAX®10
- Stratix® IV, Stratix V, Stratix 10

For synthesis of the EtherCAT IP core for Altera FPGAs, at least one of the following Altera Quartus versions is needed (with latest service pack):

- Altera Quartus® Prime Standard 23.1 – 24.1
- Altera Quartus Prime Pro 23.1 – 25.1

1.5 Tested FPGA/design flow combinations

The EtherCAT IP core has been synthesized successfully with different design flow versions and FPGA families. The following chapter lists combinations of FPGA devices and design tools versions which have been synthesized or even tested in real hardware. This list does not claim to be complete, it just illustrates that the EtherCAT IP core is designed to comply with a broad spectrum of FPGAs.

Refer to the *Hardware data sheet section III addendum* available at the Beckhoff homepage (<http://www.beckhoff.com>) for latest updates regarding device support, design flow compatibility, and known issues. This addendum is regularly updated.

Table 3: Tested FPGA/design flow combinations

IP core	Family	Device	Designflow	Test	Used Example Designs
4.0.0	Agilex 3	A3CY100	Quartus Pro 25.1	Synthesis	
	Agilex 5	A5DC051	Quartus Pro 25.1	Synthesis	
	Agilex 7	AGFA006	Quartus Pro 23.1	Synthesis	
	Arria V	5AGXFB3	Quartus Std 23.1	Synthesis	
	Arria V GZ	5AGZME5	Quartus Std 23.1	Synthesis	
	Arria 10	10AS016	Quartus Pro 23.1	Synthesis	
	Cyclone IV E	EP4CE115	Quartus Std 23.1	Hardware	DE2-115 Nios II
	Cyclone V	5CGTFD7	Quartus Std 23.1	Synthesis	
	Cyclone V SoC	5CSEMA5	Quartus Std 23.1	Synthesis	
	Cyclone 10 LP	10CL080	Quartus Std 23.1	Synthesis	
	Cyclone 10 GX	10CX105	Quartus Pro 23.1	Synthesis	
	MAX 10	10M40	Quartus Std 23.1	Synthesis	
	Stratix IV	EP4S40	Quartus Std 23.1	Synthesis	
	Stratix V	5SGSMD4	Quartus Std 23.1	Synthesis	
	Stratix 10	1SD21	Quartus Pro 23.1	Synthesis	

NOTE: Synthesis test means analysis, synthesis, fitter, and assembler. Hardware test means the design was operational using real hardware.

1.6 Release notes V4.0.0 (09/2025)

This version of the EtherCAT IP core has the following changes compared with the previous version.

Enhancements

- Physical layer
 - **Up to 4 ports (MII/RMII/RGMII) supported**
 - Separate PHY management interfaces for each port available
 - PHY register initialization available
 - MI link detection and configuration can be enabled per port
- Core basic functions
 - **Up to 16 FMMUs**
 - **Up to 16 SyncManagers**
 - SyncManager sequential mode
 - SyncManager deactivation delay
 - Distributed clocks
 - **Up to 4 distributed clocks SyncSignals**
 - **Up to 4 distributed clocks LatchSignals**
 - DC speed counter direct control register
 - Private PDI RAM (uses as shared memory for two PDIs)
- Core detail functions:
 - Extended RX error counters and RX error code registers
 - EEPROM I2C base address configurable for evaluation platforms
 - EEPROM I2C clock frequency configurable for evaluation platforms
- PDI basic functions
 - **Up to two PDI interfaces are supported**
 - **PDI performance up to 16 time faster:**
 - Internal bus width can be enhanced from 8 bit to 16/32 bit
 - Internal bus frequency can be enhanced from 25 MHz to 50/100 MHz
 - PDI SyncManager/interrupt acknowledge by write can be restricted to interrupts only
 - PDI user mode configuration
- PDI configuration
 - **AXI4 and AXI4-Lite on-chip bus is supported**
 - SPI slave supports now 1/2/4/8 MISO/MOSI signals, and bidirectional operation
 - SPI slave supports additional select signals for direct MISO/MOSI SyncManager access
 - μ Controller PDI supports now 8/16/32 bit data bus width (PDI0 only)
 - μ Controller PDI supports default busy indication
 - Digital output using OUT_START signal supported
- Integration
 - User RAM initialization can be disabled
 - IP improved for constraining
 - IP clock inputs are now directly connected in Qsys
 - Option to separately connect the DC SyncSignals as interrupts in Qsys

Features which cannot be disabled anymore:

- Manual/automatic TX shift for MII TX signals is always enabled. Connecting MII_TX_SHIFT and MII_TX_CLK signals to GND effectively disables the function.
- Distributed clocks receive times are always enabled
- PDI information register is always enabled

Discontinued features:

- Tri-state drivers inside the EtherCAT IP core are no longer supported
- AXI3 on-chip-bus is no longer supported
- Distributed clocks system time PDI-controlled is no longer supported

1.6.1 Reading IP core version from device

The IP core version, denoted as X.Y.Z (e.g., 4.0.0), consists of three values X, Y, and Z. These values can be read out in registers 0x0001 and 0x0002.

Table 4: Register revision (0x0001)

Bit	Description	ECAT	PDI	Reset Value
7:0	IP core major version X	r/-	r/-	IP core dep.

Table 5: Register build (0x0002:0x0003)

Bit	Description	ECAT	PDI	Reset Value
3:0	IP core maintenance version Z	r/-	r/-	IP core dep.
7:4	IP core minor version Y	r/-	r/-	IP core dep.
15:8	Patch level: 0x00: original release 0x01-0x0F: patch level of original release	r/-	r/-	IP core dep.

1.7 Design flow

The design flow for creating an EtherCAT slave controller based on the EtherCAT IP core is shown in the following picture:

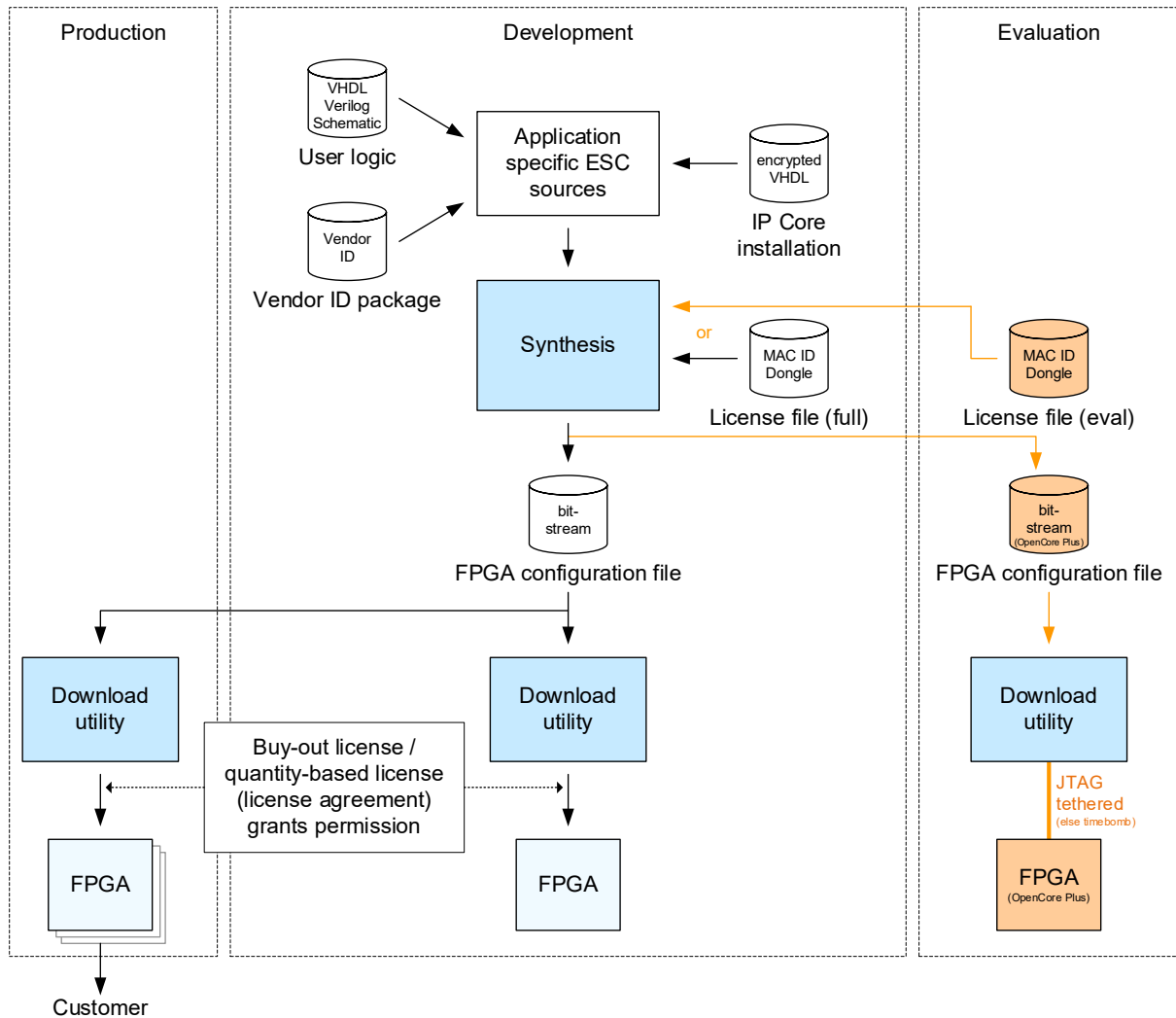


Figure 3: Design flow

1.8 Intel FPGA IP evaluation mode

The EtherCAT IP core for Altera FPGAs supports FPGA IP evaluation. A special license file with evaluation support is issued for each user, together with the IP core vendor ID package.

A design with an EtherCAT IP core using FPGA IP evaluation mode is subject to some restrictions:

- Only a time limited programming file (<design_name>_time_limited.sof) for the Altera Quartus Programmer is generated. Other programming files (e.g., .rbf, .pof) are not generated.
- For hardware testing, the ESC design has to be connected to the PC running the Altera Quartus Programmer using a programming adapter with a JTAG connection. The EtherCAT IP core is fully functional while the adapter is connected.
- If the connection is interrupted, the EtherCAT IP core will discontinue its function after approximately 1 hour.
- The evaluation version slightly increases the resource consumption of the IP core.
- The evaluation programming file must not be distributed/sold.

A vendor ID package is required for both evaluation and full license. It is recommended to use an evaluation vendor ID (package) for evaluation, and the original vendor ID for production. The evaluation vendor ID is beginning with "0xE....." and ends with the original vendor ID digits. Evaluation vendor IDs cannot pass the EtherCAT conformance tests.

Evaluation mode issues

Sometimes additional top-level pins appear in the evaluation mode design, these signals should be grounded externally if possible. This is a Quartus evaluation mode integration issue, not an EtherCAT IP core issue. The signals will not appear if a full license is used. Additionally, do not use incremental synthesis together with evaluation mode, since this was found to produce defective designs similar to the evaluation mode integration issue.

Sometimes timing requirements are not met with evaluation mode. Experience shows that timing violations related to the clock *altera_reserved_tck* can be ignored.

Upgrading to a full license

A design using an EtherCAT IP core in evaluation mode does not have to be changed when upgrading to a full license, only the full license file has to be installed instead of the evaluation mode license file. A re-generation of the EtherCAT IP core and a new synthesis run is necessary to generate the unlimited programming files.

1.9 Simulation

A behavioral simulation model of the EtherCAT IP core is not available because of its size and complexity. Thus, simulation of the entire EtherCAT IP core is not supported, and the EDA Netlist writer cannot be used for designs which contain the EtherCAT IP core. In most cases, simulation of the EtherCAT IP core is not necessary, as the IP core was thoroughly tested and the interfaces are standardized (Ethernet, Avalon) or simple and well described. Problems at the interface level can often be solved with a scope shot of the interface signals.

Nevertheless, customer designs using the Avalon or AXI on-chip bus can easily be simulated using a bus Functional model of the on-chip bus slave interface instead of a simulation model of the entire EtherCAT IP core.

From the processor's view, the EtherCAT IP core is a memory (or a bunch of registers). For processor bus verification, the EtherCAT IP core can be substituted by another IP core with Avalon/AXI slave interface which behaves like a memory as well. The EtherCAT IP core can be replaced for simulation by e.g.:

- Altera on-chip memory slave
- Avalon/AXI slave created with the Qsys

2 Features and registers

2.1 Features

Table 6: IP core feature details

Feature	IP core V4.0.0	IP core V3.0.10	Feature	IP core V4.0.0	IP core V3.0.10
EtherCAT ports	1-4	1-3	User MI initialization values	c	-
Permanent ports	1-4	1-3	MI controllable by PDI (0x0516:0x0517)	x	x
Optional bridge port 3 (EBUS or MII)	-	-	MI read error (0x0510[13])	x	x
EBUS ports	-	-	MI PHY configuration update status (0x0518[5])	x	x
MII ports	0-4	0-3	MI preamble suppression	x	x
RMII ports	0-4	0-2	Additional MCLK	x	x
RGMII ports	0-4	0-3	Gigabit PHY configuration	x	x
Port 0	x	x	Gigabit PHY register 9 detection	full	relaxed
Ports 0, 1	x	x	Clause 45 access	x	-
Ports 0, 1, 2	x	x	Transparent mode	-	-
Ports 0, 1, 3	-	-			
Ports 0, 1, 2, 3	x	-	MII features		
EtherCAT mode	Direct	Direct	CLK25OUT as PHY clock source	User logic	User logic
Slave category	Full slave	Full slave	Bootstrap TX shift settings	c	c
Position addressing	x	x	Automatic TX shift setting (with TX_CLK)	x	c
Node addressing	x	x	TX shift not necessary (PHY TX_CLK as clock source)	-	-
Logical addressing	x	x	FIFO size reduction steps	2	2
Broadcast addressing	x	x	MII back-to-back connections	x	x
Physical layer general features			PDI general features		
FIFO size configurable (0x0100[18:16])	x	x	Internal PDI clock	25/50/100 MHz	25 MHz
FIFO size default from SII EEPROM	x	x	Internal processing data bus width [bits]	8/16/32	8
Auto-forwarder checks CRC and SOF	x	x	Extended PDI configuration (0x0152:0x0153)	x	x
Forwarded RX error indication, detection and counter (0x0308:0x030B)	x	x	CPU_CLK output (10, 20, 25 MHz)	User logic	User logic
Prevention of circulating frames	x	x	CPU_CLK2 output and CPU_nRESET_OUT	-	-
Fallback: port 0 opens if all ports are closed	x	x	SOF, EOF, WD_TRIG and WD_STATE independent of PDI	x	x
VLAN tag and IP/UDP support	x	x	Available PDIs and PDI features depending on port configuration	-	-
Enhanced link detection per port configurable	x	x	PDI selection at run-time (SII EEPROM)	-	-
General Ethernet features (MII/RMII/RGMII)			PDI active immediately (SII EEPROM settings ignored)	x	x
PHY management interface (0x0510:0x051F)	c	c	PDI function acknowledge by write	c	c
Supported PHY address offsets	any	any	PDI function acknowledge SyncManager/register independently	c	-
Individual port PHY addresses	x	x	PDI information register 0x014E:0x014F	x	c
Port PHY addresses readable	x	x	Support for two PDIs	x	-
Link polarity configurable	User logic	User logic	Digital I/O PDI	PDI0, PDI1	x
Enhanced link detection supported	x	x	Digital I/O width [bits]	8/16/24/32	8/16/24/32
FX PHY support (native)	x	x			
Fast hot connect support (native)	x	x			
PHY reset out signals	x	x			
Link detection using PHY signal (LED)	x	x			
Link detection using RGMII in-band status	x	x			
MI link status and configuration	c per port	c all ports			

Feature	IP core V4.0.0	IP core V3.0.10
PDI control register value (0x0140:0x0141)	4	4
Control/status signals:	7	7
LATCH_IN	x	x
SOF	x	x
OUTVALID	x	x
WD_TRIG	x	x
OE_CONF	-	-
OE_EXT	x	x
EEPROM_Loaded	x	x
WD_STATE	x	x
EOF	x	x
OUT_START	x	-
Granularity of direction configuration [bits]	8	8
Bidirectional mode	- (User logic)	- (User logic)
Output high-Z if WD expired	User logic	User logic
Output 0 if WD expired	x	x
Output with EOF	x	x
Output with DC SyncSignals	x	x
Input with SOF	x	x
Input with DC SyncSignals	x	x
Digital I/O user mode from ECAT	x	-
Digital input register	PDI1	-
SPI slave PDI	PDI0, PDI1	x
Max. SPI clock [MHz]	30	30
SPI modes configurable (0x0150[1:0])	x	x
SPI_IRQ driver configurable (0x0150[3:2])	x	x
SPI_SEL polarity configurable (0x0150[4])	x	x
Data out sample mode configurable (0x0150[5])	x	x
Busy signaling	-	-
Wait state byte(s)	x	x
Number of address extension byte(s)	any	any
2/4 byte SPI master support	x	x
Extended error detection (read busy violation)	x	x
IRQ byte 0/1 swap	c	-
SPI user mode from PDI	c	-
SPI_IRQ delay	x	x
Status indication	x	x
EEPROM_Loaded signal	x	x
Additional select signals	c	-
SPI data width	1/2/4/8	1
SPI data bidirectional	c	-
SPI master PDI	-	-
Asynchronous µController PDI	PDI0: 8/16 bit	8/16 bit
Extended µC configuration bits 0x0150[7:4], 0x0152:0x0153	x	x
ADR[15:13] available (000s if not available)	x	x

Feature	IP core V4.0.0	IP core V3.0.10
EEPROM_Loaded signal	x	x
RD polarity configurable (0x0150[7])	-	-
Read BUSY delay (0x0152[0])	x	x
Write after first edge (0x0152[2])	c	c
Default BUSY state	c	-
ET1100 compatible timing	-	-
Synchronous µController PDI	-	-
Multiplexed asynchronous µController PDI	-	-
On-chip bus PDI	x	x
Avalon®	- ET1810: x	x
OPB	-	-
PLB v4.6	-	-
AXI3	-	x
AXI4	x	-
AXI4-Lite	x	-
Bus clock [MHz] (N=1,2,3,...)	any	any
Data bus width [bits]	8/16/32/64	8/16/32/64
Prefetch cycles	1	1
DC SyncSignals available directly and as IRQ	x	x
Bus clock multiplier in register 0x0150[4:0]	x	x
EEPROM_Loaded signal	x	x
EtherCAT bridge (port 3, EBUS/MII)	-	-
General purpose I/O	x	x
GPO bits	0/8/16/32/64	0/8/16/32/64
GPI bits	0/8/16/32/64	0/8/16/32/64
GPIO available independent of PDI or port configuration	x	x
GPIO available without PDI	x	x
Concurrent access to GPO by ECAT and PDI	x	x
Bidirectional GPIO bits	x	-
ESC information		
Basic information (0x0000:0x0006)	x	x
Port descriptor (0x0007)	x	x
ESC features supported (0x0008:0x0009)	x	x
Extended ESC feature availability in user RAM (0x0F80 ff.)	x	x
Write protection (0x0020:0x0031)	c	c
Data link layer features		
ECAT reset (0x0040)	c	c
PDI reset (0x0041)	c	c
ESC DL control (0x0100:0x0103) bytes	4	4
EtherCAT only mode (0x0100[0])	x	x
Temporary loop control (0x0100[1])	x	x
FIFO size configurable (0x0100[18:16])	x	x

Feature	IP core V4.0.0	IP core V3.0.10
Configured station address (0x0010:0x0011)	x	x
Configured station alias (0x0100[24], 0x0012:0x0013)	x	x
Physical read/write offset (0x0108:0x0109)	c	c
Application layer features		
Extended AL control/status bits (0x0120[15:5], 0x0130[15:5])	x	x
AL status emulation (0x0140[8])	x	x
AL status code (0x0134:0x0135)	c	c
Interrupts		
ECAT event mask (0x0200:0x0201)	x	x
AL event mask (0x0204:0x0207)	c	c
ECAT event request (0x0210:0x0211)	x	x
AL event request (0x0220:0x0223)	x	x
SyncManager activation changed (0x0220[4])	x	x
SyncManager watchdog expiration (0x0220[6])	x	x
DC SYNC interrupt	[3:0]	[1:0]
Error counters		
RX error counter (0x0300:0x0307)	x	x
Forwarded RX error counter (0x0308:0x030B)	x	x
ECAT processing unit error counter (0x030C)	x	c
PDI error counter (0x030D)	x	c
PDI error code (0x030E:0x030F)	x	c
Lost link counter (0x0310:0x0313)	x	c
Extended RX error counters (0x0314:0x0317)	c	-
RX error code (0x0320:0x0327)	c	-
PDI clearing error counters	-	-
Watchdog		
Watchdog divider configurable (0x0400:0x0401)	c	c
Watchdog process data	x	x
Watchdog PDI	x	x
Watchdog counter process data (0x0442)	x	x
Watchdog counter PDI (0x0443)	x	x
Watchdog status (0x0448)	x	-
SII EEPROM interface (0x0500:0x050F)		
I ² C EEPROM sizes supported	1 Kbit-4 Mbit	1 Kbit-4 Mbit
I ² C EEPROM size reflected in 0x0502[7]	x	x
EEPROM controllable by PDI	x	x
EEPROM emulation by PDI	c	c

Feature	IP core V4.0.0	IP core V3.0.10
EEPROM emulation CRC error 0x0502[11] PDI writable	x	x
Read data bytes (0x0502[6])	4	4
Internal pull-up resistors for EEPROM_CLK and EEPROM_DATA	User logic	User logic
ESC configuration area A	x	x
ESC configuration area B	x	-
I2C base address	0-7	0
I2C half speed	c	-
FMMUs	0-16	0-8
Bit-oriented operation	x	x
SyncManagers	0-16	0-8
Watchdog trigger generation for 1 byte Mailbox configuration independent of reading access	x	x
SyncManager event times (+0x8[7:6])	c	c
Buffer state (+0x5[7:6])	x	x
SyncManager sequential mode	x	-
SyncManager deactivation delay	x	-
Distributed clocks	c	c
Width	32/64	32/64
Sync/Latch signals	0-8 (0-4 Sync-Signals, 0-4 Latch-Signals)	4 (0-2 Sync-Signals, 0-2 Latch-Signals)
SyncManager event times (0x09F0:0x09FF)	c	c
DC receive times	x	c
DC time loop control controllable by PDI	-	c
DC sync/latch activation (0x0140[11:10])	-	-
DC time loop activation (0x0142[8])	-	-
Propagation delay measurement with traffic (BWR/FPWR 0x900 detected at each port)	x	x
LatchSignal state in latch status register (0x09AE:0x09AF)	x	x
SyncSignal auto-activation (0x0981[3])	x	x
SyncSignal 32 or 64 bit start time (0x0981[4])	x	x
SyncSignal late activation (0x0981[6:5])	x	x
SyncSignal debug pulse (0x0981[7])	x	x
SyncSignal activation state 0x0984	x	x
Reset filters after writing filter depth	x	x
Speed counter diff direct control (0x0938:0x0939)	x	-
ESC specific registers (0x0E00:0x0EFF)		
Product and vendor ID	x	x
POR values	-	-
FPGA update	-	-
Process RAM and user RAM		

Feature	IP core V4.0.0	IP core V3.0.10
Process RAM (0x1000 ff.) [Kbyte]	0-60	0-60
User RAM (0x0F80:0x0FFF)	x	x
User RAM ESC feature initialization	c	x
RAM BIST	-	-
Additional EEPROMs	1-2	1-2
SII EEPROM (I ² C)	c (EEPROM of μ C used)	c (EEPROM of μ C used)
FPGA configuration EEPROM	x	x
LED signals		
RUN LED	c	c
RUN LED override	c	c
Link/activity(x) LED per port	x	x
PERR(x) LED per port	-	-
Device ERR LED	c	c
STATE_RUN LED	c	c
PDI_ERR LED per PDI	-	-
Optional LED states		
RUN LED: bootstrap	x	x
RUN LED: booting	c	c
RUN LED: device identification	c	c
RUN LED: loading SII EEPROM	c	c
RUN LED: POR values signaling	-	-
Error LED: SII EEPROM loading error	c	c
Error LED: invalid hardware configuration	-	-
Error LED: process data watchdog timeout	c	c
Error LED: PDI watchdog timeout	c	c
Error LED: BIST error	-	-
Error LED: error indication 0x0130[4]	c	c
Link/activity: port closed	-	-
Link/activity: local auto-negotiation error	-	-
Link/activity: remote auto-negotiation error	-	-

Feature	IP core V4.0.0	IP core V3.0.10
Link/activity: unknown PHY auto-negotiation error	-	-
LED test	c	c
Clock supply		
Crystal	-	-
Crystal oscillator	x	x
TX_CLK from PHY	x	x
25ppm clock source accuracy	x	x
Internal PLL	User logic	User logic
Power supply voltages	FPGA dep.	FPGA dep.
I/O voltages	FPGA dep.	FPGA dep.
1.8 V	FPGA dep.	FPGA dep.
2.5 V	FPGA dep.	FPGA dep.
3.3 V	FPGA dep.	FPGA dep.
5 V	FPGA dep.	FPGA dep.
Separate I/O voltages	FPGA dep.	FPGA dep.
Core voltage	FPGA dep.	FPGA dep.
Internal regulators	-	-
Input voltage		
Output core Voltage		
Output I/O Voltage		
JTAG	-	-
Package	FPGA dep.	FPGA dep.
Size [mm ²]	FPGA dep.	FPGA dep.
Original release date	09/2025	1/2015
Configuration and pinout calculator	-	-
Register configuration	individual	individual
Evaluation version	x	x
Vendor ID package	x	x
Internal tri-state drivers	-	c
IP core configuration tool	x ET1810: -	-
XPS IP cores	-	-
Vivado IP cores	ET1815: x	-
Propel IP cores	ET1825: x	-
Example designs/ pre-synthesized time-limited evaluation core included in setup	separate/0	4/4

Table 7: ESC feature legend

Symbol	Description
x	available
-	not available
c	configurable
User logic	Functionality can be added by user logic inside the FPGA
POR	Value can be changed by power-on reset values (strapping)
PROM	Value can be changed by SII EEPROM content
OTP	Value can be changed by OTP content
red	Feature changed in this version

2.2 Registers

An EtherCAT slave controller (ESC) has an address space of 64Kbyte. The first block of 4Kbyte (0x0000:0x0FFF) is dedicated for registers. The process data RAM starts at address 0x1000, its size is configurable.

Some registers are implemented depending on the configuration.

Table 8 gives an overview of the available registers.

Table 8: Register availability

Address	Length (byte)	Description	IP core V4.0.0	IP core V3.0.0-V3.0.10	IP core option
0x0000	1	Type	x	x	
0x0001	1	Revision	x	x	
0x0002:0x0003	2	Build	x	x	
0x0004	1	FMMUs supported	x	x	
0x0005	1	SyncManagers supported	x	x	
0x0006	1	RAM size	x	x	
0x0007	1	Port descriptor	x	x	
0x0008:0x0009	2	ESC features supported	x	x	
0x0010:0x0011	2	Configured station address	x	x	
0x0012:0x0013	2	Configured station alias	x	x	
0x0020	1	Write register enable	c	c	<i>Write protection</i>
0x0021	1	Write register protection	c	c	<i>Write protection</i>
0x0030	1	ESC write enable	c	c	<i>Write protection</i>
0x0031	1	ESC write protection	c	c	<i>Write protection</i>
0x0040	1	ESC reset ECAT	c	c	<i>Reset slave by ECAT/PDI</i>
0x0041	1	ESC reset PDI	c	c	<i>Reset slave by ECAT/PDI</i>
0x0100:0x0101	2	ESC DL control	x	x	
0x0102:0x0103	2	Extended ESC DL control	x	x	
0x0108:0x0109	2	Physical read/write offset	c	c	<i>Read/write offset</i>
0x0110:0x0111	2	ESC DL status	x	x	
0x0120	5 bits [4:0]	AL control	x	x	
0x0120:0x0121	2	AL control	x	x	
0x0130	5 bits [4:0]	AL status	x	x	
0x0130:0x0131	2	AL status	x	x	
0x0134:0x0135	2	AL status Code	c	c	<i>AL status code register</i>
0x0138	1	RUN LED override	c	c	<i>Extended RUN/ERR LED</i>
0x0139	1	ERR LED override	c	c	<i>Extended RUN/ERR LED</i>
0x0140	1	PDI0 control	x	x	
0x0141	1	ESC configuration A0	x	x	
0x0142:0x0143	2	ESC configuration A5	-	-	
0x0144:0x0145	2	ESC configuration A6	-	-	
0x014E:0x014F	2	PDI0 information	x	c	<i>PDI information register</i>
0x0150	1	PDI0 configuration	x	x	

Address	Length (byte)	Description	IP core V4.0.0	IP core V3.0.0-V3.0.10	IP core option
0x0151	1	DC sync/latch configuration	x	x	
0x0152:0x0153	2	Extended PDI0 configuration	x	x	
0x0158:0x0159	2	PDI0 user mode from ECAT	c	-	<i>PDI usermode</i>
0x015C:0x015D	2	PDI0 user mode from PDI	c	-	<i>PDI usermode</i>
0x0180	1	PDI1 control	c	-	<i>Enable PDI1</i>
0x0181	1	ESC configuration B0	c	-	<i>Enable PDI1</i>
0x0182:0x0183	2	ESC configuration B5	-	-	
0x0188:0x0189	2	ESC configuration B4	-	-	
0x018E:0x018F	2	PDI1 information	c	-	<i>Enable PDI1</i>
0x0190	1	PDI1 configuration	c	-	<i>Enable PDI1</i>
0x0191	1	Reserved	c	-	<i>Enable PDI1</i>
0x0192:0x0193	2	Extended PDI1 configuration	c	-	<i>Enable PDI1</i>
0x0198:0x0199	2	PDI1 user mode from ECAT	c	-	<i>PDI usermode</i>
0x019C:0x019D	2	PDI1 user mode from PDI	c	-	<i>PDI usermode</i>
0x0200:0x0201	2	ECAT event mask	x	x	
0x0204:0x0207	4	PDI0 AL event mask	r/c	r/c	<i>AL event mask register</i>
0x020A:0x020D	4	PDI1 AL event mask	c	-	<i>AL event mask register and enable PDI1</i>
0x0210:0x0211	2	ECAT event request	x	x	
0x0220:0x0223	4	AL event request	x	x	
0x0300:0x0307	4x2	RX error counter[3:0]	x	x	
0x0308:0x030B	4x1	Forwarded RX error counter[3:0]	x	x	
0x030C	1	ECAT processing unit error counter	x	c	<i>EPU and PDI error counter</i>
0x030D	1	PDI0 error counter	x	c	<i>EPU and PDI error counter</i>
0x030E:0x030F	2	PDI0 error code	x	c	<i>EPU and PDI error counter</i>
0x0310:0x0313	4x1	Lost link counter[3:0]	x	c	<i>Lost link counter</i>
0x0314:0x0317	4x1	Extended RX error counter[3:0]	c	-	<i>Extended RX error counter & code</i>
0x0320:0x0327	4x2	RX error code[3:0]	c	-	<i>Extended RX error counter & code</i>
0x0340	1	PDI1 error counter	c	-	<i>Enable PDI1</i>
0x0341	1	PDI1 error code	c	-	<i>Enable PDI1</i>
0x0400:0x0401	2	Watchdog divider	r/c	r/c	<i>Extended watchdog</i>
0x0410:0x0411	2	Watchdog time PDI0	c	c	<i>Extended watchdog</i>
0x0412:0x0413	2	Watchdog time PDI1	c	-	<i>Enable PDI1</i>
0x0420:0x0421	2	Watchdog time process data	x	x	
0x0440:0x0441	2	Watchdog status process data	x	x	
0x0442	1	Watchdog counter process data	c	c	<i>Watchdog counter</i>
0x0443	1	Watchdog counter PDI0	c	c	<i>Watchdog counter</i>

Address	Length (byte)	Description	IP core V4.0.0	IP core V3.0.0-V3.0.10	IP core option
0x0444	1	Watchdog counter PDI1	c	-	Watchdog counter and enable PDI1
0x0448	1	Watchdog status PDI	x	-	
0x0500:0x050F	16	SII EEPROM interface	x	x	
0x0510:0x0515	6	PHY management interface	c	c	PHY management interface
0x0516:0x0517	2	PHY management access state	c	c	PHY management interface
0x0518:0x051B	4	PHY port status[3:0]	c	c	MI link detection and configuration
0x0600:0x06FC	16x13	FMMU[15:0]	0-16	0-8	Number of FMMU
0x0800:0x087F	16x8	SyncManager[15:0]	0-16	0-8	Number of SyncManagers
0x0900:0x090F	4x4	DC – receive times	x	RT	Refer to legend
0x0910:0x0917	8	DC – system time	DC	DC	Refer to legend
0x0918:0x091F	8	DC – receive time EPU	DC	DC	Refer to legend
0x0920:0x0927	8	DC – system time offset	DC	DC	Refer to legend
0x0928:0x092B	4	DC – system time delay	DC	DC	Refer to legend
0x092C:0x092F	4	DC – system time difference	DC	DC	Refer to legend
0x0930:0x0931	2	DC – speed counter start	DC	DC	Refer to legend
0x0932:0x0933	2	DC – speed counter diff	DC	DC	Refer to legend
0x0934	1	DC – system time difference filter depth	DC	DC	Refer to legend
0x0935	1	DC – speed counter filter depth	DC	DC	Refer to legend
0x0936	1	DC – receive time latch mode	-	-	
0x0938:0x0939	2	DC – speed counter diff direct control	c	-	DC speed counter direct control
0x0940:0x0943	4	DC – SYNC2 cycle time	S2	-	Refer to legend
0x0944:0x0947	4	DC – SYNC3 cycle time	S3	-	Refer to legend
0x0950:0x0957	8	DC – next SYNC2 pulse	S2	-	Refer to legend
0x0958:0x095F	8	DC – next SYNC3 pulse	S3	-	Refer to legend
0x0980	1	DC – cyclic unit control	SL	SL	Refer to legend
0x0981	1	DC – activation	S0	S0	Refer to legend
0x0982:0x0983	2	DC – pulse length of SyncSignals	S0	S0	Refer to legend
0x0984	1	DC – activation status	S0	S0	Refer to legend
0x0986	1	DC – activation SYNC2/3	S2	-	Refer to legend
0x098C	1	DC – SYNC2 status	S2	-	Refer to legend
0x098D	1	DC – SYNC3 status	S3	-	Refer to legend
0x098E	1	DC – SYNC0 status	S0	S0	Refer to legend
0x098F	1	DC – SYNC1 status	S1	S1	Refer to legend
0x0990:0x0997	8	DC – start time cyclic operation/next SYNC0 pulse	S0	S0	Refer to legend
0x0998:0x099F	8	DC – next SYNC1 pulse	S1	S1	Refer to legend
0x09A0:0x09A3	4	DC – SYNC0 cycle time	S0	S0	Refer to legend
0x09A4:0x09A7	4	DC – SYNC1 cycle time	S1	S1	Refer to legend

Address	Length (byte)	Description	IP core V4.0.0	IP core V3.0.0-V3.0.10	IP core option
0x09A8	1	DC – latch0 control	L0	L0	Refer to legend
0x09A9	1	DC – latch1 control	L1	L1	Refer to legend
0x09AA	1	DC – latch2 control	L2	-	Refer to legend
0x09AB	1	DC – latch3 control	L3	-	Refer to legend
0x09AC	1	DC – latch2 status	L2	-	Refer to legend
0x09AD	1	DC – latch3 status	L3	-	Refer to legend
0x09AE	1	DC – latch0 status	L0	L0	Refer to legend
0x09AF	1	DC – latch1 status	L1	L1	Refer to legend
0x09B0:0x09B7	8	DC – latch0 positive edge	L0	L0	Refer to legend
0x09B8:0x09BF	8	DC – latch0 negative edge	L0	L0	Refer to legend
0x09C0:0x09C7	8	DC – latch1 positive edge	L1	L1	Refer to legend
0x09C8:0x09CF	8	DC – latch1 negative edge	L1	L1	Refer to legend
0x09D0:0x09D7	8	DC – latch2 positive edge	L2	-	Refer to legend
0x09D8:0x09DF	8	DC – latch2 negative edge	L2	-	Refer to legend
0x09E0:0x09E7	8	DC – latch3 positive edge	L3	-	Refer to legend
0x09E8:0x09EF	8	DC – latch3 negative edge	L3	-	Refer to legend
0x09F0:0x09F3	4	DC – EtherCAT buffer change event time	ET	ET	Refer to legend
0x09F8:0x09FB	4	DC – PDI buffer start event time	ET	ET	Refer to legend
0x09FC:0x09FF	4	DC – PDI buffer change event time	ET	ET	Refer to legend
0x0E00:0x0E07	8	Power-on values [bits]	-	-	
0x0E00:0x0E07	8	Product ID	x	x	
0x0E08:0x0E0F	8	Vendor ID	x	x	
0x0E10:0x0E17	8	ESC health status	c	-	Evaluation version, except ET1810
0x0F00:0x0F03	4	Digital I/O output data	io	io	Selected PDI (PDI0/PDI1)=Digital IO
0x0F08:0x0F0B	4	Digital I/O input data	c	-	Enable PDI1 and Selected PDI (PDI1)=Digital IO
0x0F10:0x0F17	8	General purpose outputs [byte]	0-8	0-8	Number of GPIOs
0x0F18:0x0F1F	8	General purpose inputs [byte]	0-8	0-8	Number of GPIOs
0x0F80:0x0FFF	128	User RAM	x	x	
0x1000:0x1003	4	Digital I/O input data	io	io	Selected PDI (PDI0)=Digital IO and Process data RAM>0
0x1000 ff.		Process data RAM [Kbyte]	0-60	0-60	Process data RAM

Table 9: ESC register legend

Symbol	Description	Option IP core V4.0.0	Option IP core V3.0.0-V3.0.10
x	Available		
-	Not available		
r	Read only		
c	Configurable		
RT	DC receive times enabled	-	Receive times
DC	DC time loop control enabled	Distributed clocks	Distributed clocks
SL	DC sync unit and/or latch unit enabled	DC SyncSignals>0, or DC LatchSignals>0	DC SyncSignals>0, or DC LatchSignals>0
S0	DC SyncSignal 0 enabled	DC SyncSignals>0	DC SyncSignals>0
S1	DC SyncSignal 1 enabled	DC SyncSignals>1	DC SyncSignals>1
S2	DC SyncSignal 2 enabled	DC SyncSignals>2	
S3	DC SyncSignal 3 enabled	DC SyncSignals>3	
L0	DC LatchSignal 0 enabled	DC LatchSignals>0	DC LatchSignals>0
L1	DC LatchSignal 1 enabled	DC LatchSignals>1	DC LatchSignals>1
L2	DC LatchSignal 2 enabled	DC LatchSignals>2	
L3	DC LatchSignal 3 enabled	DC LatchSignals>3	
ET	DC SyncManager event times are enabled	SyncManager event times	SyncManager event times
io	Available if digital I/O PDI is selected		
red	Register changed in this version		

2.3 ESC features (power-on values of user RAM)

Table 10: ESC features (reset values of user RAM – 0x0F80:0x0FFF)

Addr.	Bit	Feat.	Description	Reset value IP core
			ESC extended features:	Depends on ESC: 0: Not available 1: Available c: Configurable
0F81	0	0	Extended DL control register (0x0102:0x0103)	1
	1	1	AL status code register (0x0134:0x0135)	c
	2	2	ECAT interrupt mask (0x0200:0x0201)	1
	3	3	Configured station alias (0x0012:0x0013)	1
	4	4	General purpose inputs (0x0F18:0x0F1F)	c
	5	5	General purpose outputs (0x0F10:0x0F17)	c
	6	6	AL event mask writable (0x0204:0x0207)	c
	7	7	Physical read/write offset (0x0108:0x0109)	c
0F82	0	8	Watchdog divider writable (0x0400:0x0401) and Watchdog PDI (0x0410:0x0411)	c
	1	9	Watchdog counters (0x0442:0x0443)	c
	2	10	Write protection (0x0020:0x0031)	c
	3	11	Reset (0x0040:0x0041)	c
	4	12	Reserved	0
	5	13	DC SyncManager event times (0x09F0:0x09FF)	c
	6	14	ECAT processing unit/PDI error counter (0x030C:0x030D)	1
	7	15	EEPROM size configurable (0x0502[7]): 0: EEPROM size fixed to sizes up to 16 Kbit 1: EEPROM size configurable	1
0F83	0	16	EEPROM control by PDI possible	1
	1	17	Reserved	0
	2	18	Reserved	0
	3	19	Lost link counter (0x0310:0x0313)	1
	4	20	PHY management interface (0x0510:0x0515)	c
	5	21	Enhanced link detection MII	c
	6	22	Enhanced link detection EBUS	0
	7	23	Run LED	c
0F84	0	24	Link/Activity LED	1
	1	25	Reserved	0
	2	26	Reserved	1
	3	27	DC latch in unit	c
	4	28	Reserved	0
	5	29	DC Sync unit	c
	6	30	DC time loop control assigned to PDI	c
	7	31	Link detection and configuration by MI	c

Addr.	Bit	Feat.	Description	Reset value IP core
0F85	0	32	MI control by PDI possible	1
	1	33	Automatic TX shift	c
	2	34	EEPROM emulation by µController	c
	3	35	Reserved	0
	4	36	Reserved	0
	5	37	Disable digital I/O register (0x0F00:0x0F03)	c
	6	38	Reserved	0
	7	39	Reserved	0
0F86	0	40	PDI user mode (0x0158, 0x015C)	c
	1	41	Extended RX error counter (0x0314:0x0317)	c
	2	42	RUN/ERR LED override (0x0138:0x0139)	c
	3	43	Reserved	0
	4	44	Reserved	1
	5	45	Reserved	0
	6	46	Reserved	0
	7	47	Reserved	0
0F87	0	48	Reserved	0
	1	49	Reserved	0
	2	50	Reserved	0
	3	51	DC SYNC1 disable	c
	4	52	Reserved	0
	5	53	Reserved	0
	6	54	DC receive times (0x0900:0x090F)	1
	7	55	DC system time (0x0910:0x0936)	c
0F88	0	56	DC 64 bit	c
	1	57	Reserved	0
	2	58	PDI clears error counter	0
	3	59	Avalon PDI	c
	4	60	OPB PDI	0
	5	61	PLB PDI	0
	6	62	Reserved	0
	7	63	Reserved	0
0F89	0	64	Reserved	0
	1	65	Reserved	0
	2	66	Reserved	0
	3	67	Reserved	0
	4	68	Reserved	0
	5	69	Reserved	0
	6	70	Reserved	0
	7	71	Direct RESET	0

Addr.	Bit	Feat.	Description	Reset value IP core
0F8A	0	72	Reserved	0
	1	73	Reserved	1
	2	74	DC LATCH1 disable	c
	3	75	AXI PDI	c
	4	76	Reserved	0
	5	77	Reserved	0
	6	78	PDI function acknowledge by PDI write	c
	7	79	PDI information register (0x014E:0x014F, 0x018E:0x018F)	1
0F8B	0	80	Reserved	1
	1	81	Reserved	1
	2	82	Reserved	0
	3	83	LED test	c
	4	84	Reserved	0
	5	85	Reserved	0
	6	86	Reserved	0
	7	87	Reserved	0
0F8C	3:0	91:88	Reserved	0
	7:4	95:92	Reserved	0
0F8D	3:0	99:96	Reserved	0
	7:4	103:100	Reserved	0
0F8E	3:0	107:104	Reserved	0
	4	108	Reserved	0
	5	109	Reserved	0
	7:6			
0F8F	0	112:110	Digital I/O PDI byte size	c
	1	113	Reserved	0
	2	114	Digital I/O PDI	c
	3	115	SPI slave PDI	c
	4	116	Asynchronous µC PDI	c
	5	117	Reserved	0
	6	118	Reserved	1
	7	119	Reserved	1

Addr.	Bit	Feat.	Description	Reset value IP core
0F90	0	120	Reserved	0
	1	121	PDI 1	c
	2	122	Digital I/O input register (0x0F08:0x0F0B)	c
	3	123	Reserved	c
	4	124	Reserved	0
	5	125	DC LATCH2	c
	6	126	DC LATCH3	c
	7	127	DC SYNC2	c
0F91	0	128	DC SYNC3	c
	1	129	Reserved	0
	2	130	Reserved	0
	3	131	Reserved	0
	4	132	Reserved	0
	5	133	Reserved	0
	6	134	Reserved	0
	7	135	Reserved	0
0F92	0	136	Reserved	0
	1	137	Reserved	0
	2	138	Reserved	0
	3	139	PDI watchdog status register (0x0448)	1
	4	140	Reserved	1
	5	141	Reserved	0
	6	142	Reserved	0
	7	143	Reserved	0
0F93	0	144	RGMII	0
	1	145	Individual PHY address read out (0x0510[7:3])	c
	2	146	CLK_PDI_EXT is asynchronous	c
	3	147	CLK_PDI1_EXT is asynchronous	c
	4	148	Use RGMII GTX_CLK phase shifted clock input	1
	5	149	RMII	0
	6	150	Security CPLD protection	0
	7			
0F94	1:0	153:151	EEPROM I2C address offset	c
	5:2	157:154	Reserved	0
	6	158	Reserved	0
	7	159	Reserved	0

Addr.	Bit	Feat.	Description	Reset value IP core
0F95	0	160	Reserved	0
	1	161	Reserved	0
	4:2	164:162	Digital I/O PDI1 byte size	c
	5	165	Reserved	0
	6	166	Reserved	0
	7	167	Reserved	0
0F96	0	168	Digital I/O PDI1	c
	1	169	SPI slave PDI1	c
	2	170	Reserved	0
	3	171	Reserved	0
	4	172	Reserved	0
	5	173	Reserved	0
	6	174	Reserved	0
	7	175	Reserved	0
0F97	0	176	Avalon PDI1	c
	1	177	Reserved	0
	2	178	Reserved	0
	3	179	AXI PDI1	c
	4	180	Reserved	0
	5	181	Reserved	0
	6	182	Reserved	0
	7	183	Reserved	0
0F98	0	184	Reserved	0
	1	185	PDI private RAM	c
	2	186	Reserved	0
	3	187	PHY initialization user values	c
	4	188	Reserved	0
	5	189	Reserved	0
	7:6	191:190	Reserved	0

Addr.	Bit	Feat.	Description	Reset value IP core
0F99	2:0	194:192	Reserved	0
	3	195	Reserved	0
	4	196	Reserved	0
	5	197	Reserved	0
	6	198	SyncManager deactivation delay	1
	7	199	SyncManager sequential mode	c
0F9A	0	200	Reserved	c
	1	201	Reserved	0
	2	202	Reserved	0
	3	203	ESC configuration area B supported	c
	4	204	Reserved	0
	5	205	Reserved	0
	6	206	Disable RAM initialization	c
0F9B	7	207	Reserved	0
	0	208	Reserved	0
	1	209	Reserved	0
	2	210	Reserved	0
	3	211	Reserved	0
	4	212	Reserved	0
	5	213	Reserved	0
0F9C	6	214	Reserved	0
	7	219:215	Number of FMMUs	c
0F9D	3:0	224:220	Number of SyncManagers	c
	7:4			
	0	225	Reserved	0
	1	226	Reserved	0
	2	227	Enable additional SPI slave features PDI0	c
	3	228	Enable additional SPI slave features PDI1	c
	4	229	Reserved	0
	5	230	Reserved	0
	6	231	Reserved	0
0F9E	7	232	Reserved	0
	0	233	Use half I2C speed	c
	1	234	Reserved	0
	2	235	Reserved	0
	3	236	Reserved	0
	4	237	Reserved	0
	5	238	Reserved	0
0F9F	6	239	Reserved	1
	0	240	Separate MI interface for each port	c
	1	241	Reserved	0
	2	242	Reserved	0
	3	243	Reserved	0
	4	244	Reserved	0
	5	245	Reserved	0
0FA0	6	246	Reserved	0
	7	247	Reserved	0
0FA0	0	248	Reserved	0

Addr.	Bit	Feat.	Description	Reset value IP core
	1	249	Reserved	0
	2	250	Reserved	0
	3	251	Reserved	0
	4	252	Reserved	0
	5	253	Reserved	0
	6	254	Reserved	0
	7	255	Reserved	0
0FDF: 0FA1			Reserved	0

2.4 ESC port features (power-on values of user RAM)

Addr.	Bit	Feat.	Description	Reset value IP core
			Port features for port 0-3 (64 bit each)	
0FE0/ 0FE8/ 0FF0/ 0FF8	0	0	Reserved	0
	1	1	MII supported	c
	2	2	RMII supported	c
	3	3	RGMII supported	c
	4	4	Reserved	0
	5	5	FX supported	c
	6	6	Reserved	0
	7	7	Reserved	0
0FE1/ 0FE9/ 0FF1/ 0FF9	0	8	EtherCAT 100 Mbps supported	1
	1	9	Reserved	0
	2	10	MI link detection and configuration supported	c
	3	11	Reserved	0
	4	12	Enhanced link detection supported	c
	5	13	Reserved	0
	6	14	Reserved	0
	7	15	Reserved	0
0FE2/ 0FEA/ 0FF2/ 0FFA	0	16	Reserved	0
	1	17	Reserved	0
	2	18	Reserved	0
	3	19	Reserved	0
	4	20	Reserved	0
	5	21	Reserved	0
	6	22	Reserved	0
	7	23	Reserved	0
0FE3/ 0FEB/ 0FF3/ 0FFB	0	24	Reserved	0
	1	25	Reserved	0
	2	26	Reserved	0
	3	27	Reserved	0
	4	28	Reserved	0
	5	29	Reserved	0
	6	30	Reserved	0
	7	31	Reserved	0

Addr.	Bit	Feat.	Description	Reset value IP core
0FE4: 0FE7/ 0FEC: 0FEF/ 0FF4: 0FF7/ 0FFC: 0FFF			Reserved	0

3 IP core installation

3.1 System requirements

The system requirements of the Altera design tools are applicable.

3.2 EtherCAT IP core installation

3.2.1 EtherCAT IP core ZIP file

The EtherCAT IP core ZIP file

“EtherCAT IP core for Altera FPGAs <version>.zip”

is extracted to any place on your disk (referenced as *<IPInst_dir>*). The ZIP file has the following content:

Table 11: EtherCAT IP core ZIP content

Folder/file name	Description
/ethercat_altera.ecwxml	XML description of EtherCAT IP
/doc/	Documentation
/ethercat_ip/	EtherCAT IP for Quartus: Add the content of this folder to your Quartus project, or add this folder to the Quartus IP search path
/ethercat_ip/BECKHOFF_EtherCAT_<version>.ipx	IP description for Quartus
/ethercat_ip/beckhoff_ethercat_<version>/core/ETHERCAT_IPCORE.VHD	Encrypted EtherCAT IP core source code
/ethercat_ip/beckhoff_ethercat_<version>/lib/beckhoff.jpg	IP catalog image
/ethercat_ip/beckhoff_ethercat_<version>/lib/ethercat.qprs	IP configuration presets
/ethercat_ip/beckhoff_ethercat_<version>/lib/ethercat_hw.tcl	IP configuration TCL
/ethercat_ip/beckhoff_ethercat_<version>/lib/ETHERCAT_IPCORE_V2.ocp	IP evaluation description
/ethercat_ip/beckhoff_ethercat_<version>/vendor_id/	Folder for your vendor ID package
/example/constraints/EtherCAT_settings.sdc	EtherCAT IP constraint configuration example
/example/constraints/EtherCAT_timing.sdc	EtherCAT IP constraints example
/example/init/ETHERCAT_IPCORE_PHY_INIT.VHD	PHY initialization ROM example

3.2.2 Vendor ID package

The vendor ID is assigned by the ETG, it allows identification of the origin of an EtherCAT slave controller. The vendor ID can be read out in register 0x0E08:0x0E0B.

A vendor ID is required for both evaluation and full license. It is recommended to use an evaluation vendor ID (package) for evaluation, and the original vendor ID for production. The evaluation vendor ID is beginning with “0xE.....” and ends with the original vendor ID digits. Evaluation vendor IDs cannot pass the EtherCAT conformance tests.

The vendor ID package is required by the EtherCAT IP core source code, and it contains your company’s unique vendor ID. The vendor ID package is not part of the IP ZIP file, it is delivered separately.

Copy your vendor ID package (*pk_ECAT_VENDORID_<company>_Altera.vhd* or similar) into the *vendor_id* folder of the IP core installation directory:

<IPInst_dir>/ethercat_ip/beckhoff_ethercat_<version>/vendor_id/

3.2.3 License file

For information regarding license setup, refer to the “FPGA Licensing Support Center”, found at the Altera homepage <http://www.altera.com>.

3.2.3.1 Node locked license

The license file for the EtherCAT IP core (license_<company>_<MAC ID>.dat) has to be made available to the Altera Quartus development environment. The EtherCAT IP core can only be used with a license file.

The location of your Altera license file can be found in the Altera Quartus license setup ("Tools – License Setup..." from the menu.):

There are three options:

- Add the path of the license file (separated by a semicolon) to the license file input box in the Altera Quartus license setup.
- Add the path of the license file to the LM_LICENSE_FILE environment variable (separated by a semicolon) if the LM_LICENSE_FILE variable is used.

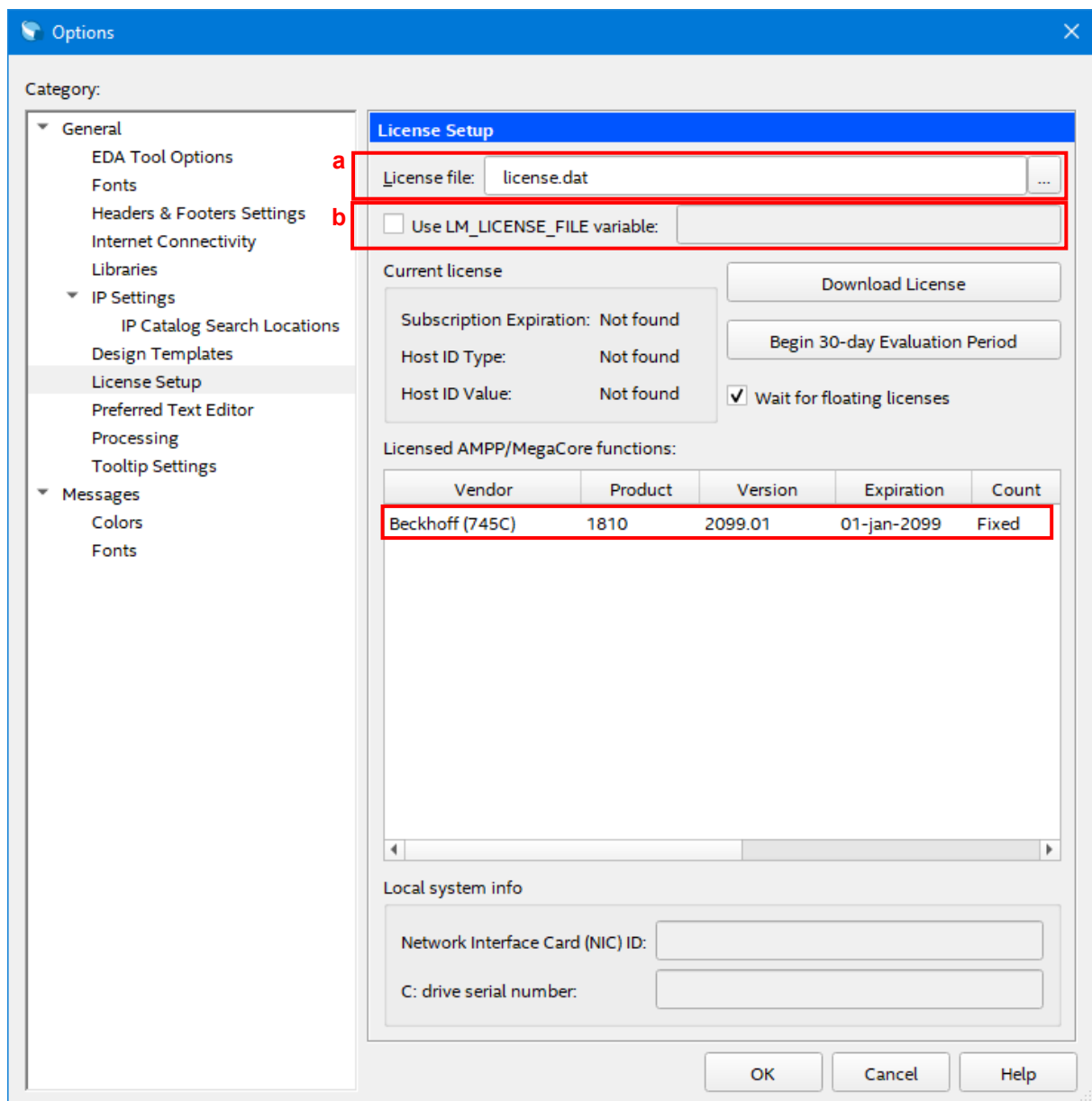


Figure 4: License setup

The EtherCAT IP core license is shown in licensed AMPP/MegaCore® functions list: the vendor is Beckhoff (745C), and the product number is 1810.

3.2.3.2 Floating license

Add the license feature lines for the EtherCAT IP core to the license file of your license server.

4 IP core usage

Quick summary:

1. Add EtherCAT IP core to Quartus IP catalog
2. Select EtherCAT IP from IP catalog, configure it, and generate IP
3. Connect EtherCAT IP signals
4. Setup timing constraints

Add to IP catalog

The EtherCAT IP core is configured in Quartus, like any other IP. The EtherCAT IP core is added to the IP catalog in one of two ways:

- Add the content of the `<IPInst_dir>/ethercat_ip/` folder to your Quartus project folder, so that the .ipx file is in the same folder as the .qpf file. Quartus projects, and Platform Designer projects can find the IP if they are located in the same folder
or
- Add the `<IPInst_dir>/ethercat_ip/` folder to your Quartus IP search path. If you want to use a project specific search path, you need to add it to your Platform Designer search path as well. A global search path is used by both Quartus and Platform Designer.

Using the EtherCAT IP

You can use the EtherCAT IP core either in Quartus directly, or in the Quartus platform designer. In both ways, you select the EtherCAT IP core from the IP catalog and add it to your design.

When used in the platform designer, you just connect the configured EtherCAT IP core to your system, and to external ports.

When used in Quartus directly, a wrapper file is generated which will be instantiated by your logic. For your convenience, a package containing a VHDL component declaration of the wrapper is also generated.

Quartus will write several files, the major files required for synthesis are:

- The EtherCAT IP core source code file (ETHERCAT_IPCORE.VHD), which contains all the configurable EtherCAT slave functionality.
- The vendor ID package (VHDL file)
- A generated VHDL wrapper file (`<design_name>.vhd`), which instantiates the EtherCAT IP core source code file with selected features and signals.

Additional files are also generated, depending on the configuration:

- The IP component or platform designer file which contains the configuration (`<design_name>.qsys`)
- An instantiation and component declaration example (`<design_name>_inst.vhd`, `<design_name>.cmp`)
- A Quartus IP file (`<design_name>.cmp`)
- The Open-Core Plus file

The source code of the EtherCAT IP core is compatible with VHDL-1993.

4.1 IP core configuration

The EtherCAT IP core is configured in Quartus, starting with a typical/recommended configuration. The output of the tool is a VHDL wrapper for the EtherCAT IP core library file. The wrapper file makes only those interfaces visible, which were selected by the user, and it configures the EtherCAT IP Core using generics as desired.

4.1.1 Overview

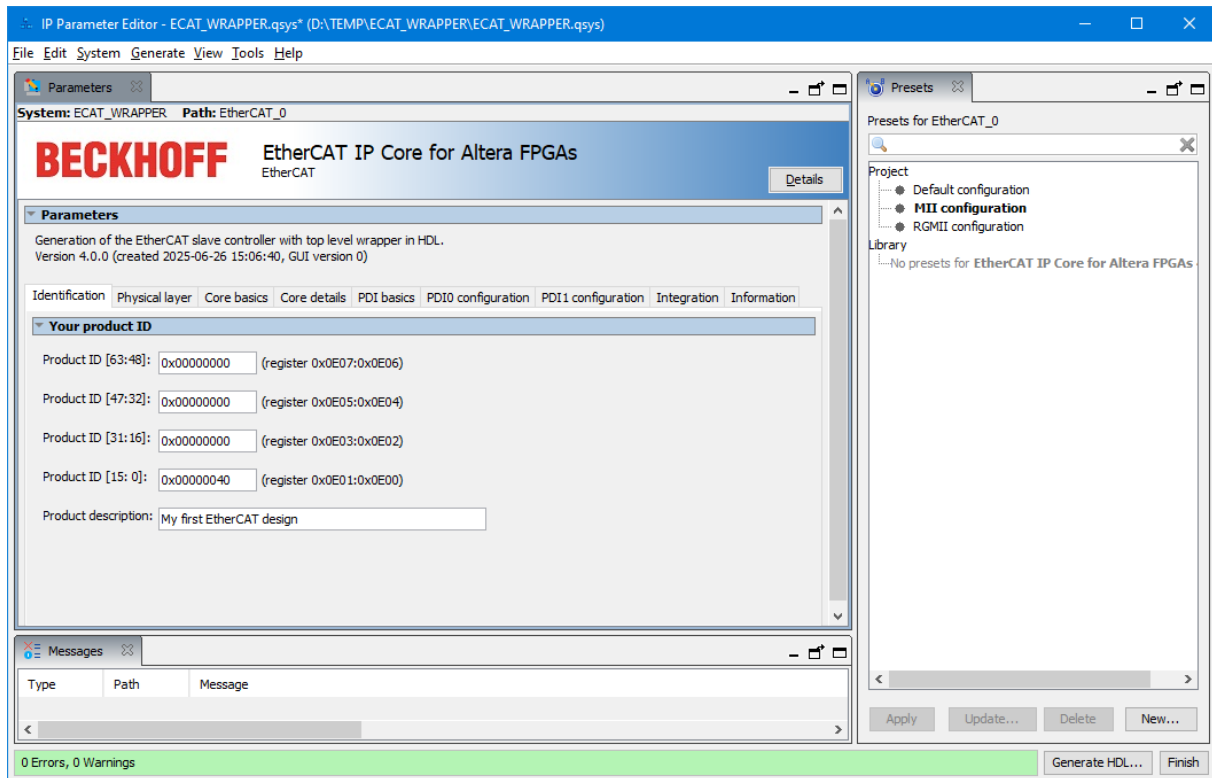


Figure 5: EtherCAT IP core configuration interface

Details button

Links to documentation on the IP, and an overview of the configuration options can be found here.

Parameters pane (left)

The configuration options for the EtherCAT IP core are available in the IP core parameters pane on the left side.

Presets pane (right)

Common feature sets can be selected. Presets can be modified by adding/disabling individual features in the parameters pane.

Message pane (bottom)

In the lower box additional information like warnings, errors, and recommendations are displayed.

4.1.2 Identification tab

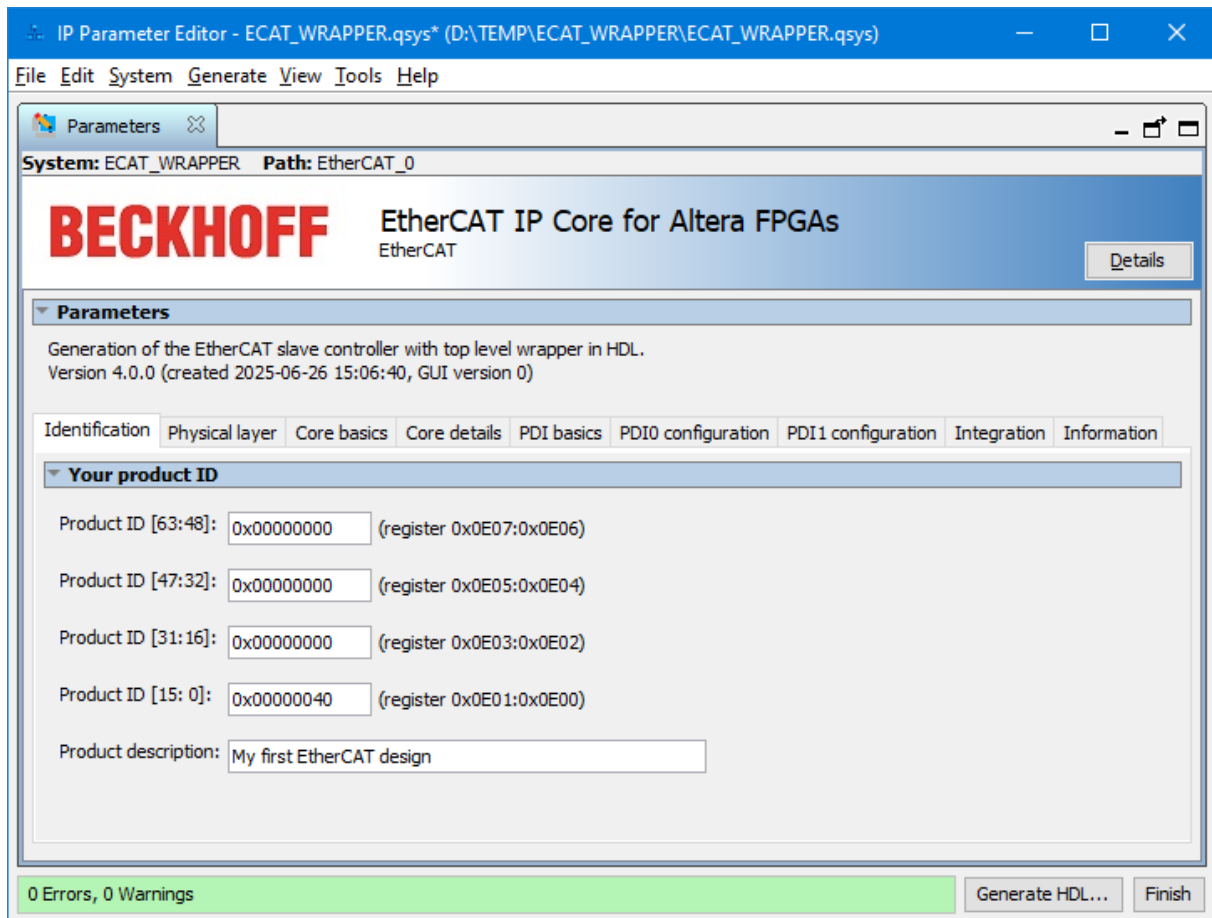


Figure 6: Identification tab

Your product ID

The product ID can be chosen freely, and it is used for your purposes. It can be read out in register 0x0E00:0x0E07. The product ID is meant to identify different configurations of the IP core, or products for which it is used. It does not have to reflect the EtherCAT slave product code, which is part of the EEPROM/XML device description. In case of doubt, enter zero.

The product ID has to be entered in hexadecimal format as a number between 0x0000 and 0xFFFF for each of the four 16 bit fields (representing a 16 bit part of the 64 bit product ID each).

Product description

The product description can be chosen freely, and it is used for your purposes. During synthesis, it is shown as a message as part of the EtherCAT IP configuration.

4.1.3 Physical layer tab

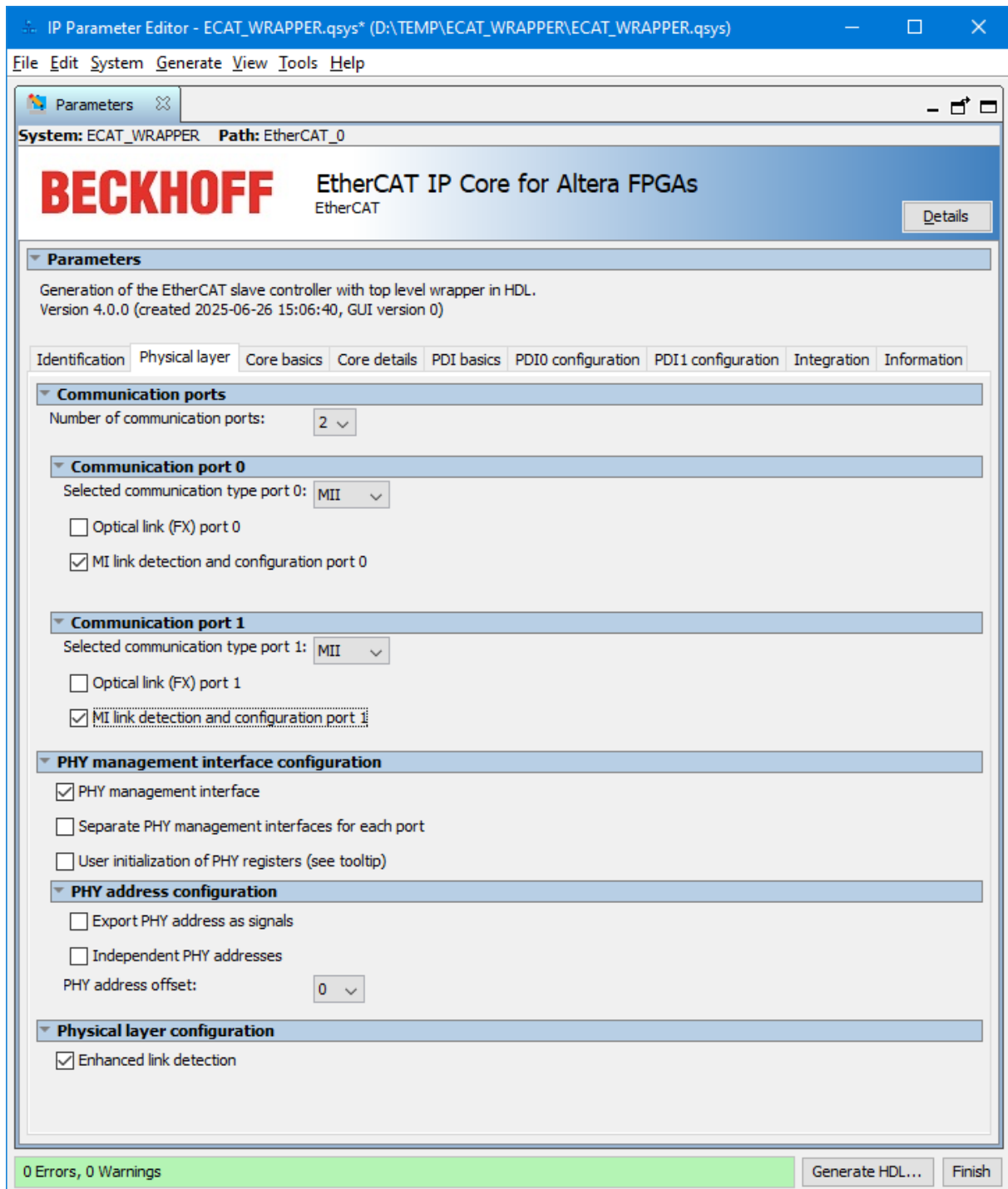


Figure 7: Physical layer tab

Communication ports

The number of EtherCAT communication ports can be selected here. The default value is two.

Selected communication type

The PHY interface, MII, RMII, or RGMII can be selected. It is recommended to use MII because the latency is much better. All ports have to use the same type.

Optical link (FX)

Each port can be configured to be an FX port (fiber optic), which has influence on enhanced link detection and MI link detection and configuration, since FX connections do not use auto-negotiation.

MI link detection and configuration

Each port can be configured to use MI link detection and configuration. Ethernet PHYs are configured to use only 100 Mbit/s full duplex, and link status is checked via the MII management interface. A link signal or RGMII in-band status have to be used for link (down) detection regardless of this feature.

PHY management interface

The PHY management interface function can be enabled, it is strongly recommended.

Separate PHY management interfaces for each port

Usually, all PHYs are connected to the same management bus. Using this feature, individual interfaces for each port are used.

User initialization of PHY registers

If MI link detection and configuration is enabled, it is possible to let the EtherCAT IP core configure some PHY registers before operation is started. A simple ROM interface is used by the EtherCAT IP core, do retrieve the write values. An example implementation of such a ROM is part of the IP core package.

Export PHY address as signals

Enable for dynamically changing PHY addresses (the PHY address configuration is exported as signals), otherwise the PHY address configuration is static.

Independent PHY addresses

Enable if the PHY addresses are not consecutive. If enabled, the PHY addresses of each port can be configured individually.

PHY address offset

Configure the base PHY address (belonging to port 0) if the PHY addresses are consecutive (independent PHY addresses disabled).

PHY address

Configure the PHY address for each port individually (independent PHY addresses enabled).

Enhanced link detection

Enhanced link detection is a mechanism of informing link partners of receive errors. The technical support for this feature is enabled here, but it is finally activated based on the EEPROM SII content. It is recommended to enable this feature, which required a PHY management interface and proper PHY address configuration.

4.1.4 Core basics tab

IP Parameter Editor - ECAT_WRAPPER.qsys* (D:\TEMP\ECAT_WRAPPER\ECAT_WRAPPER.qsys)

File Edit System Generate View Tools Help

Parameters

System: ECAT_WRAPPER Path: EtherCAT_0

BECKHOFF EtherCAT IP Core for Altera FPGAs
EtherCAT

Details

Parameters

Generation of the EtherCAT slave controller with top level wrapper in HDL.
Version 4.0.0 (created 2025-06-26 15:06:40, GUI version 0)

Identification Physical layer **Core basics** Core details PDI basics PDI0 configuration PDI1 configuration Integration Information

FMMU configuration

Number of FMMU: 3

SyncManager configuration

Number of SyncManager: 4

☐ SyncManager deactivation delay for PDI

☒ SyncManager sequential mode for ECAT

Process data RAM configuration

Process data RAM: 4 kBytes

Private PDI RAM: 0 kBytes

Distributed clocks configuration

☐ Receive times enabled (register 0x0900:0x90F)

☒ Distributed clocks enabled (includes receive times)

DC SyncSignals: Sync0/1

DC LatchSignals: Latch0/1

Distributed clocks width: ☒ 32 Bits ☐ 64 Bits

Cyclic pulse length: 100 *10ns (0 == acknowledge mode)

☒ SyncManager event times

☐ DC speed counter direct control (register 0x0938:0x0939)

Mapping to global IRQ

☒ Map SyncSignals[0/2] to AL event register 0x0220

☒ Map SyncSignals[1/3] to AL event register 0x0220

0 Errors, 0 Warnings

Generate HDL... Finish

Figure 8: Core basics tab

Number of FMMU

Number of FMMU instances. Between 0 and 16 FMMUs are possible, recommended minimum is 3.

Number of SyncManager

Number of SyncManager instances. Between 0 and 16 SyncManagers are possible, recommended minimum is 4.

SyncManager deactivation delay for PDI

Some applications require that the PDI is able to read the last SyncManager buffer data, even if the EtherCAT master has already disabled the SyncManager (e.g. because OPERATIONAL mode has been left). This option allows to read the last buffer consistently from PDI.

SyncManager sequential mode for ECAT

Some applications use very large SyncManager buffer sizes, which might require three or more EtherCAT frames for data transfer. This feature should be activated to guarantee consistency for situations when one of the middle frames is lost.

Process data RAM

The size of the process data memory can be determined in this dialog. Minimum memory size is 0 Kbyte, maximum memory size is 60 Kbyte.

Private PDI RAM

A certain amount of the process data RAM can be reserved for PDI usage only, without access for the EtherCAT master. The purpose is to support a scratch pad memory for data transfer between μ Controllers attached to PDI0 and PDI1. The PDI private RAM is located at the end of the process data RAM.

Receive times enabled

The distributed clocks receive times are always enabled.

Distributed clocks enabled

The distributed clocks feature comprises synchronized distributed clocks, receive times, and distributed clocks system time.

DC SyncSignals

Select the number of SyncSignals from 0 to 4.

DC LatchSignals

Select the number of LatchSignals from 0 to 4.

Distributed clocks width

The width of the distributed clocks can be selected to be either 32 bit or 64 bit. DC with 64 bit require more FPGA resources. DC with 32 bit and DC with 64 bit are interoperable.

Cyclic pulse length

Determines the length of SyncSignal output (register 0x0982:0x0983).

SyncManager event times

Distributed Clocks SyncManager event times (0x09F0:0x09FF) are available if checked. Used for debugging SyncManager interactions.

DC speed counter direct control

The distributed clocks speed is usually automatically derived from time stamps received from the reference clock. With this feature, it is also possible to directly control the speed. This is rarely used.

Map SyncSignals[*] to AL event register 0x0220

DC SyncSignals 0/2 and 1/3 can additionally be mapped internally to the AL event register 0x220, and therefore to the μ Controller interrupt output. This might be a good solution if a μ Controller interface is short on interrupt inputs. However, the sync signals will remain available as separate outputs.

4.1.5 Core details tab

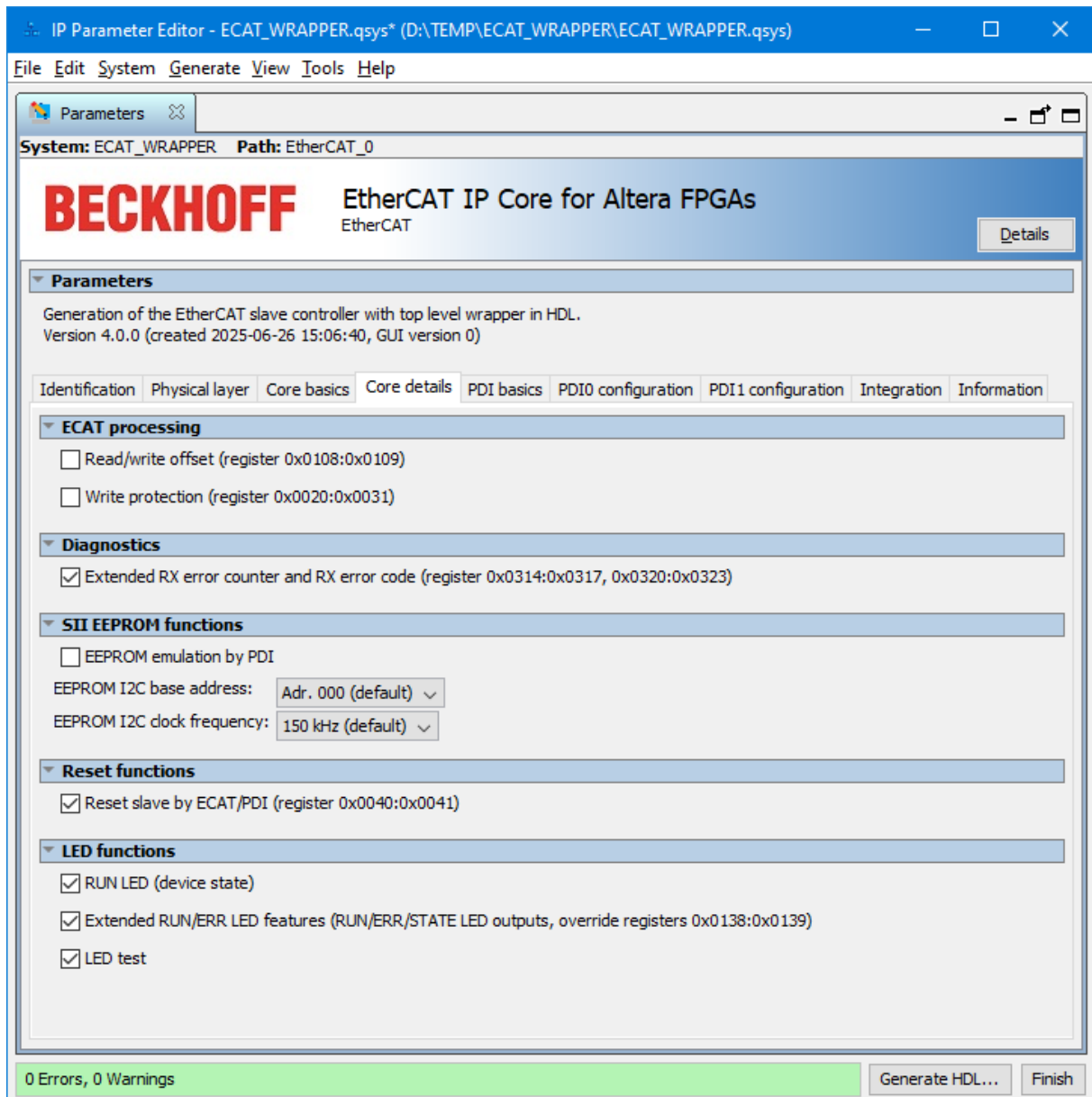


Figure 9: Core details tab

Read/write offset

Physical read/write offset (0x00108:0x0109) is available if checked.

Write protection

Register write protection and ESC write protection (0x0020:0x0031) are available if checked.

Extended RX error counter and RX error code

Extended RX error counter registers (0x0314:0x0317) and RX error code registers (0x0320:0x0323) are available if checked.

EEPROM emulation by PDI

If this feature is disabled, an external I²C EEPROM is required for SII data storage. With this features enabled, no external I²C EEPROM is required. Instead, the SII data is stored in a non-volatile memory, accessible by a μ Controller. The μ Controller transfers read data from the NVRAM to the EtherCAT IP core, and write data from the EtherCAT IP core to the NVRAM, using register polling or interrupts.

EEPROM I2C base address

The I2C base address for accessing word 0 of the SII can be set. The three topmost bits can be chosen. It is recommended to use base address 0, other values are only useful for existing evaluation boards.

EEPROM I2C clock frequency

The I2C clock frequency can be reduced for certain evaluation boards only. It is recommend to use the default speed.

RESET slave by ECAT/PDI

The reset registers (0x0040:0x0041) and the RESET_OUT signal is available if this feature is checked.

RUN_LED

RUN LED output indicates AL status (0x0130) if activated. Otherwise RUN LED has to be controlled by a µController. Always activated if no PDI is selected or if digital I/O PDI is selected.

Extended RUN/ERR LED features

Support for ERR LED and STATE LED, direct control of RUN/ERR LED via RUN/ERR LED override register (0x0138:0x0139).

LED test

A short LED flash after reset for all LED signals is enabled if this feature is selected.

4.1.6 PDI basics tab

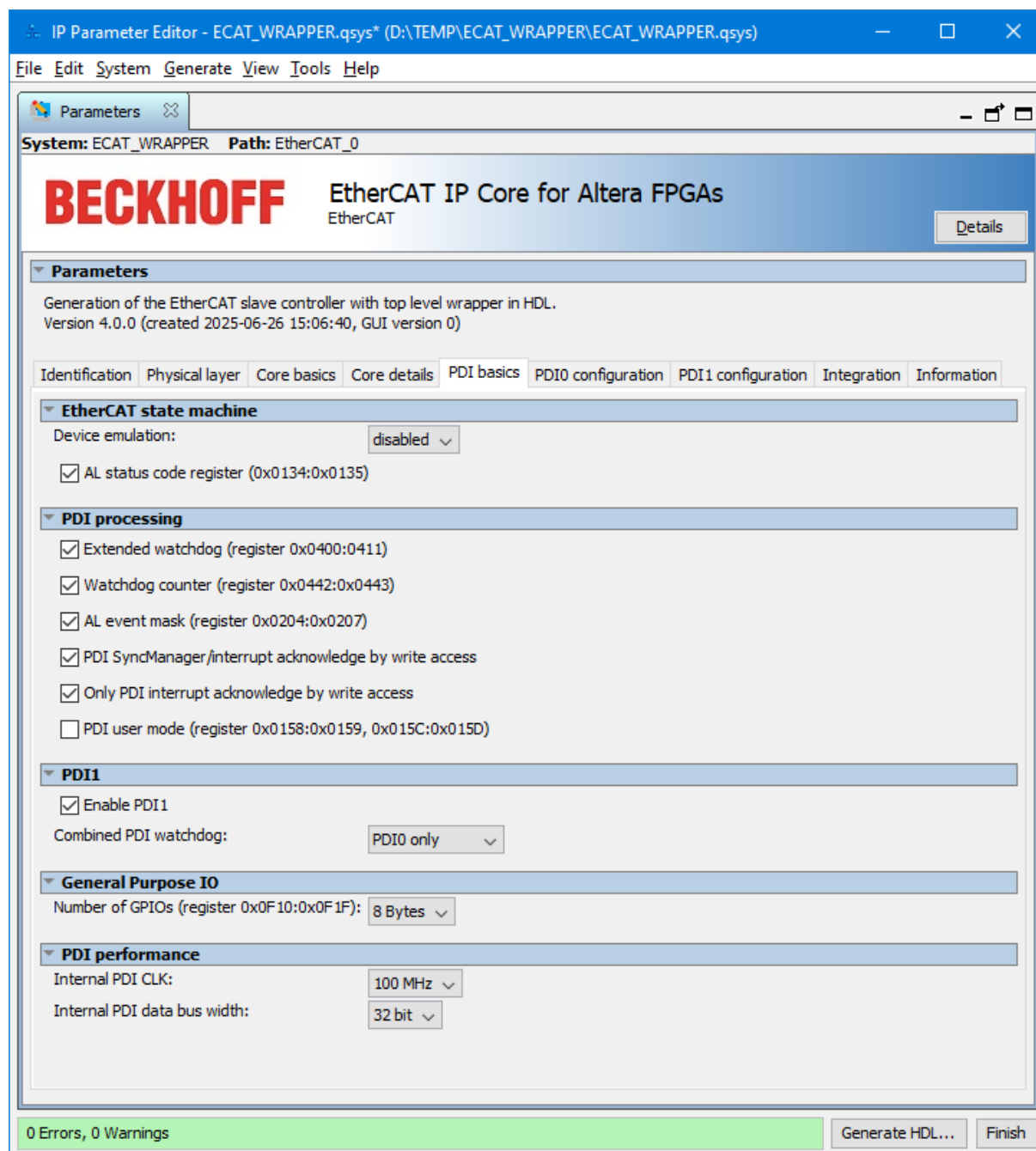


Figure 10: PDI basics tab

Device emulation

Enable device emulation (0x0141[0]). This feature should be enabled for simple slaves without processors (no PDI or digital IO only).

AL status code register

AL status code register (0x0134:0x0135) is available if checked. It is mandatory for complex slaves with μ Controllers.

Extended watchdog

Watchdog divider (0x0400:0x0401) is configurable and PDI watchdogs (PDI0: 0x0410:0x0411, PDI1: 0x0412:0x0413, and 0x0100.1) are available if checked.

AL event mask register

AL event mask registers (PDI0: 0x0204:0x0207, PDI1: 0x020A:0x020D) is available if checked.

PDI SyncManager/interrupt acknowledge by write access

Some ESC functions are triggered by reading from the PDI. Since PDI data bus widths are increasing up to 64 bit and beyond, it is not possible to read individual bytes anymore because most μ Controllers do not support byte enable signals for read commands. In order to prevent accidentally reading of trigger addresses (like SyncManager buffer end or interrupt acknowledge registers), this option allows to use write commands (with byte enables) to trigger the functions.

Only PDI interrupt acknowledge by write access

The above feature can be restricted to only interrupt registers. SyncManager buffers will still use read commands for buffer switching. This is usually possible by separating SyncManager buffers, so that accidental access to other SyncManagers cannot occur.

PDI user mode

Some PDI configurations can be changed during run-time using registers (PD0: 0x0158:0x0159, 0x015C:0x015D, PD1: 0x0198:0x0199, 0x019C:0x019D)

Enable PDI1

Enable second PDI interface.

Combined PDI watchdog

Configure how the two PDI watchdogs are combined for register 0x0100[1] and the error LED.

Number of GPIOs

General purpose I/O signals can be added to any selected PDI. The number of GPIO bytes is configurable to 0, 1, 2, 4, or 8 bytes. Both general purpose outputs and general purpose inputs of the selected width are available.

Internal PDI CLK

The internal PDI clock frequency determines the potential PDI performance, but it makes timing closure more difficult. Possible values are 25/50/100 MHz.

Internal PDI data bus width

The internal PDI data bus width determines the potential PDI performance. Possible values are 8/16/32 bit. Please be aware that values higher than 8 bit may require more spacing between SyncManager buffer areas. Please refer to section I regarding SyncManager memory usage with different processing data widths.

4.1.7 PDI0/PDI1 configuration tab

Up to two PDI interfaces between ESC and the application are available, supporting each one of several interface standards:

- Digital I/O
- Asynchronous μ Controller (PDI0 only)
- SPI slave
- Avalon memory-mapped on-chip bus
- AXI4/AXI4-Lite on-chip bus

In addition, general purpose I/O are also available.

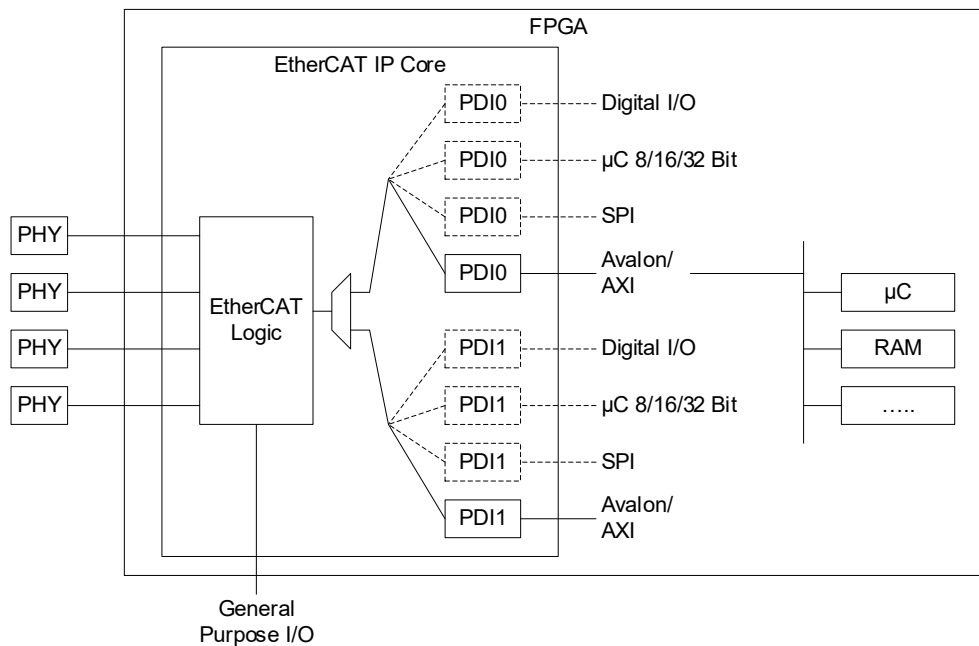


Figure 11: Available PDI interfaces

The PDI can be selected from the pull down menu for PDI0 and PDI1. Individual settings for the selected PDI are shown and can be changed. Both PDI0 and PDI1 support the same settings.

If there is no PDI interface selected, communication with the application is not possible – except for general purpose I/O.

4.1.7.1 Digital I/O configuration

The digital I/O PDI supports up to 4 bytes of digital I/O signals. Each byte can be assigned as input or output byte.

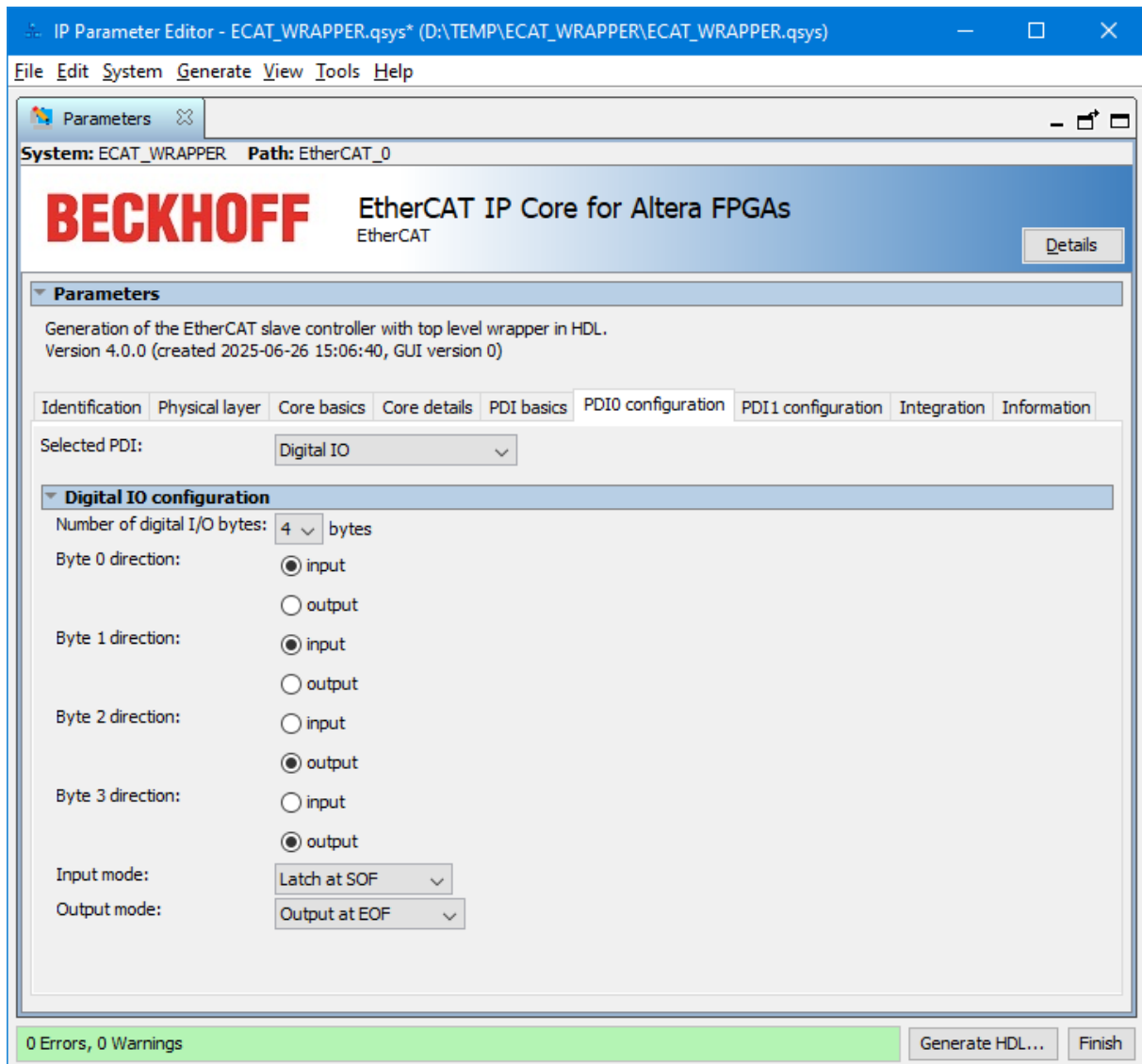


Figure 12: PDI configuration tab – digital I/O configuration

Number of digital I/O bytes

Total number of I/Os. Possible values are 1, 2, 3 or 4 bytes.

Byte 0-3 direction

Defining byte-wise if digital I/Os are used as input or output byte

Input mode

Defines the latch signal which is used to take over input data.

- Latch at SOF (Start of frame)
The inputs are latched just before the data have to be written in the frame.
- Latch with ext. signal
Connected to DIGI_LATCH_IN. Application controls latching
- Latch at Dist-Sync0
Latch input data with distributed clock Sync0 signal
- Latch at Dist-Sync1
Latch input data with distributed clock Sync1 signal

Output mode

Defines the trigger signal for data output.

- Output at EOF (End of frame)
The outputs will be set if the frame containing the data is received complete and error free.
- Output at OUT_START signal event
Outputs will be set with a rising edge of the OUT_START signal.
- Output at Dist-Sync0
Outputs will be set with Sync0 signal if distributed clocks are enabled.
- Output at Dist-Sync1
Outputs will be set with Sync1 signal if distributed clocks are enabled.

4.1.7.2 μ Controller configuration (8/16/32 bit, PDI0 only)

The 8/16/32 bit μ Controller interface is an asynchronous parallel interface for μ Controllers. The data bus is either 8, 16, or 32 bit wide.

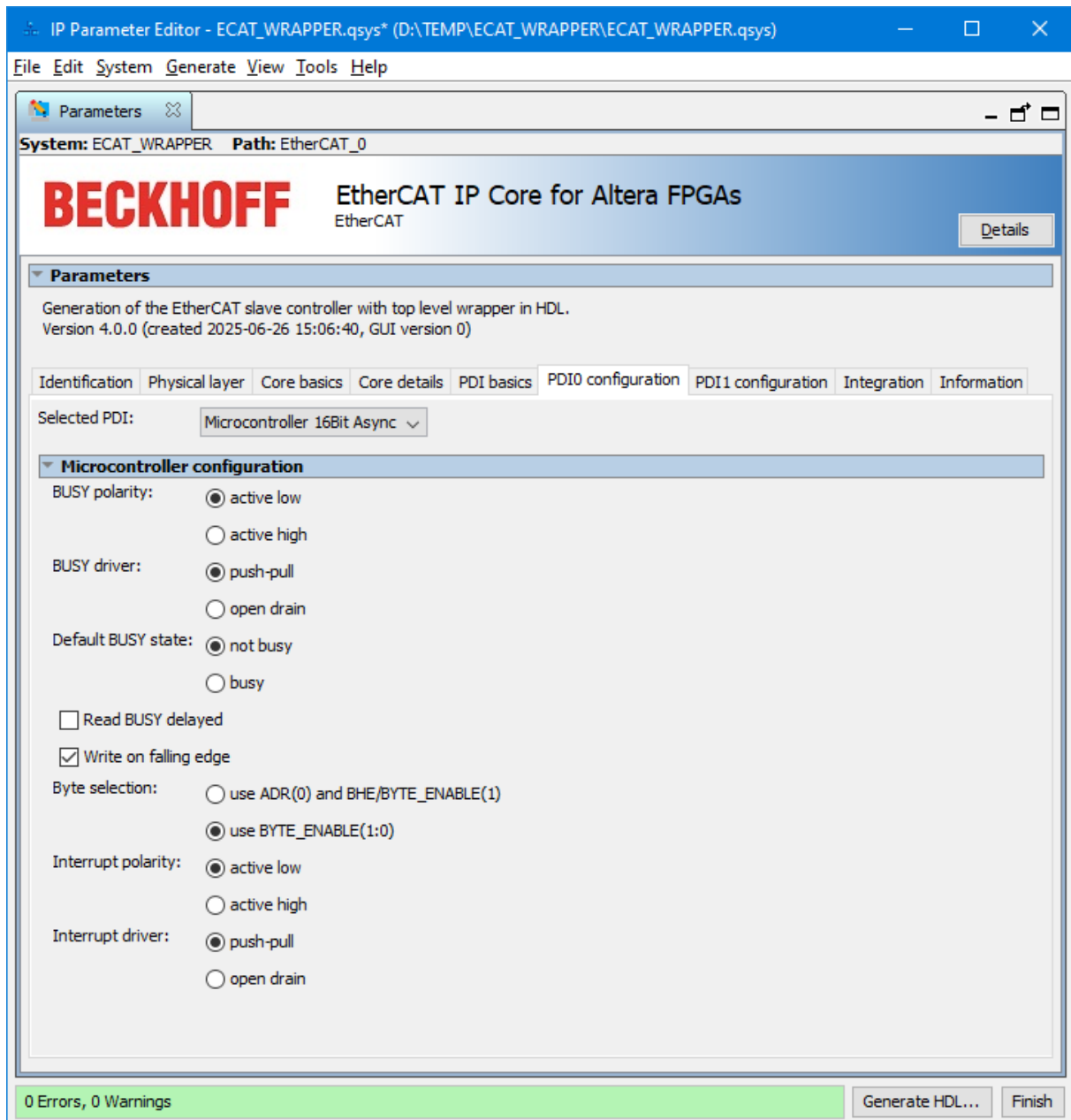


Figure 13: PDI configuration tab – μ C configuration

BUSY polarity, BUSY driver

Electrical definition of the busy signal driver

Default BUSY state

Default BUSY state signalling before an access has been detected (refer to register 0x00152[4]).

Read BUSY delayed

Delay the output of the BUSY signal by ~20 ns (refer to register 0x00152.0).

Write on falling edge

Start write access earlier with falling edge of nWR. Single write accesses will become slower, but maximum write access time becomes faster.

Byte selection

For 16 bit μ Controller only, you can select between a BHE (byte high enable) signal and the lowest address bit to select the byte lanes, or use two BYTE_ENABLE signals for selecting byte lanes.

Interrupt polarity, interrupt driver

Electrical definition of the interrupt signal driver

4.1.7.3 SPI configuration

The SPI interface is a serial slave interface for μ Controllers.

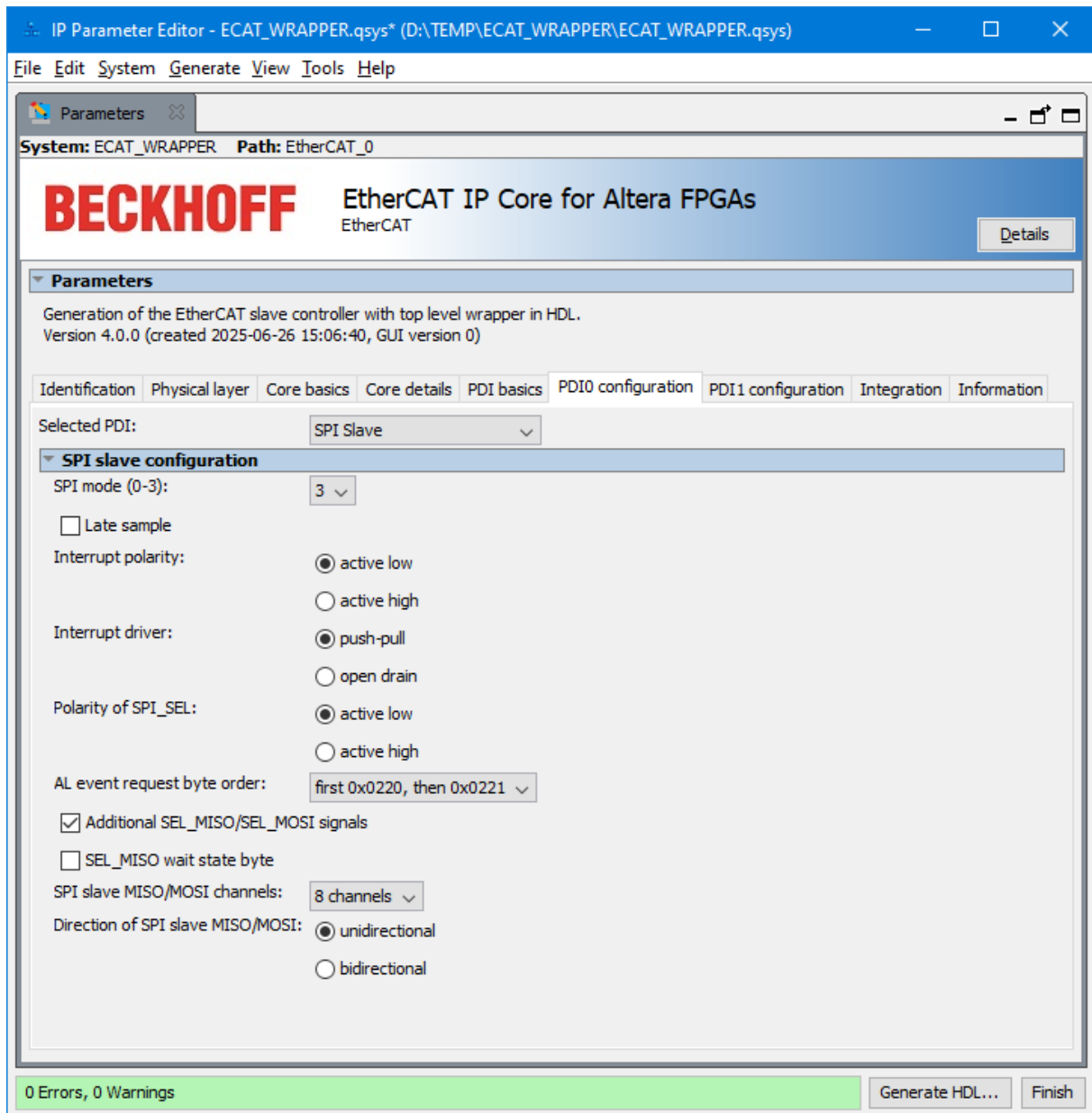


Figure 14: PDI configuration tab – SPI configuration

SPI mode

The SPI mode determines the SPI timing. Refer to SPI PDI description for details.

Late sample

The late sample configuration determines the SPI timing. Refer to SPI PDI description for details. It is recommended to leave this unchecked for slave sample code.

Interrupt polarity, interrupt driver

SPI_IRQ output driver configuration.

Polarity of SPI_SEL

SPI_SEL signal polarity.

AL event request byte order

The unidirectional SPI interface presents AL event request values on MISO during the address phase. You can choose the order or bytes.

Additional SEL_MISO/SEL_MOSI signals

Add two additional select signals for direct access to process data SyncManager buffers.

SEL_MISO wait state byte

When enabled, the first byte transferred with SEL_MISO active is a wait state byte. Otherwise, the μ Controller has to guarantee a minimum delay between SEL_MISO and first SPI_CLK.

SPI slave MISO/MOSI channels

The SPI slaves supports transfer of 1/2/4/8 MISO/MOSI bits per SPI clock event in parallel.

Direction of SPI slave MISO/MOSI

For unidirectional operation, separate MISO/MOSI signals are available, data transfer is full-duplex. In bidirectional operation, shared MISO/MOSI signals are used, the protocol defines data flow direction.

4.1.7.4 Avalon configuration

The Avalon PDI connects the IP core with an Avalon master (e.g., Altera NIOS). The Avalon PDI uses memory addressing/dynamic bus sizing.

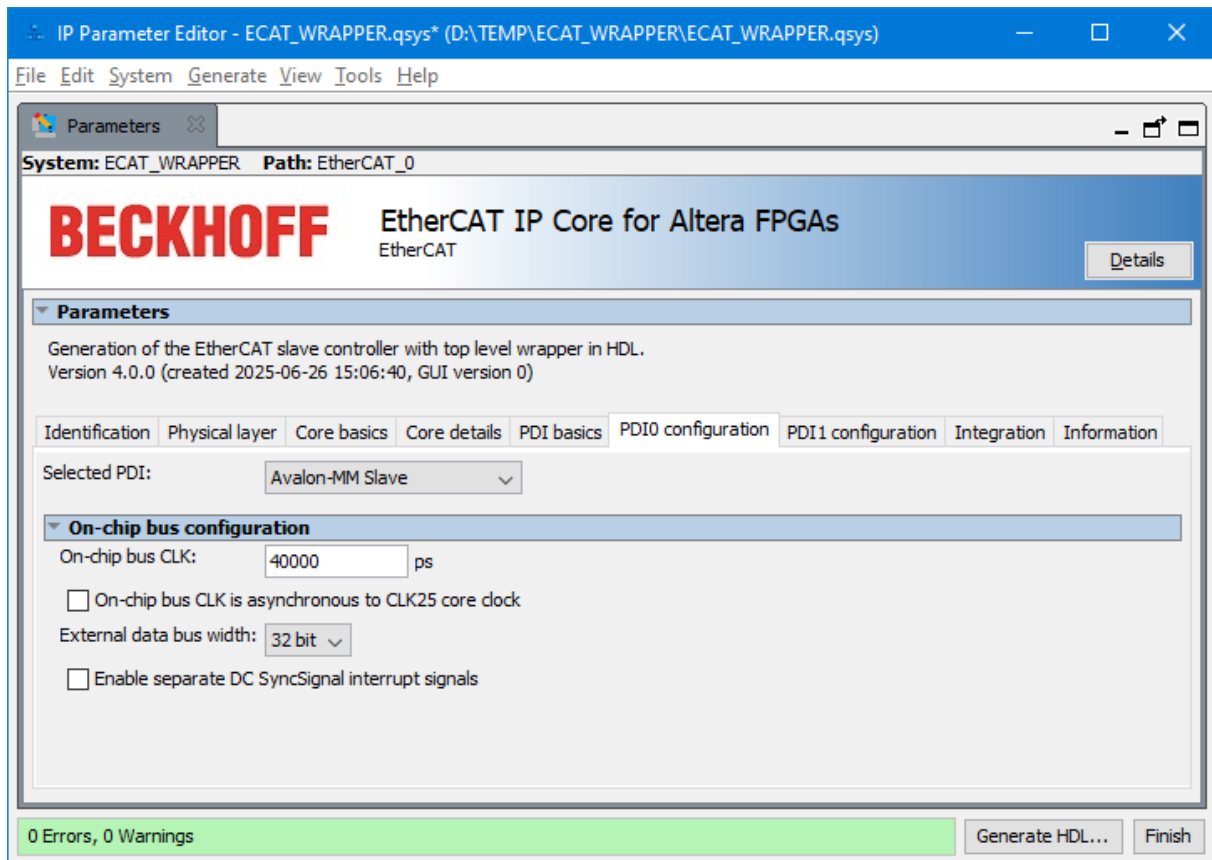


Figure 15: Register PDI – Avalon interface configuration

On-chip bus CLK

This is the clock period of the Avalon bus clock for communication between ESC and the Avalon master. This configuration option is not editable if the clock period can be derived from the Platform designer clock connections.

On-chip bus CLK is asynchronous to CLK25 core clock

Enable if the On-chip BUS CLK is asynchronous to CLK25. Additional synchronization stages are added in this case.

External data bus width

Select the Avalon data bus width (8/16/32/64 bit) of the Avalon slave interface.

Enable separate DC SyncSignal interrupt signals

If enabled, the DC SyncSignals are additionally offered as interrupt sources for connection in the Platform designer.

4.1.7.5 AXI4/AXI4-Lite configuration

The AXI PDI connects the IP core with an AXI master. The data bus width is variable 8/16/32/64 bit.

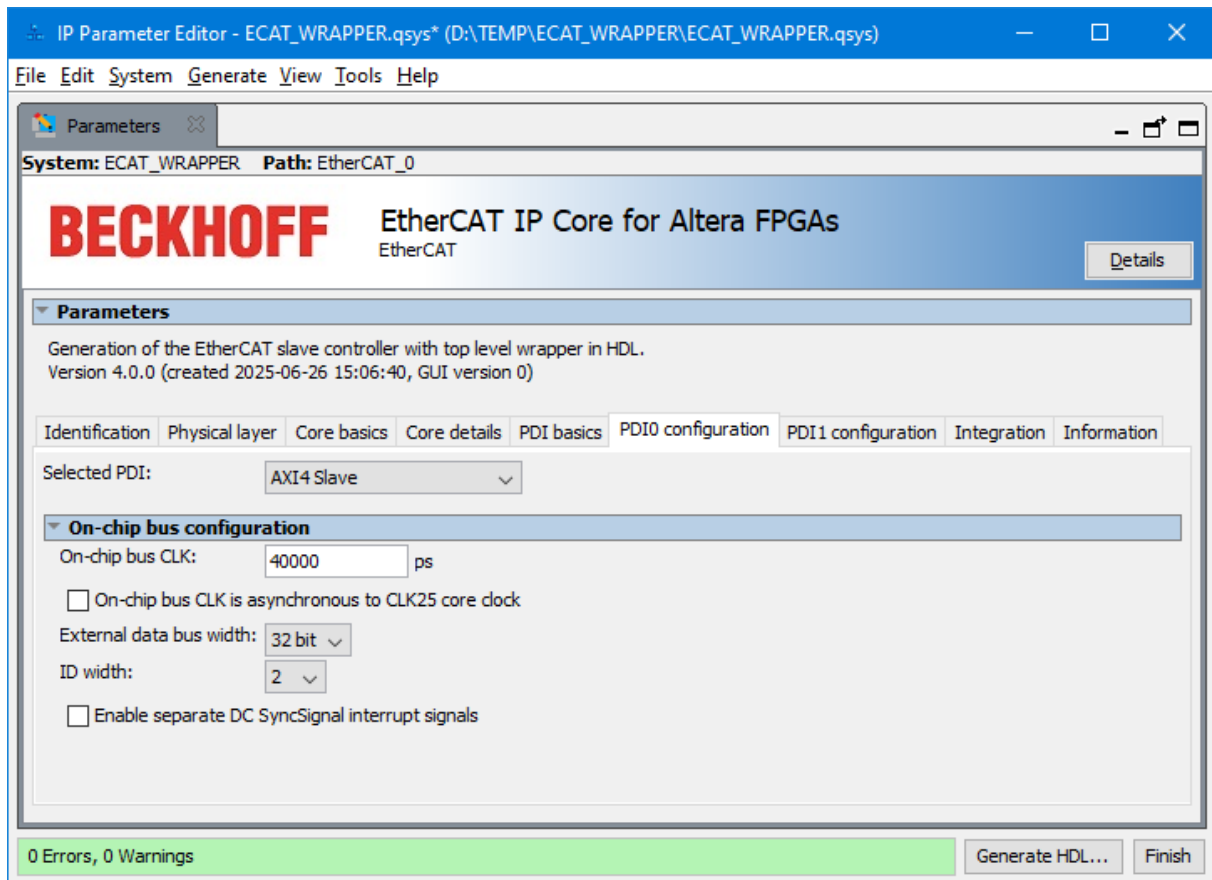


Figure 16: PDI configuration tab – AXI interface configuration

On-chip bus CLK

This is the clock period of the AXI bus clock for communication between ESC and the AXI master. This configuration option is not editable if the clock period can be derived from the Platform designer clock connections.

On-chip bus CLK is asynchronous to CLK25 core clock

Enable if the on-chip bus clock is asynchronous to CLK25. Additional synchronization stages are added in this case.

External data bus width

Select the AXI data bus width (8/16/32/64 bit) of the AXI slave interface.

ID width

Width of the AXI ID signals.

Enable separate DC SyncSignal interrupt signals

If enabled, the DC SyncSignals are additionally offered as interrupt sources for connection in the Platform designer.

4.1.8 Integration tab

The settings on the integration tab define how the IP core will be used in the tool flow.

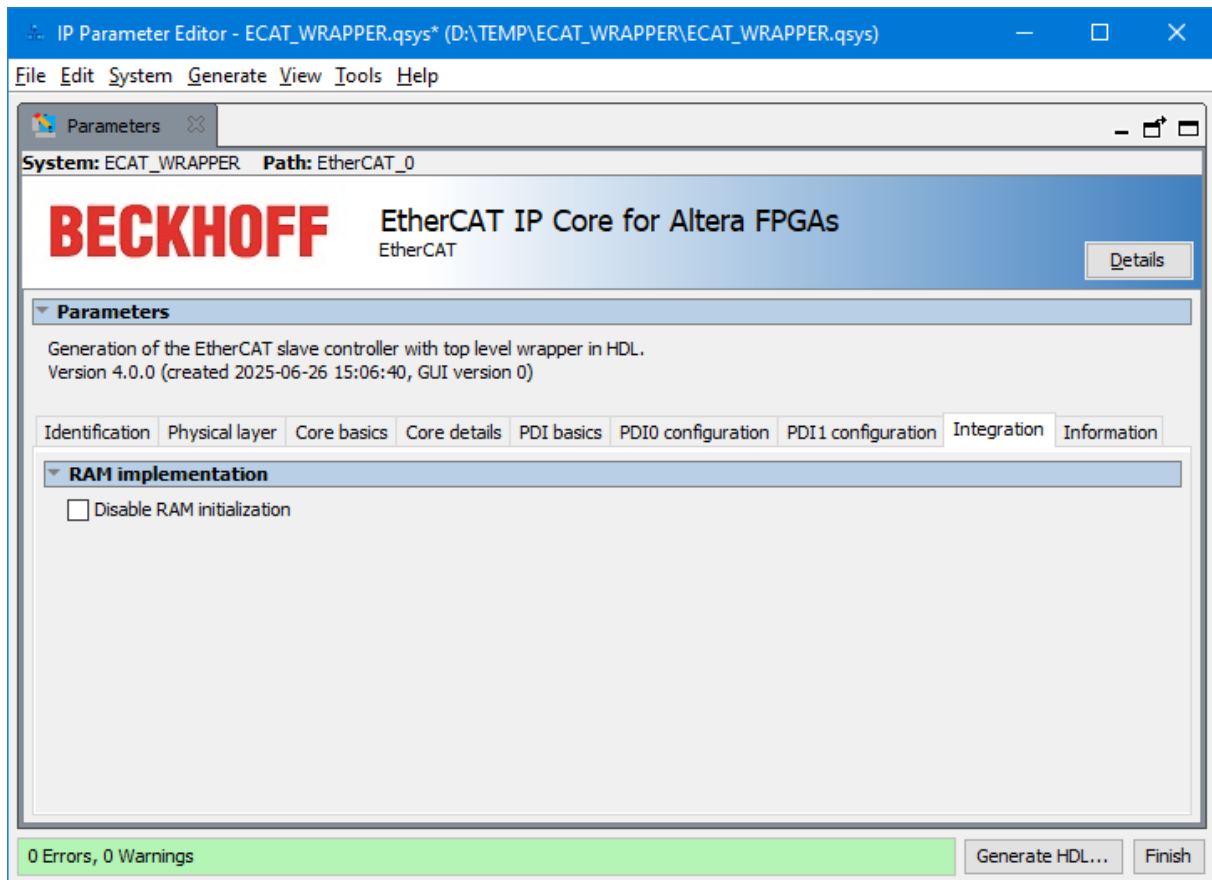


Figure 17: Integration tab

Disable RAM initialization

The EtherCAT IP core uses RAM initialization for the User RAM, which contains the actually configured feature set. This is only used for debugging. The process RAM is initialized with zeros. All the initialization can be disabled by this option.

4.1.9 Information tab

The information tab shows results of the settings on the other tabs. These settings are also shown by the IP core itself during synthesis (as long as the tool supports this), so differences can be detected easily.

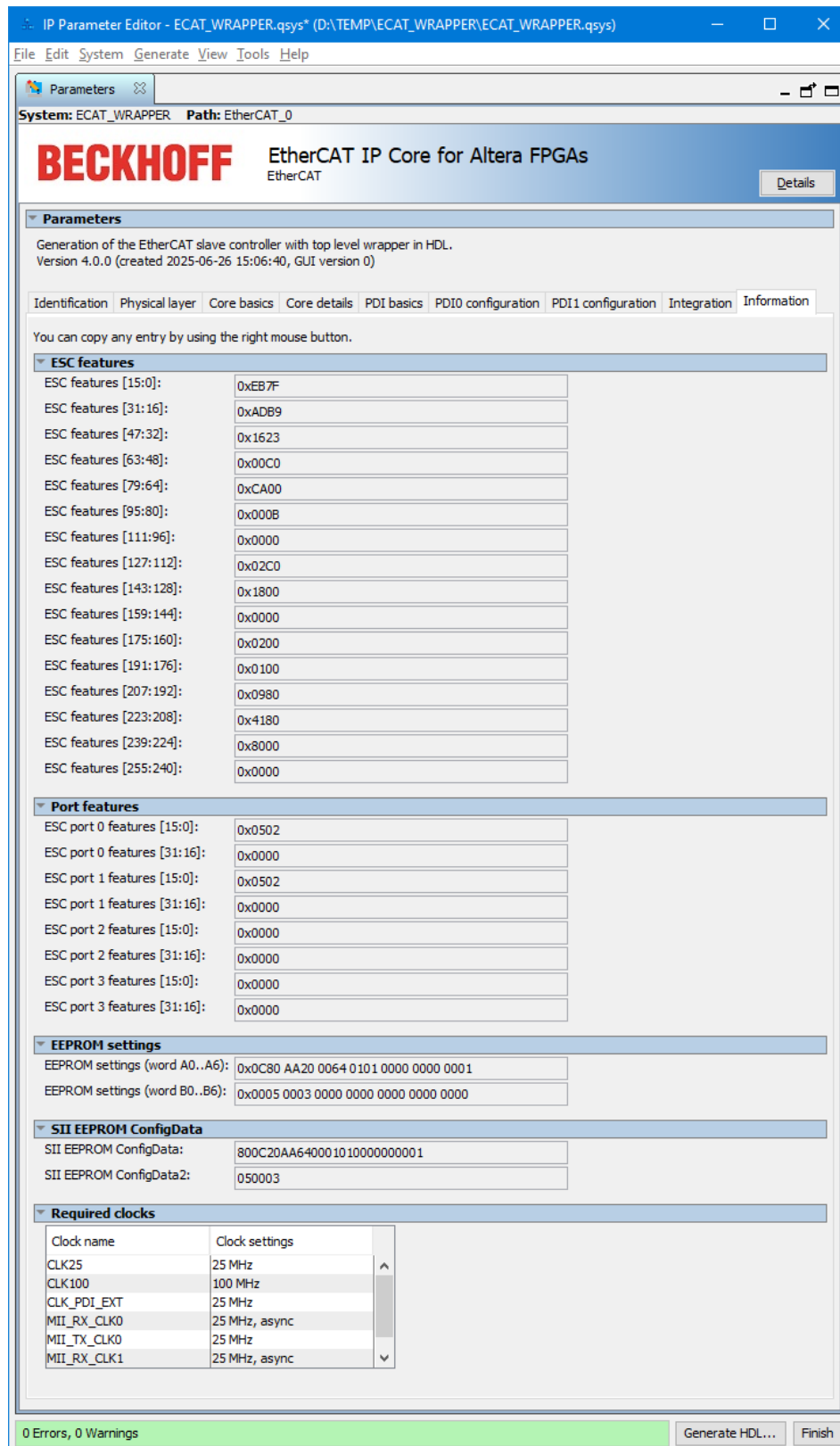


Figure 18: Information tab

ESC features

The ESC features define which features are enabled for the IP core.

Port features

The port features define which features are enabled for the ports of the IP core.

EEPROM settings and SII EEPROM ConfigData

Although most SII EEPROM settings are fixed for the IP core, the EEPROM content should reflect the current configuration. The current configuration is shown as words and as template for the ConfigData and ConfigData2 entries of the XML.

Required clocks

The required clock inputs depending on the configuration are shown here.

5 FPGA resource consumption

The resource consumption figures shown in this chapter reflect results of example synthesis runs and can only be used for rough resource estimations. The figures are subject to quite large variations depending on design tools and version, FPGA type, constraints (e.g., area vs. speed), total FPGA utilization (design tools typically stop optimization if the timing goal is reached), etc. No extra effort was undertaken to achieve optimum results, i.e. by sophisticated constraining and design flow setting.

For accurate resource consumption figures, please use the evaluation license of the EtherCAT IP core and synthesize your individual configuration for the desired FPGA.

The figures of the following table do not imply that the individual features are operational in the selected FPGA (i.e., that the resources are sufficient or that timing closure is achievable). The synthesis runs were performed without timing constraints, without location constraints, and without bitstream generation.

The EtherCAT IP core resource consumption overview figures are based on EtherCAT IP core for Altera FPGAs Version 4.0.0, Altera Quartus Prime Standard 23.1, and Altera Cyclone IV (LE) or Cyclone V (ALM) devices. For estimations, use the base value, and add all configurable functions.

Table 12: Approximate resource requirements for main configurable functions

Configurable Function	Reg.	LC	ALM	Details
Base	1,500	2,200	850	calculation base (no ports, no PDI, no features)
Additional port	850	1,550	650	all diagnostic port features enabled (with DC receive times, PHY management functions, MI link detection)
PHY management interface	300	650	300	PHY management features, including enhanced link detection (except MI link detection)
MI link detection and configuration	250	1,200	500	MI link detection and configuration, including PHY user initialization (4 ports)
SyncManager	300	950	450	per SyncManager, including SM features
FMMU	350	700	250	per FMMU
DPRAM	0	200	100	60 KB
Distributed clocks	400	550	300	SyncManager event times and DC speed counter direct control
				32 bit distributed clocks
	900	1,250	600	System time
	650	950	400	SyncSignal 0
	400	750	350	Each additional SyncSignal 1-3
	350	350	100	LatchSignal 0
	300	400	150	Each additional LatchSignal 1-3
				64 bit distributed clocks
	1,150	1,650	750	System time
	1,150	1,900	800	SyncSignal 0
	600	1,200	500	Each additional SyncSignal 1-3
	600	850	250	LatchSignal 0
	550	750	250	Each additional LatchSignal 1-3
	350	700	250	Core details: read/write offset, write protection, reset slave, all LED functions, PDI basics: all PDI processing features except "only PDI interrupt acknowledge by write access"
	350	700	250	Core details: read/write offset, write protection, reset slave, all LED functions, PDI basics: all PDI processing features except "only PDI interrupt acknowledge by write access"
GPIO	300	450	200	8 byte
Second PDI enabled	150	300	150	
PDI (each)				
32 bit digital I/O	400	650	350	
SPI slave	750	1,500	650	
8 bit µController	450	1,000	500	
16 bit µController	600	1,350	650	
32 bit µController	700	1,450	650	
Avalon	450	1,050	550	25 MHz, 32 bit, synchronous
AXI4-Lite	600	1,350	650	25 MHz, 32 bit, synchronous
AXI4	1,000	2,300	950	25 MHz, 32 bit, synchronous

The EtherCAT IP core resource consumption figures for typical EtherCAT devices are based on EtherCAT IP core for Altera FPGAs Version 4.0.0, Altera Quartus Prime Standard 23.1, and Altera Cyclone IV (LE) or Cyclone V (ALM) devices.

Table 13: EtherCAT IP core configuration for typical EtherCAT devices

EtherCAT device	Ports	SM	FMMU	DPRAM [kByte]	PDI	DC	Reg.	LC	ALM
Digital IO	2	2	2	1	32 Bit digital I/O	-	5,000	9,000	3,700
Processor based typical	2	4	3	4	AXI4-Lite	32 bit, 1x Sync, 1x Latch	7,900	15,400	6,600
Processor based large	2	4	4	8	AXI4	64 bit, 2x Sync, 2x Latch	10,700	20,700	8,500
Minimum	1	0	0	0	-	-	2,300	3,700	1,500
Maximum	4	16	16	60	AXI4	64 bit, 4x Sync, 4x Latch	24,000	51,000	21,500

6 IP core signals

The available signals depend on the IP core configuration.

6.1 General signals

Table 14: General signals

Condition	Name	Direction	Description
	NRESET	INPUT	Resets all registers of the IP core, active low
Reset slave by ECAT/PDI	RESET_OUT	OUTPUT	Reset by ECAT (reset register 0x0040), active high. RESET_OUT has to trigger nRESET, which clears RESET_OUT.
	CLK25	INPUT	25 MHz clock signal from PLL (rising edge synchronous with rising edge of CLK100)
Internal PDI CLK = 50 MHz or Port0 = RMII	CLK50	INPUT	50 MHz reference clock signal from PLL (rising edge synchronous with rising edge of CLK100), for RMII also connected to PHY
	CLK100	INPUT	100 MHz clock signal from PLL
Port0 = RGMII	CLK25_2NS	INPUT	25 MHz clock signal from PLL (rising edge 2 ns after rising edge of CLK25), used for RGMII GTX_CLK
PDI0=Avalon or PDI0=AXI4 or PDI0=AXI4-Lite	CLK_PDI_EXT	INPUT	Avalon/AXI bus clock signal for PDI0 from PLL (either multiple of CLK25 and rising edge synchronous with rising edge of CLK25, or asynchronous to CLK25)
PDI1=Avalon or PDI1=AXI4 or PDI1=AXI4-Lite	CLK_PDI1_EXT	INPUT	Avalon/AXI bus clock signal for PDI1 from PLL (either multiple of CLK25 and rising edge synchronous with rising edge of CLK25, or asynchronous to CLK25)

6.1.1 Clock source example schematics

Typically, the clock inputs of the EtherCAT IP core (CLK25, CLK100, and optionally CLK50 and CLK25_2NS) are sourced by a PLL inside the FPGA.

The EtherCAT IP core and the Ethernet PHYs have to share the same clock source for MII and RMII. The PLL has to use a configuration which guarantees a fixed phase relation between clock input and clock outputs, in order to enable TX shift compensation for the MII TX signals.

The initial accuracy of the EtherCAT IP clock source has to be 25ppm or better (at room temperature).

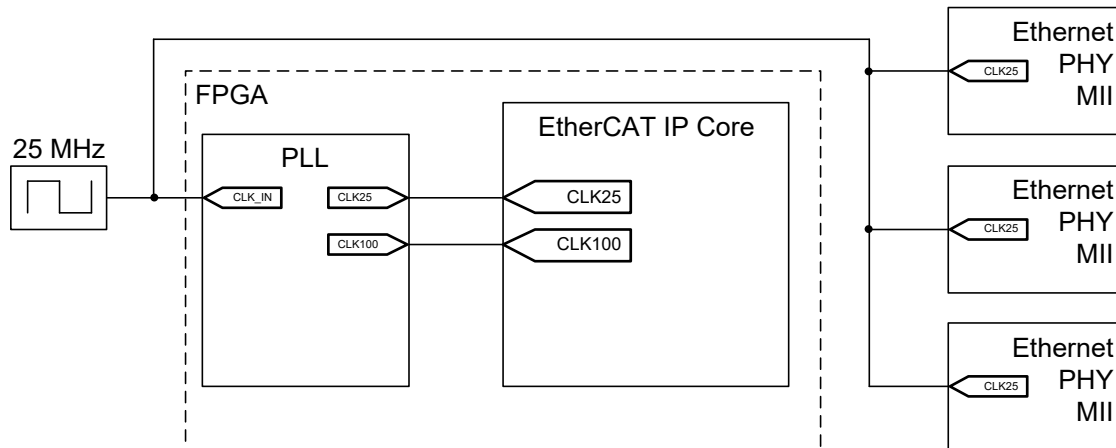


Figure 19: EtherCAT IP core clock source (MII)

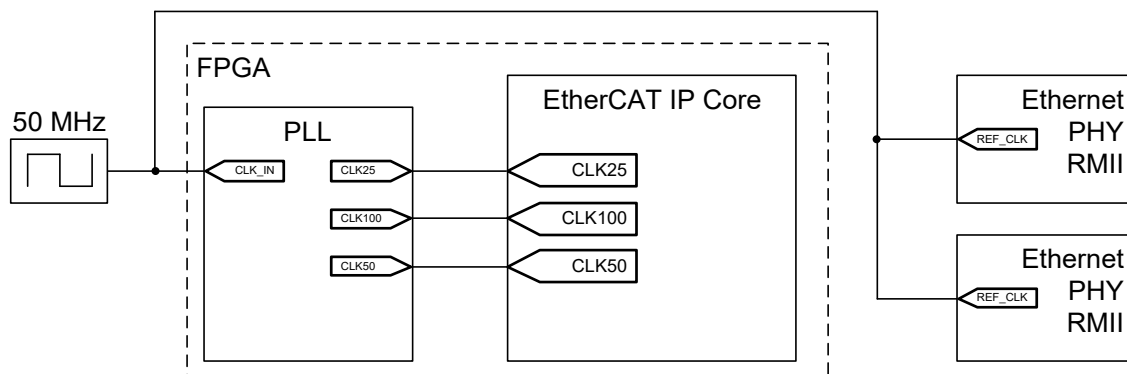


Figure 20: EtherCAT IP core clock source (RMII)

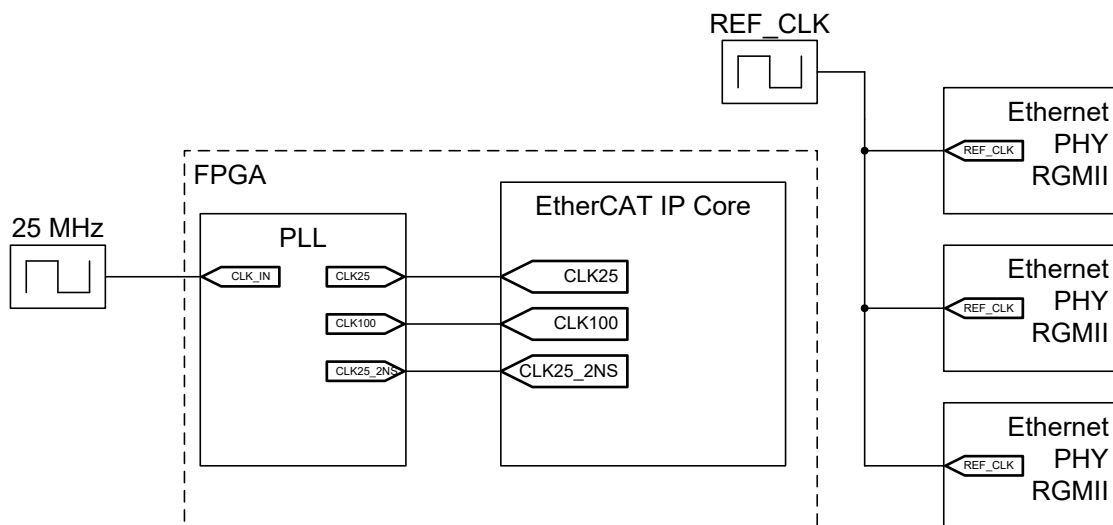


Figure 21: EtherCAT IP core clock source (RGMII)

6.2 SII EEPROM interface signals

Table 15: SII EEPROM signals

Condition	Name	Direction	Description
EEPROM emulation disabled	PROM_SIZE	INPUT	Sets EEPROM size 0: up to 16 kbit EEPROM 1: 32 kbit-4Mbit EEPROM
	PROM_CLK_IN	INPUT	EEPROM I ² C clock input (unused)
	PROM_CLK_OUT	OUTPUT	EEPROM I ² C clock output driver value (always 0)
	PROM_CLK_ENA	OUTPUT	EEPROM I ² C clock output driver enable (act. high)
	PROM_DATA_IN	INPUT	EEPROM I ² C data input
	PROM_DATA_OUT	OUTPUT	EEPROM I ² C data output driver value (always 0)
	PROM_DATA_ENA	OUTPUT	EEPROM I ² C data output driver enable (act. high)
	PROM_LOADED	OUTPUT	0: EEPROM is not loaded 1: EEPROM is loaded
PDI1 enable	PROM_LOADED_PDI1	OUTPUT	0: EEPROM is not loaded 1: EEPROM is loaded

6.3 LED signals

Table 16 lists the signals used for the LEDs. The LED signals are active high. Link/activity and RUN LEDs shall be green, ERR LED shall be red. The STATUS LED shall be green and red.

Table 16: LED signals

Condition	Name	Direction	Description
Port0 enabled	LED_LINK_ACT[0]	OUTPUT	Link/activity LED for ethernet port 0 (act. high)
Port1 enabled	LED_LINK_ACT[1]	OUTPUT	Link/activity LED for ethernet port 1 (act. high)
Port2 enabled	LED_LINK_ACT[2]	OUTPUT	Link/activity LED for Ethernet port 2 (act. high)
Port3 enabled	LED_LINK_ACT[3]	OUTPUT	Link/activity LED for Ethernet port 3 (act. high)
RUN LED enabled	LED_RUN	OUTPUT	RUN LED for device status (act. high)
Extended RUN/ERR LED enabled	LED_ERR	OUTPUT	ERR LED for device status (act. high)
	LED_STATE_RUN	OUTPUT	Connect to RUN pin of dual-color STATE LED (act. high), connect LED_ERR to ERR pin of STATUS LED

NOTE: The application ERR LED and STATE LED can alternatively be controlled by a µController if required.

6.4 Distributed clocks SyncSignals/LatchSignals

Table 17 lists the signals used with distributed clocks. The DC_SYNC[3:0] signals are active high.

Table 17: DC SyncSignals/LatchSignals

Condition	Name	Direction	Description
Distributed clocks and SYNC0 enabled	DC_SYNC[0]	OUTPUT	DC SyncSignal output 0
Distributed clocks and SYNC1 enabled	DC_SYNC[1]	OUTPUT	DC SyncSignal output 1
Distributed clocks and SYNC2 enabled	DC_SYNC[2]	OUTPUT	DC SyncSignal output 2
Distributed clocks and SYNC3 enabled	DC_SYNC[3]	OUTPUT	DC SyncSignal output 3
Distributed clocks and Latch0 enabled	DC_LATCH[0]	INPUT	DC LatchSignal input 0
Distributed clocks and Latch1 enabled	DC_LATCH[1]	INPUT	DC LatchSignal input 1
Distributed clocks and Latch2 enabled	DC_LATCH[2]	INPUT	DC LatchSignal input 2
Distributed clocks and Latch3 enabled	DC_LATCH[3]	INPUT	DC LatchSignal input 3

6.5 Physical layer interface

The IP core is connected with Ethernet PHYs using MII/RMII/RGMII interfaces.

Table 18 lists the general PHY interface signals.

Table 18: Physical layer general

Condition	Name	Direction	Description
PHY management interface enabled and Export PHY address as signals and not(independent PHY addresses)	PHY_OFFSET_VEC[4:0]	INPUT	PHY address offset
PHY management interface enabled and Export PHY address as signals and independent PHY addresses	PHY_ADR_PORT0[4:0]	INPUT	PHY address port 0
PHY management interface enabled and Export PHY address as signals and independent PHY addresses and port 1	PHY_ADR_PORT1[4:0]	INPUT	PHY address port 1
PHY management interface enabled and Export PHY address as signals and independent PHY addresses and port 2	PHY_ADR_PORT2[4:0]	INPUT	PHY address port 2
PHY management interface enabled and Export PHY address as signals and independent PHY addresses and port 3	PHY_ADR_PORT3[4:0]	INPUT	PHY address port 3
Port0 enabled	nPHY_RESET_OUT0	OUTPUT	PHY reset port 0 (act. low)
Port1 enabled	nPHY_RESET_OUT1	OUTPUT	PHY reset port 1 (act. low)
Port2 enabled	nPHY_RESET_OUT2	OUTPUT	PHY reset port 2 (act. low)
Port3 enabled	nPHY_RESET_OUT3	OUTPUT	PHY reset port 3 (act. low)
User initialization of PHY registers	PHY_INIT_NRESET	OUTPUT	PHY init ROM access reset (act. low)
	PHY_INIT_CLK	OUTPUT	PHY init ROM access clock
	PHY_INIT_ENA	OUTPUT	PHY init ROM enable (act. high)
	PHY_INIT_ADR[7:0]	OUTPUT	PHY init read address
	PHY_INIT_DATA[23:0]	INPUT	PHY init read data

Condition	Name	Direction	Description
PHY management interface enabled, not separate for each port	MCLK	OUTPUT	PHY management clock
	MDIO_DATA_IN	INPUT	PHY management data input
	MDIO_DATA_OUT	OUTPUT	PHY management data output value
	MDIO_DATA_ENA	OUTPUT	PHY management data output driver enable (act. high)
PHY management interface enabled, separate for each port	MCLK0	OUTPUT	PHY management clock port 0
	MDIO0_DATA_IN	INPUT	PHY management data port 0 input
	MDIO0_DATA_OUT	OUTPUT	PHY management data port 0 output value
	MDIO0_DATA_ENA	OUTPUT	PHY management data port 0 output driver enable (act. high)
PHY management interface enabled, separate for each port and port 1	MCLK1	OUTPUT	PHY management clock port 1
	MDIO1_DATA_IN	INPUT	PHY management data port 1 input
	MDIO1_DATA_OUT	OUTPUT	PHY management data port 1 output value
	MDIO1_DATA_ENA	OUTPUT	PHY management data port 1 output driver enable (act. high)
PHY management interface enabled, separate for each port and port 2	MCLK2	OUTPUT	PHY management clock port 2
	MDIO2_DATA_IN	INPUT	PHY management data port 2 input
	MDIO2_DATA_OUT	OUTPUT	PHY management data port 2 output value
	MDIO2_DATA_ENA	OUTPUT	PHY management data port 2 output driver enable (act. high)
PHY management interface enabled, separate for each port and port 3	MCLK3	OUTPUT	PHY management clock port 3
	MDIO3_DATA_IN	INPUT	PHY management data port 3 input
	MDIO3_DATA_OUT	OUTPUT	PHY management data port 3 output value
	MDIO3_DATA_ENA	OUTPUT	PHY management data port 3 output driver enable (act. high)

NOTE: MDIO, MDIO0-MDIO3 must have a pull-up resistor (4.7kΩ recommended for ESCs).

6.5.1 MII interface

Table 19 lists the signals used with MII. It is recommended to use automatic TX shift compensation by connecting MII_TX_CLK.

Table 19: PHY interface MII

Condition	Name	Direction	Description
Port0 = MII	NMII_LINK0	INPUT	0: 100 Mbit/s (Full Duplex) link at port 0 1: no link at port 0
	MII_RX_CLK0	INPUT	Receive clock port 0
	MII_RX_DV0	INPUT	Receive data valid port 0
	MII_RX_DATA0[3:0]	INPUT	Receive data port 0
	MII_RX_ERR0	INPUT	Receive error port 0
	MII_TX_ENA0	OUTPUT	Transmit enable port 0
	MII_TX_DATA0[3:0]	OUTPUT	Transmit data port 0
	MII_TX_CLK0	INPUT	Transmit clock port 0 for automatic TX shift configuration. Set to 0 for manual TX shift configuration.
Port1 = MII	MII_TX_SHIFT0[1:0]	INPUT	Manual TX shift configuration port 0, set to 0 for automatic TX shift compensation. Additional TX signal delay: 00: 0 ns 01: 10 ns 10: 20 ns 11: 30 ns
	NMII_LINK1	INPUT	0: 100 Mbit/s (Full Duplex) link at port 1 1: no link at port 1
	MII_RX_CLK1	INPUT	Receive clock port 1
	MII_RX_DV1	INPUT	Receive data valid port 1
	MII_RX_DATA1[3:0]	INPUT	Receive data port 1
	MII_RX_ERR1	INPUT	Receive error port 1
	MII_TX_ENA1	OUTPUT	Transmit enable port 1
	MII_TX_DATA1[3:0]	OUTPUT	Transmit data port 1
	MII_TX_CLK1	INPUT	Transmit clock port 1 for automatic TX shift configuration. Set to 0 for manual TX shift configuration.
	MII_TX_SHIFT1[1:0]	INPUT	Manual TX shift configuration port 0, set to 0 for automatic TX shift compensation. Additional TX signal delay: 00: 0 ns 01: 10 ns 10: 20 ns 11: 30 ns

Condition	Name	Direction	Description
Port2 = MII	NMII_LINK2	INPUT	0: 100 Mbit/s (Full Duplex) link at port 2 1: no link at port 2
	MII_RX_CLK2	INPUT	Receive clock port 2
	MII_RX_DV2	INPUT	Receive data valid port 2
	MII_RX_DATA2[3:0]	INPUT	Receive data port 2
	MII_RX_ERR2	INPUT	Receive error port 2
	MII_TX_ENA2	OUTPUT	Transmit enable port 2
	MII_TX_DATA2[3:0]	OUTPUT	Transmit data port 2
	MII_TX_CLK2	INPUT	Transmit clock port 2 for automatic TX shift configuration. Set to 0 for manual TX shift configuration.
	MII_TX_SHIFT2[1:0]	INPUT	Manual TX shift configuration port 0, set to 0 for automatic TX shift compensation. Additional TX signal delay: 00: 0 ns 01: 10 ns 10: 20 ns 11: 30 ns
Port3 = MII	NMII_LINK3	INPUT	0: 100 Mbit/s (Full Duplex) link at port 3 1: no link at port 3
	MII_RX_CLK3	INPUT	Receive clock port 3
	MII_RX_DV3	INPUT	Receive data valid port 3
	MII_RX_DATA3[3:0]	INPUT	Receive data port 3
	MII_RX_ERR3	INPUT	Receive error port 3
	MII_TX_ENA3	OUTPUT	Transmit enable port 3
	MII_TX_DATA3[3:0]	OUTPUT	Transmit data port 3
	MII_TX_CLK3	INPUT	Transmit clock port 3 for automatic TX shift configuration. Set to 0 for manual TX shift configuration.
	MII_TX_SHIFT3[1:0]	INPUT	Manual TX shift configuration port 0, set to 0 for automatic TX shift compensation. Additional TX signal delay: 00: 0 ns 01: 10 ns 10: 20 ns 11: 30 ns

6.5.2 RMII interface

Table 20 lists the signals used with RMII.

Table 20: PHY interface RMII

Condition	Name	Direction	Description
Port0 = RMII	NRMII_LINK0	INPUT	0: 100 Mbit/s (Full Duplex) link at port 0 1: no link at port 0
	RMII_RX_DV0	INPUT	Carrier sense/receive data valid port 0
	RMII_RX_DATA0[1:0]	INPUT	Receive data port 0
	RMII_RX_ERR0	INPUT	Receive error port 0
	RMII_TX_ENA0	OUTPUT	Transmit enable port 0
	RMII_TX_DATA0[1:0]	OUTPUT	Transmit data port 0
Port1 = RMII	NRMII_LINK1	INPUT	0: 100 Mbit/s (Full Duplex) link at port 1 1: no link at port 1
	RMII_RX_DV1	INPUT	Carrier sense/receive data valid port 1
	RMII_RX_DATA1[1:0]	INPUT	Receive data port 1
	RMII_RX_ERR1	INPUT	Receive error port 1
	RMII_TX_ENA1	OUTPUT	Transmit enable port 1
	RMII_TX_DATA1[1:0]	OUTPUT	Transmit data port 1
Port2 = RMII	NRMII_LINK2	INPUT	0: 100 Mbit/s (Full Duplex) link at port 2 1: no link at port 2
	RMII_RX_DV2	INPUT	Carrier sense/receive data valid port 2
	RMII_RX_DATA2[1:0]	INPUT	Receive data port 2
	RMII_RX_ERR2	INPUT	Receive error port 2
	RMII_TX_ENA2	OUTPUT	Transmit enable port 2
	RMII_TX_DATA2[1:0]	OUTPUT	Transmit data port 2
Port3 = RMII	NRMII_LINK3	INPUT	0: 100 Mbit/s (Full Duplex) link at port 3 1: no link at port 3
	RMII_RX_DV3	INPUT	Carrier sense/receive data valid port 3
	RMII_RX_DATA3[1:0]	INPUT	Receive data port 3
	RMII_RX_ERR3	INPUT	Receive error port 3
	RMII_TX_ENA3	OUTPUT	Transmit enable port 3
	RMII_TX_DATA3[1:0]	OUTPUT	Transmit data port 3

6.5.3 RGMII interface

Table 21 lists the signals used with RGMII.

Table 21: PHY interface RGMII

Condition	Name	Direction	Description
Port0 = RGMII	NRGMII_LINK0	INPUT	0: 100 Mbit/s (Full Duplex) link at port 0 1: no link at port 0
	RGMII_RX_CLK0	INPUT	Receive clock port 0
	RGMII_RX_CTL_DATA_DDR_CLK0	OUTPUT	Receive control/data DDR input clock port 0
	RGMII_RX_CTL_DATA_DDR_NRESET0	OUTPUT	Receive control/data DDR input reset (act. low) port 0
	RGMII_RX_CTL_DDR_L0	INPUT	Receive control DDR input low port 0
	RGMII_RX_CTL_DDR_H0	INPUT	Receive control DDR input high port 0
	RGMII_RX_DATA_DDR_L0	INPUT	Receive data DDR input low port 0
	RGMII_RX_DATA_DDR_H0	INPUT	Receive data DDR input high port 0
	RGMII_TX_CLK_DDR_CLK0	OUTPUT	Transmit clock DDR output clock port 0
	RGMII_TX_CLK_DDR_NRESET0	OUTPUT	Transmit clock DDR output reset (port 0, act. low)
	RGMII_TX_CLK_DDR_L0	OUTPUT	Transmit clock DDR output low port 0
	RGMII_TX_CLK_DDR_H0	OUTPUT	Transmit clock DDR output high port 0
	RGMII_TX_CTL_DATA_DDR_CLK0	OUTPUT	Transmit control/data DDR output clock port 0
	RGMII_TX_CTL_DATA_DDR_NRESET0	OUTPUT	Transmit control/data DDR output reset (port 0, act. low)
	RGMII_TX_CTL_DDR_L0	OUTPUT	Transmit control DDR output low port 0
	RGMII_TX_CTL_DDR_H0	OUTPUT	Transmit control DDR output high port 0
	RGMII_TX_DATA_DDR_L0	OUTPUT	Transmit data DDR output low port 0

Condition	Name	Direction	Description
Port1 = RGMII	NRGMII_LINK1	INPUT	0: 100 Mbit/s (Full Duplex) link at port 1 1: no link at port 1
	RGMII_RX_CLK1	INPUT	Receive clock port 1
	RGMII_RX_CTL_DATA_DDR_CLK1	OUTPUT	Receive control/data DDR input clock port 1
	RGMII_RX_CTL_DATA_DDR_NRESET1	OUTPUT	Receive control/data DDR input reset (port 1, act. low)
	RGMII_RX_CTL_DDR_L1	INPUT	Receive control DDR input low port 1
	RGMII_RX_CTL_DDR_H1	INPUT	Receive control DDR input high port 1
	RGMII_RX_DATA_DDR_L1	INPUT	Receive data DDR input low port 1
	RGMII_RX_DATA_DDR_H1	INPUT	Receive data DDR input high port 1
	RGMII_TX_CLK_DDR_CLK1	OUTPUT	Transmit clock DDR output clock port 1
	RGMII_TX_CLK_DDR_NRESET1	OUTPUT	Transmit clock DDR output reset (port 1, act. low)
	RGMII_TX_CLK_DDR_L1	OUTPUT	Transmit clock DDR output low port 1
	RGMII_TX_CLK_DDR_H1	OUTPUT	Transmit clock DDR output high port 1
	RGMII_TX_CTL_DATA_DDR_CLK1	OUTPUT	Transmit control/data DDR output clock port 1
	RGMII_TX_CTL_DATA_DDR_NRESET1	OUTPUT	Transmit control/data DDR output reset (port 1, act. low)
	RGMII_TX_CTL_DDR_L1	OUTPUT	Transmit control DDR output low port 1
	RGMII_TX_CTL_DDR_H1	OUTPUT	Transmit control DDR output high port 1
	RGMII_TX_DATA_DDR_L1	OUTPUT	Transmit data DDR output low port 1

Condition	Name	Direction	Description
Port2 = RGMII	NRGMII_LINK2	INPUT	0: 100 Mbit/s (Full Duplex) link at port 2 1: no link at port 2
	RGMII_RX_CLK2	INPUT	Receive clock port 2
	RGMII_RX_CTL_DATA_DDR_CLK2	OUTPUT	Receive control/data DDR input clock port 2
	RGMII_RX_CTL_DATA_DDR_NRESET2	OUTPUT	Receive control/data DDR input reset (port 2, act. low)
	RGMII_RX_CTL_DDR_L2	INPUT	Receive control DDR input low port 2
	RGMII_RX_CTL_DDR_H2	INPUT	Receive control DDR input high port 2
	RGMII_RX_DATA_DDR_L2	INPUT	Receive data DDR input low port 2
	RGMII_RX_DATA_DDR_H2	INPUT	Receive data DDR input high port 2
	RGMII_TX_CLK_DDR_CLK2	OUTPUT	Transmit clock DDR output clock port 2
	RGMII_TX_CLK_DDR_NRESET2	OUTPUT	Transmit clock DDR output reset (port 2, act. low)
	RGMII_TX_CLK_DDR_L2	OUTPUT	Transmit clock DDR output low port 2
	RGMII_TX_CLK_DDR_H2	OUTPUT	Transmit clock DDR output high port 2
	RGMII_TX_CTL_DATA_DDR_CLK2	OUTPUT	Transmit control/data DDR output clock port 2
	RGMII_TX_CTL_DATA_DDR_NRESET2	OUTPUT	Transmit control/data DDR output reset (port 2, act. low)
	RGMII_TX_CTL_DDR_L2	OUTPUT	Transmit control DDR output low port 2
	RGMII_TX_CTL_DDR_H2	OUTPUT	Transmit control DDR output high port 2
	RGMII_TX_DATA_DDR_L2	OUTPUT	Transmit data DDR output low port 2

Condition	Name	Direction	Description
Port3 = RGMII	NRGMII_LINK3	INPUT	0: 100 Mbit/s (Full Duplex) link at port 3 1: no link at port 3
	RGMII_RX_CLK3	INPUT	Receive clock port 3
	RGMII_RX_CTL_DATA_DDR_CLK3	OUTPUT	Receive control/data DDR input clock port 3
	RGMII_RX_CTL_DATA_DDR_NRESET3	OUTPUT	Receive control/data DDR input reset (port 3, act. low)
	RGMII_RX_CTL_DDR_L3	INPUT	Receive control DDR input low port 3
	RGMII_RX_CTL_DDR_H3	INPUT	Receive control DDR input high port 3
	RGMII_RX_DATA_DDR_L3	INPUT	Receive data DDR input low port 3
	RGMII_RX_DATA_DDR_H3	INPUT	Receive data DDR input high port 3
	RGMII_TX_CLK_DDR_CLK3	OUTPUT	Transmit clock DDR output clock port 3
	RGMII_TX_CLK_DDR_NRESET3	OUTPUT	Transmit clock DDR output reset (port 3, act. low)
	RGMII_TX_CLK_DDR_L3	OUTPUT	Transmit clock DDR output low port 3
	RGMII_TX_CLK_DDR_H3	OUTPUT	Transmit clock DDR output high port 3
	RGMII_TX_CTL_DATA_DDR_CLK3	OUTPUT	Transmit control/data DDR output clock port 3
	RGMII_TX_CTL_DATA_DDR_NRESET3	OUTPUT	Transmit control/data DDR output reset (port 3, act. low)
	RGMII_TX_CTL_DDR_L3	OUTPUT	Transmit control DDR output low port 3
	RGMII_TX_CTL_DDR_H3	OUTPUT	Transmit control DDR output high port 3
	RGMII_TX_DATA_DDR_L3	OUTPUT	Transmit data DDR output low port 3

6.6 PDI signals

6.6.1 General PDI signals

Table 23 lists the signals available independent of the PDI configuration.

Table 22: General PDI signals

Condition	Name	Direction	Description
	PDI_SOF	OUTPUT	Ethernet Start-of-frame (act. high)
	PDI_EOF	OUTPUT	Ethernet End-of-frame (act. high)
	PDI_WD_TRIGGER	OUTPUT	Process data watchdog trigger (act. high)
	PDI_WD_STATE	OUTPUT	Process data watchdog state 0: Expired 1: Not expired
GPIO bytes > 0	PDI_GPI[8*bytes-1:0]	INPUT	General purpose inputs (width configurable, 1/2/4/8 bytes)
GPIO bytes > 0	PDI_GPO[8*bytes-1:0]	OUTPUT	General purpose outputs (width N:0 configurable, 1/2/4/8 bytes)

6.6.2 Digital I/O interface

These are the signals used for the digital I/O PDI.

Table 23: Digital I/O PDI0

Condition	Name	Direction	Description
Byte 0 is output	PDI_DIGI_DATA_OUT0 [7:0]	OUTPUT	Digital output byte 0
Byte 0 is input	PDI_DIGI_DATA_IN0 [7:0]	INPUT	Digital input byte 0
Byte 1 is output	PDI_DIGI_DATA_OUT1 [7:0]	OUTPUT	Digital output byte 1
Byte 1 is input	PDI_DIGI_DATA_IN1 [7:0]	INPUT	Digital input byte 1
Byte 2 is output	PDI_DIGI_DATA_OUT2 [7:0]	OUTPUT	Digital output byte 2
Byte 2 is input	PDI_DIGI_DATA_IN2 [7:0]	INPUT	Digital input byte 2
Byte 3 is output	PDI_DIGI_DATA_OUT3 [7:0]	OUTPUT	Digital output byte 3
Byte 3 is input	PDI_DIGI_DATA_IN3 [7:0]	INPUT	Digital input byte 3
If both, digital input and output selected	PDI_DIGI_DATA_ENA	OUTPUT	Digital output enable
any digital input selected and input mode=LATCH_IN signal	PDI_DIGI_LATCH_IN	INPUT	Latch digital input (act. high)
any digital output selected and output mode=OUT_START signal	PDI_DIGI_OUT_START	INPUT	Update digital output (act. high)
any digital output selected	PDI_DIGI_OE_EXT	INPUT	External output enable (act. high)
	PDI_DIGI_OUTVALID	OUTPUT	Output event: output valid (act. high)

Table 24: Digital I/O PDI1

Condition	Name	Direction	Description
Byte 0 is output	PDI1_DIGI_DATA_OUT0 [7:0]	OUTPUT	Digital output byte 0
Byte 0 is input	PDI1_DIGI_DATA_IN0 [7:0]	INPUT	Digital input byte 0
Byte 1 is output	PDI1_DIGI_DATA_OUT1[7:0]	OUTPUT	Digital output byte 1
Byte 1 is input	PDI1_DIGI_DATA_IN1[7:0]	INPUT	Digital input byte 1
Byte 2 is output	PDI1_DIGI_DATA_OUT2[7:0]	OUTPUT	Digital output byte 2
Byte 2 is input	PDI1_DIGI_DATA_IN2[7:0]	INPUT	Digital input byte 2
Byte 3 is output	PDI1_DIGI_DATA_OUT3 [7:0]	OUTPUT	Digital output byte 3
Byte 3 is input	PDI1_DIGI_DATA_IN3[7:0]	INPUT	Digital input byte 3
If both, digital input and output selected	PDI1_DIGI_DATA_ENA	OUTPUT	Digital output enable
any digital input selected and input mode=LATCH_IN signal	PDI1_DIGI_LATCH_IN	INPUT	Latch digital input (act. high)
any digital output selected and output mode=OUT_START signal	PDI1_DIGI_OUT_START	INPUT	Update digital output (act. high)
any digital output selected	PDI1_DIGI_OE_EXT	INPUT	External output enable (act. high)
	PDI1_DIGI_OUTVALID	OUTPUT	Output event: output valid (act. high)

6.6.3 SPI slave interface

These are the signals used for an SPI slave PDI.

Table 25: SPI slave PDI0

Condition	Name	Direction	Description
	PDI_SPI_CLK	INPUT	SPI clock
	PDI_SPI_SEL	INPUT	SPI select
	PDI_SPI_IRQ_OUT	OUTPUT	SPI interrupt
	PDI_SPI_IRQ_ENA	OUTPUT	SPI interrupt
	PDI_SPI_DO_OUT	OUTPUT	SPI slave data out: IP core → μ C
	PDI_SPI_DO_ENA	OUTPUT	0: disable output driver for PDI_SPI_DO_OUT 1: enable output driver for PDI_SPI_DO_OUT
Unidirectional	PDI_SPI_MOSI[7:0]	INPUT	MOSI input value, 1/2/4/8 bit used
	PDI_SPI_MISO_OUT[7:0]	OUTPUT	MISO output driver value, 1/2/4/8 bit used
	PDI_SPI_MISO_ENA[7:0]	OUTPUT	MISO output driver enable (act. high), 1/2/4/8 bit used
Bidirectional	PDI_SPI_DATA[7:0]	INPUT	Data input value, 1/2/4/8 bit used
	PDI_SPI_DATA_OUT[7:0]	OUTPUT	Data output driver value, 1/2/4/8 bit used
	PDI_SPI_DATA_ENA[7:0]	OUTPUT	Data output driver enable (act. high), 1/2/4/8 bit used
Additional SEL_MISO/ SEL_MOSI signals	PDI_SPI_SEL_MISO	INPUT	SPI slave MISO select
	PDI_SPI_SEL_MOSI	INPUT	SPI slave MOSI select

Table 26: SPI slave PDI1

Condition	Name	Direction	Description
	PDI1_SPI_CLK	INPUT	SPI clock
	PDI1_SPI_SEL	INPUT	SPI select
	PDI1_SPI_IRQ_OUT	OUTPUT	SPI interrupt
	PDI1_SPI_IRQ_ENA	OUTPUT	SPI interrupt
	PDI1_SPI_DO_OUT	OUTPUT	SPI slave data out: IP core → μ C
	PDI1_SPI_DO_ENA	OUTPUT	0: disable output driver for PDI_SPI_DO_OUT 1: enable output driver for PDI_SPI_DO_OUT
Unidirectional	PDI1_SPI_MOSI[7:0]	INPUT	MOSI input value, 1/2/4/8 bit used
	PDI1_SPI_MISO_OUT[7:0]	OUTPUT	MISO output driver value, 1/2/4/8 bit used
	PDI1_SPI_MISO_ENA[7:0]	OUTPUT	MISO output driver enable (act. high), 1/2/4/8 bit used
Bidirectional	PDI1_SPI_DATA[7:0]	INPUT	Data input value, 1/2/4/8 bit used
	PDI1_SPI_DATA_OUT[7:0]	OUTPUT	Data output driver value, 1/2/4/8 bit used
	PDI1_SPI_DATA_ENA[7:0]	OUTPUT	Data output driver enable (act. high), 1/2/4/8 bit used
Additional SEL_MISO/ SEL_MOSI signals	PDI1_SPI_SEL_MISO	INPUT	SPI slave MISO select
	PDI1_SPI_SEL_MISO	INPUT	SPI slave MOSI select

6.6.4 Asynchronous 8/16/32 bit μ Controller interface

These are the signals used for the asynchronous μ Controller PDI. This interface is only available for PDI0.

Table 27: 8/16/32 Bit μ C PDI0

Condition	Name	Direction	Description
	PDI_UC_ADR[15:0]	INPUT	μ C address bus
	PDI_UC_NRD	INPUT	μ C read access (act. low)
	PDI_UC_NWR	INPUT	μ C write access (act. low)
	PDI_UC_NCS	INPUT	μ C chip select (act. low)
	PDI_UC_IRQ_OUT	OUTPUT	Interrupt output driver value
	PDI_UC_IRQ_ENA	OUTPUT	Interrupt output driver enable (act. high)
	PDI_UC_BUSY_OUT	OUTPUT	PDI busy output driver value
	PDI_UC_BUSY_ENA	OUTPUT	PDI busy output driver enable (act. high)
	PDI_UC_DATA_IN[31:0]	INPUT	μ C data bus input value (8/16/32 bit used)
	PDI_UC_DATA_OUT[31:0]	OUTPUT	μ C data bus output driver value (8/16/32 bit used)
	PDI_UC_DATA_ENA	OUTPUT	μ C data bus output driver enable (act. high)
16/32 bit	PDI_UC_BYTE_ENA	INPUT	μ C byte high enable (act. high)

6.6.5 Avalon on-chip bus

These are the signals used for the Avalon PDI.

PDI_EXT_BUS_WIDTH is the external data bus width of PDI0.

Table 28: Avalon PDI0

Condition	Name	Direction	Description
	PDI_AVALON_ADR [18-ld(PDI_EXT_BUS_WIDTH):0]	INPUT	Avalon address
	PDI_AVALON_BE [PDI_EXT_BUS_WIDTH/8 -1:0]	INPUT	Avalon byte enable
	PDI_AVALON_RD_DATA [PDI_EXT_BUS_WIDTH -1:0]	OUTPUT	Avalon slave read data
	PDI_AVALON_WR_DATA [PDI_EXT_BUS_WIDTH:0]	INPUT	Avalon write data
	PDI_AVALON_READ	INPUT	Avalon read access
	PDI_AVALON_WRITE	INPUT	Avalon write access
	PDI_AVALON_CS	INPUT	Avalon chip select
	PDI_AVALON_BUSY	OUTPUT	Avalon slave busy
	PDI_AVALON_IRQ	OUTPUT	Avalon slave main interrupt
	PDI_AVALON_SYNC0	OUTPUT	DC SyncSignal 0 interrupt output
Enable separate DC SyncSignal interrupts, and SyncSignal present	PDI_AVALON_SYNC1	OUTPUT	DC SyncSignal 1 interrupt output
	PDI_AVALON_SYNC2	OUTPUT	DC SyncSignal 2 interrupt output
	PDI_AVALON_SYNC3	OUTPUT	DC SyncSignal 3 interrupt output

NOTE: If the EtherCAT IP core is used inside the platform designer, PDI_AVALON_SYNC0-3 are declared as interrupt signals for the processor (PDI0_DC_SYNC_irq). Use DC_SYNC[3:0] signals for external use of the SyncSignals.

PDI1_EXT_BUS_WIDTH is the external data bus width of PDI1.

Table 29: Avalon PDI1

Condition	Name	Direction	Description
	PDI1_AVALON_ADR [18- Id(PDI1_EXT_BUS_WIDTH):0]	INPUT	Avalon address
	PDI1_AVALON_BE [PDI1_EXT_BUS_WIDTH/8 -1:0]	INPUT	Avalon byte enable
	PDI1_AVALON_RD_DATA [PDI1_EXT_BUS_WIDTH -1:0]	OUTPUT	Avalon slave read data
	PDI1_AVALON_WR_DATA [PDI1_EXT_BUS_WIDTH:0]	INPUT	Avalon write data
	PDI1_AVALON_READ	INPUT	Avalon read access
	PDI1_AVALON_WRITE	INPUT	Avalon write access
	PDI1_AVALON_CS	INPUT	Avalon chip select
	PDI1_AVALON_BUSY	OUTPUT	Avalon slave busy
	PDI1_AVALON_IRQ	OUTPUT	Avalon slave main interrupt
	PDI1_AVALON_SYNC0	OUTPUT	DC SyncSignal 0 interrupt output
Enable separate DC SyncSignal interrupts, and SyncSignal present	PDI1_AVALON_SYNC1	OUTPUT	DC SyncSignal 1 interrupt output
	PDI1_AVALON_SYNC2	OUTPUT	DC SyncSignal 2 interrupt output
	PDI1_AVALON_SYNC3	OUTPUT	DC SyncSignal 3 interrupt output

NOTE: If the EtherCAT IP core is used inside the platform designer, PDI1_AVALON_SYNC0-3 are declared as interrupt signals for the processor (PDI1_DC_SYNC_irq). Use DC_SYNC[3:0] signals for external use of the SyncSignals.

6.6.6 AXI4 / AXI4-Lite on-chip bus

These are the signals used for the AXI4 and AXI4-Lite PDI.

PDI_EXT_BUS_WIDTH is the external data bus width of PDI0.

PDI_BUS_ID_WIDTH is the ID width.

Table 30: AXI4 / AXI4-Lite PDI0

Condition	Name	Direction	Description
AXI4 or AXI4-Lite	PDI_AXI_AWADDR[15:0]	INPUT	Write address
	PDI_AXI_AWPROT[2:0]	INPUT	Write protection type
	PDI_AXI_AWVALID	INPUT	Write address valid
	PDI_AXI_AWREADY	OUTPUT	Write address ready
	PDI_AXI_WDATA [PDI_EXT_BUS_WIDTH-1:0]	INPUT	Write data
	PDI_AXI_WSTRB [PDI_EXT_BUS_WIDTH/8-1:0]	INPUT	Write data byte enable
	PDI_AXI_WVALID	INPUT	Write data valid
	PDI_AXI_WREADY	OUTPUT	Write data ready
	PDI_AXI_BRESP[1:0]	OUTPUT	Write response
	PDI_AXI_BVALID	OUTPUT	Write response valid
	PDI_AXI_BREADY	INPUT	Write response ready
	PDI_AXI_ARADDR[15:0]	INPUT	Read address
	PDI_AXI_ARPROT[2:0]	INPUT	Read protection type
	PDI_AXI_ARVALID	INPUT	Read address valid
	PDI_AXI_ARREADY	OUTPUT	Read address ready
	PDI_AXI_RDATA [PDI_EXT_BUS_WIDTH-1:0]	OUTPUT	Read data
	PDI_AXI_RRESP[1:0]	OUTPUT	Read response
	PDI_AXI_RVALID	OUTPUT	Read data valid
	PDI_AXI_RREADY	INPUT	Read data ready
	PDI_AXI_IRQ_MAIN	OUTPUT	Interrupt
Enable separate DC SyncSignal interrupts, and SyncSignal present	PDI_AXI_IRQ_DC_SYNC0	OUTPUT	DC SyncSignal 0 interrupt output
	PDI_AXI_IRQ_DC_SYNC1	OUTPUT	DC SyncSignal 1 interrupt output
	PDI_AXI_IRQ_DC_SYNC2	OUTPUT	DC SyncSignal 2 interrupt output
	PDI_AXI_IRQ_DC_SYNC3	OUTPUT	DC SyncSignal 3 interrupt output

Condition	Name	Direction	Description
AXI4	PDI_AXI_AWID [PDI_BUS_ID_WIDTH-1:0]	INPUT	Write address ID
	PDI_AXI_AWLEN[7:0]	INPUT	Write length
	PDI_AXI_AWSIZE[2:0]	INPUT	Write size
	PDI_AXI_AWBURST[1:0]	INPUT	Write burst type
	PDI_AXI_AWLOCK	INPUT	Write lock
	PDI_AXI_AWCACHE[3:0]	INPUT	Write cache type
	PDI_AXI_AWREGION[3:0]	INPUT	Write region identifier
	PDI_AXI_AWQOS[3:0]	INPUT	Write QoS identifier
	PDI_AXI_WLAST	INPUT	Write data last
	PDI_AXI_BID[PDI_BUS_ID_WIDTH-1:0]	OUTPUT	Write response ID
	PDI_AXI_ARID[PDI_BUS_ID_WIDTH-1:0]	INPUT	Read address ID
	PDI_AXI_ARLEN[7:0]	INPUT	Read length
	PDI_AXI_ARSIZE[2:0]	INPUT	Read size
	PDI_AXI_ARBURST[1:0]	INPUT	Read burst type
	PDI_AXI_ARLOCK	INPUT	Read lock
	PDI_AXI_ARCACHE[3:0]	INPUT	Read cache type
	PDI_AXI_ARREGION[3:0]	INPUT	Read region identifier
	PDI_AXI_ARQOS[3:0]	INPUT	Read QoS identifier
	PDI_AXI_RID [PDI_BUS_ID_WIDTH-1:0]	OUTPUT	Read data ID
	PDI_AXI_RLAST	OUTPUT	Read data last

PDI1_EXT_BUS_WIDTH is the external data bus width of PDI1.

PDI1_BUS_ID_WIDTH is the ID width.

Table 31: AXI4 / AXI4-Lite PDI1

Condition	Name	Direction	Description
AXI4 or AXI4-Lite	PDI1_AXI_AWADDR[15:0]	INPUT	Write address
	PDI1_AXI_AWPROT[2:0]	INPUT	Write protection type
	PDI1_AXI_AWVALID	INPUT	Write address valid
	PDI1_AXI_AWREADY	OUTPUT	Write address ready
	PDI1_AXI_WDATA [PDI1_EXT_BUS_WIDTH-1:0]	INPUT	Write data
	PDI1_AXI_WSTRB [PDI1_EXT_BUS_WIDTH/8-1:0]	INPUT	Write data byte enable
	PDI1_AXI_WVALID	INPUT	Write data valid
	PDI1_AXI_WREADY	OUTPUT	Write data ready
	PDI1_AXI_BRESP[1:0]	OUTPUT	Write response
	PDI1_AXI_BVALID	OUTPUT	Write response valid
	PDI1_AXI_BREADY	INPUT	Write response ready
	PDI1_AXI_ARADDR[15:0]	INPUT	Read address
	PDI1_AXI_ARPROT[2:0]	INPUT	Read protection type
	PDI1_AXI_ARVALID	INPUT	Read address valid
	PDI1_AXI_ARREADY	OUTPUT	Read address ready
	PDI1_AXI_RDATA [PDI1_EXT_BUS_WIDTH-1:0]	OUTPUT	Read data
	PDI1_AXI_RRESP[1:0]	OUTPUT	Read response
	PDI1_AXI_RVALID	OUTPUT	Read data valid
	PDI1_AXI_RREADY	INPUT	Read data ready
	PDI1_AXI_IRQ_MAIN	OUTPUT	Interrupt

Condition	Name	Direction	Description
Enable separate DC SyncSignal interrupts, and SyncSignal present	PDI1_AXI_IRQ_DC_SYNC0	OUTPUT	DC SyncSignal 0 interrupt output
	PDI1_AXI_IRQ_DC_SYNC1	OUTPUT	DC SyncSignal 1 interrupt output
	PDI1_AXI_IRQ_DC_SYNC2	OUTPUT	DC SyncSignal 2 interrupt output
	PDI1_AXI_IRQ_DC_SYNC3	OUTPUT	DC SyncSignal 3 interrupt output
AXI4	PDI1_AXI_AWID [PDI1_BUS_ID_WIDTH-1:0]	INPUT	Write address ID
	PDI1_AXI_AWLEN[7:0]	INPUT	Write length
	PDI1_AXI_AWSIZE[2:0]	INPUT	Write size
	PDI1_AXI_AWBURST[1:0]	INPUT	Write burst type
	PDI1_AXI_AWLOCK	INPUT	Write lock
	PDI1_AXI_AWCACHE[3:0]	INPUT	Write cache type
	PDI1_AXI_AWREGION[3:0]	INPUT	Write region identifier
	PDI1_AXI_AWQOS[3:0]	INPUT	Write QoS identifier
	PDI1_AXI_WLAST	INPUT	Write data last
	PDI1_AXI_BID [PDI1_BUS_ID_WIDTH-1:0]	OUTPUT	Write response ID
	PDI1_AXI_ARID [PDI1_BUS_ID_WIDTH-1:0]	INPUT	Read address ID
	PDI1_AXI_ARLEN[7:0]	INPUT	Read length
	PDI1_AXI_ARSIZE[2:0]	INPUT	Read size
	PDI1_AXI_ARBURST[1:0]	INPUT	Read burst type
	PDI1_AXI_ARLOCK	INPUT	Read lock
	PDI1_AXI_ARCACHE[3:0]	INPUT	Read cache type
	PDI1_AXI_ARREGION[3:0]	INPUT	Read region identifier
	PDI1_AXI_ARQOS[3:0]	INPUT	Read QoS identifier
	PDI1_AXI_RID [PDI1_BUS_ID_WIDTH-1:0]	OUTPUT	Read data ID
	PDI1_AXI_RLAST	OUTPUT	Read data last

7 Ethernet interface

The IP core is connected with Ethernet PHYs using MII, RMII, or RGMII interfaces. MII is recommended since the PHY delay (and delay jitter) is smaller in comparison to RMII and RGMII.

7.1 PHY management interface

7.1.1 PHY management interface signals

The PHY management interface of the IP core has the following signals:

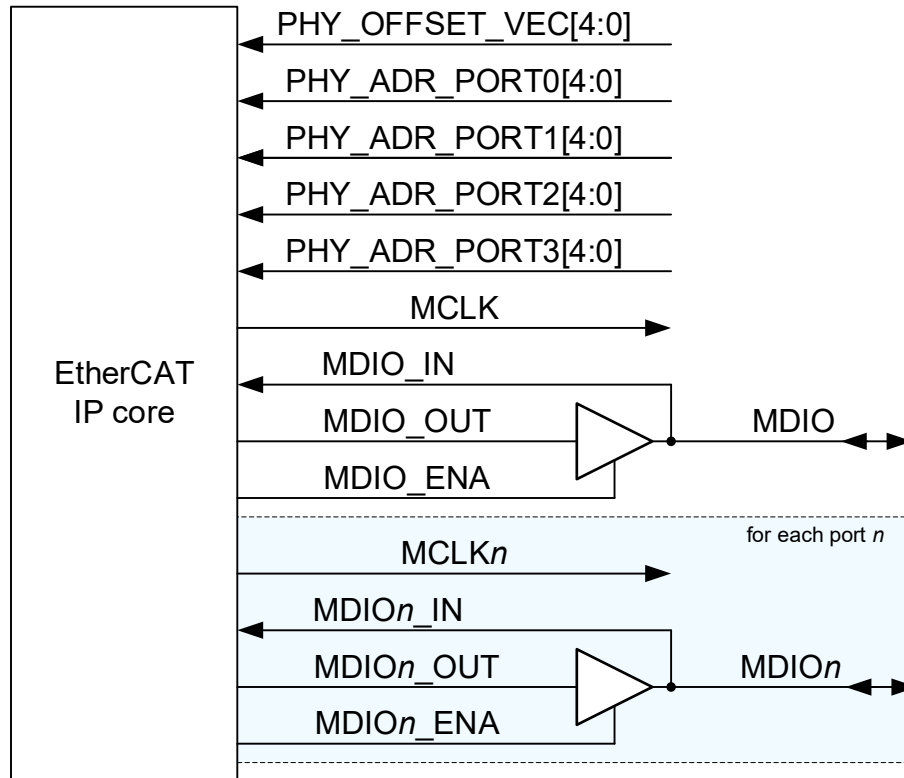


Figure 22: PHY management interface signals

Table 32: PHY management interface signals

Signal	Direction	Description
PHY_OFFSET_VEC[4:0]	INPUT	PHY address offset (consecutive PHY addresses, address of port 0)
PHY_ADR_PORT0[4:0]	INPUT	PHY address port 0 (individual PHY addresses)
PHY_ADR_PORT1[4:0]	INPUT	PHY address port 1 (individual PHY addresses)
PHY_ADR_PORT2[4:0]	INPUT	PHY address port 2 (individual PHY addresses)
PHY_ADR_PORT3[4:0]	INPUT	PHY address port 3 (individual PHY addresses)
Shared PHY management interface		
MCLK	OUT	Management interface clock
MDIO_IN	INPUT	Management interface data (alias MDIO): input
MDIO_OUT	OUTPUT	Management interface data (alias MDIO): output value
MDIO_ENA	OUTPUT	Management interface data (alias MDIO): output enable (act. high)
Separate PHY management interfaces for port n (0-3)		
MCLK $_n$	OUT	Management interface clock port n
MDIO $_n$ _IN	INPUT	Management interface data (alias MDIO): input
MDIO $_n$ _OUT	OUTPUT	Management interface data (alias MDIO): output value
MDIO $_n$ _ENA	OUTPUT	Management interface data (alias MDIO): output enable (act. high)

MDIO and MDIO0-3 must have a pull-up resistor (4.7 kΩ recommended for ESCs), either integrated into the ESC or externally. MCLK/CLK0-3 is driven rail-to-rail, idle value is high.

7.1.2 PHY address configuration

The EtherCAT IP core addresses Ethernet PHYs typically using logical port number plus PHY address offset. Ideally, the Ethernet PHY addresses should correspond with the logical port number, so PHY addresses 0-3 are used.

A PHY address offset of 0-31 can be applied which moves the PHY addresses to any consecutive address range. The IP core expects logical port 0 to have PHY address 0 plus PHY address offset (and so on).

Alternatively, the PHY addresses can be configured individually for each port.

Since the PHY addresses are static in most cases, they are set in the IP core configuration GUI. If the PHY addresses are changing dynamically, their configuration can be done by signals (*Export PHY address as signals* feature enabled).

7.1.3 MI link detection and configuration

MI link detection and configuration checks and writes the following registers.

Table 33: MI link detection for 100Base-TX

PHY register	Read mask	Read value	Write value	Comment
0	0xDE80	0x1000	0x3300	Write only if any register does not have expected value
1	0x403D	0x402D		Check link up status
4	0x05FF	0x0181	0x0111	Advertise 100BaseTX only
5	0x611F	0x4101		Check auto-negotiation performed, 100Base-TX full-duplex
9 (port 0)	0xE300	0x0000	0x0000	Prefer slave
9 (port 1-3)	0xE300	0x0000	0x0400	Prefer master
15	0x3000	0x1000 or 0x2000		Check for support of 1000Base-T
7.60	0x7FFF	0x0000	0x0000	Disable EEE advertisement (using MMD access control/data registers 13&14)

Table 34: MI link detection for 100Base-FX

PHY register	Read mask	Read value	Write value	Comment
0	0xFFE0	0x2100	0x2100	Write only if any register does not have expected value
1	0x4004	0x4004		Check link up status

7.1.3.1 User initialization of PHY registers

PHY registers can be initialized (written) by MI link detection and configuration, before the regular operation begins. The EtherCAT IP core will initialize all PHYs with the same values, as presented at the ROM interface for PHY initialization.

The ROM interface for PHY initialization has the following signals:

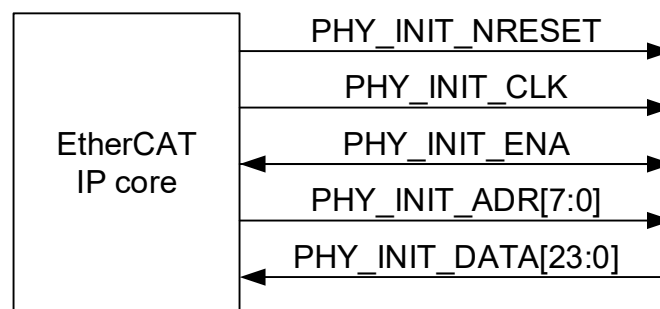


Figure 23: PHY initialization ROM interface signals

Table 35: PHY initialization ROM interface signals

Signal	Direction	Description
PHY_INIT_NRESET	OUTPUT	PHY init ROM interface reset (act. low), is usually not connected
PHY_INIT_CLK	OUTPUT	PHY init ROM access clock (25 MHz)
PHY_INIT_ENA	OUTPUT	PHY init ROM enable (act. high)
PHY_INIT_ADR[7:0]	OUTPUT	PHY init read address
PHY_INIT_DATA[23:0]	INPUT	PHY init read data

The EtherCAT IP core will read the ROM starting at address 0, each 24 bit word represents one write command to the PHYs. The IP core continues reading commands until the “more commands” bit is cleared.

The EtherCAT IP core will read the ROM starting at address 0, each 24 bit word represents one write command to the PHYs. The IP core continues reading commands until the “more commands” bit is cleared. Each line can represent an IEEE 802.3 clause 22 access, or a clause 45 command.

An example implementation of a PHY user initialization ROM is available in the ZIP package, in the *example/init* folder. The VHDL file of the PHY initialization ROM has to be compiled into the same library as the EtherCAT IP core.

The ROM content is organized as follows (PHY_INIT_DATA[23:0] for each address):

Table 36: PHY initialization ROM layout

Data bits	Description clause 22 access	Description clause 45 access
[23]	More commands following: 0: no more commands, this is the last 1: more commands follow	
[22]	0: clause 22 access	1: clause 45 access
[21]	0	0: write command 1: set address command
[20:16]	Register address	Device address
[15:0]	Write value	Write command: write value Set address command: register address

7.1.4 MII management timing specifications

For MII management interface timing diagrams refer to section I.

Table 37: MII management timing characteristics

Parameter	Min	Typ	Max	Comment
PRELIMINARY TIMING				
t _{MI_startup}		335 ms		Time between NPHY_RESET_OUT reset end and the first access via management interface
t _{Clk}		400 ns		MI clock period
t _{Write}		~ 25.6 µs		MI write access time
t _{Read}		~ 25.4 µs		MI read access time

7.2 PHY reset

The individual PHY reset signals for each port have this functionality:

- Keep the PHY in reset while the ESC is in reset
- If an FX link is used, and enhanced link detection is active, a reset for the PHY and the transceiver is applied when too many RX_ERR are received
- The PHY reset is extended to a minimum time $t_{\text{PHY_Reset_out}}$

Table 38: PHY reset signals

Signal	Direction	Description
NPHY_RESET_OUT n	OUT	PHY reset for port n (0-3, act. low), resets PHY while ESC is in reset state, and, for FX PHYs, if enhanced link detection detects a lost link

NOTE: A pull-down resistor is typically required for NPHY_RESET_OUT to hold the PHY in reset state until the FPGA is programmed and operational, since this pin is floating or even pulled up during that time.

NOTE: If it is not possible to use the individual PHY reset signals for each port, you must use the global NRESET of the ESC. Do not use the reset for port 0 for all other ports.

7.2.1 PHY reset timing specifications

Table 39: PHY reset timing characteristics

Parameter	Min	Typ	Max	Comment
$t_{\text{PHY_Reset_Out}}$	1.2 ms			PHY reset active time

7.3 MII interface

The MII interface of the IP core is optimized for low processing/forwarding delays by omitting a transmit FIFO. To allow this, the IP core has additional requirements to Ethernet PHYs, which are easily accomplished by several PHY vendors.



Refer to “section I – technology” for Ethernet PHY requirements.

Additional information regarding the IP core:

- The clock source of the PHYs is the same as for the FPGA (25 MHz quartz oscillator)
- The signal polarity of NMII_LINK is not configurable inside the IP core, NMII_LINK is active low. If necessary, the signal polarity must be swapped by user logic outside the IP core.
- The IP core can be configured to use the MII management interface for link detection and link configuration.
- The IP core supports arbitrary PHY addresses.

For details about the ESC MII interface refer to section I.

7.3.1 MII interface signals

The MII interface of the IP core has the following signals:

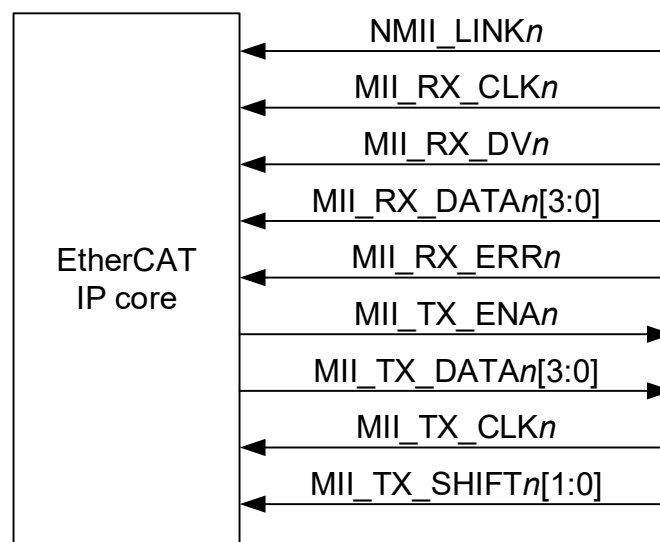


Figure 24: MII interface signals for port n

Table 40: MII interface signals for port n

Signal	Direction	Description
NMII_LINK n	IN	Input signal provided by the PHY if a 100 Mbit/s (Full Duplex) link is established (alias LINK_MII)
MII_RX_CLK n	IN	Receive clock
MII_RX_DV n	IN	Receive data valid
MII_RX_DATA n [3:0]	IN	Receive data (alias RXD)
MII_RX_ERR n	IN	Receive error (alias RX_ER)
MII_TX_ENA n	OUT	Transmit enable (alias TX_EN)
MII_TX_DATA n [3:0]	OUT	Transmit data (alias TXD)
MII_TX_CLK n	IN	Transmit clock for automatic TX shift compensation
MII_TX_SHIFT n [1:0]	IN	Manual TX shift compensation with additional registers

7.3.2 TX shift compensation

Since IP core and the Ethernet PHYs share the same clock source, TX_CLK from the PHY has a fixed phase relation to MII_TX_ENA/MII_TX_DATA from the IP core. Thus, the delay of a TX FIFO inside the IP core is saved.

In order to fulfill the setup/hold requirements of the PHY, the phase shift between TX_CLK and MII_TX_ENA/MII_TX_DATA has to be controlled. There are several alternatives:

- Automatic TX shift compensation if the TX shift feature is selected: connect MII_TX_CLK and the automatic TX shift compensation will determine correct shift settings (recommended). For guaranteed timings, maximum clock-to-output times for MII_TX_ENA/MII_TX_DATA should be applied, too. Set manual TX shift compensation to 0 in this case.
- TX shift compensation by specifying/verifying minimum and maximum clock-to-output times for MII_TX_ENA/MII_TX_DATA with respect to CLK_IN (PHY and PLL clock source).
- TX shift compensation with additional delays for MII_TX_ENA/MII_TX_DATA of 10, 20, or 30 ns. Such delays can be added using the TX shift feature and applying MII_TX_SHIFT[1:0]. MII_TX_SHIFT[1:0] determine the delay in multiples of 10 ns for each port. For guaranteed timings, maximum clock-to-output times for MII_TX_ENA/MII_TX_DATA should be applied, too. Set MII_TX_CLK to 0 if manual TX shift compensation is used.

MII_TX_ENA and MII_TX_DATA are generated synchronous to CLK25, although the source registers are both CLK25 and CLK100 registers.

The PLL has to use a configuration which guarantees a fixed phase relation between clock input and CLK25/CLK100 output, in order to enable TX shift compensation for the MII TX signals.

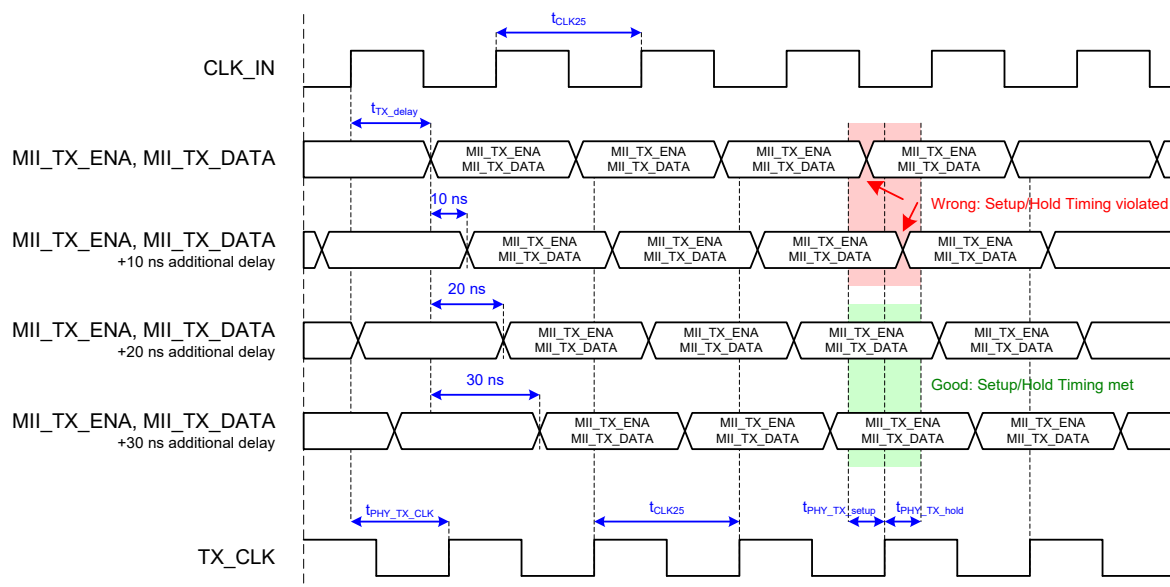


Figure 25: MII TX timing diagram

Table 41: MII TX timing characteristics

Parameter	Comment
t _{CLK25}	25 MHz quartz oscillator (CLK_IN)
t _{TX_delay}	MII_TX_ENA/MII_TX_DATA[3:0] delay after rising edge of CLK_IN, depends on synthesis results
t _{PHY_TX_CLK}	Delay between PHY clock source and TX_CLK output of the PHY, PHY dependent
t _{PHY_TX_setup}	PHY setup requirement: TX_ENA/TX_DATA with respect to TX_CLK (PHY dependent, IEEE802.3 limit is 15 ns)
t _{PHY_TX_hold}	PHY hold requirement: TX_ENA/TX_DATA with respect to TX_CLK (PHY dependent, IEEE802.3 limit is 0 ns)

If the phase shift between CLK25 and TX_CLK should not be constant for a some special PHYs, additional FIFOs for MII_TX_ENA/MII_TX_DATA are necessary. The FIFO input uses CLK25, the FIFO output TX_CLK[0] or TX_CLK[1] respectively.

NOTE: The phase shift can be adjusted by displaying TX_CLK of a PHY and MII_TX_ENA/MII_TX_DATA[3:0] on an oscilloscope. MII_TX_ENA/MII_TX_DATA[3:0] is allowed to change between 0 ns and 25 ns after a rising edge of TX_CLK (according to IEEE802.3 – check your PHY's documentation). Setup phase shift so that MII_TX_ENA/MII_TX_DATA[3:0] change near the middle of this range. MII_TX_ENA/MII_TX_DATA[3:0] signals are generated at the same time.

7.3.3 MII timing specifications

Table 42: MII timing characteristics

Parameter	Min	Typ	Max	Comment
t _{RX_CLK}		40 ns ± 100 ppm		RX_CLK period (100 ppm with maximum FIFO Size only)
t _{RX_setup}	x ¹			RX_DV/RX_DATA/RX_D[3:0] valid before rising edge of RX_CLK
t _{RX_hold}	x ¹			RX_DV/RX_DATA/RX_D[3:0] valid after rising edge of RX_CLK

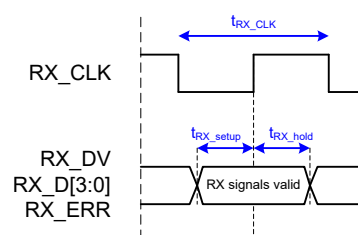


Figure 26: MII timing RX signals

¹ EtherCAT IP core: time depends on synthesis results

7.3.4 MII example schematic

Refer to chapter 6.5 for more information on special markings (!). Take care of proper compensation of the TX_CLK phase shift.

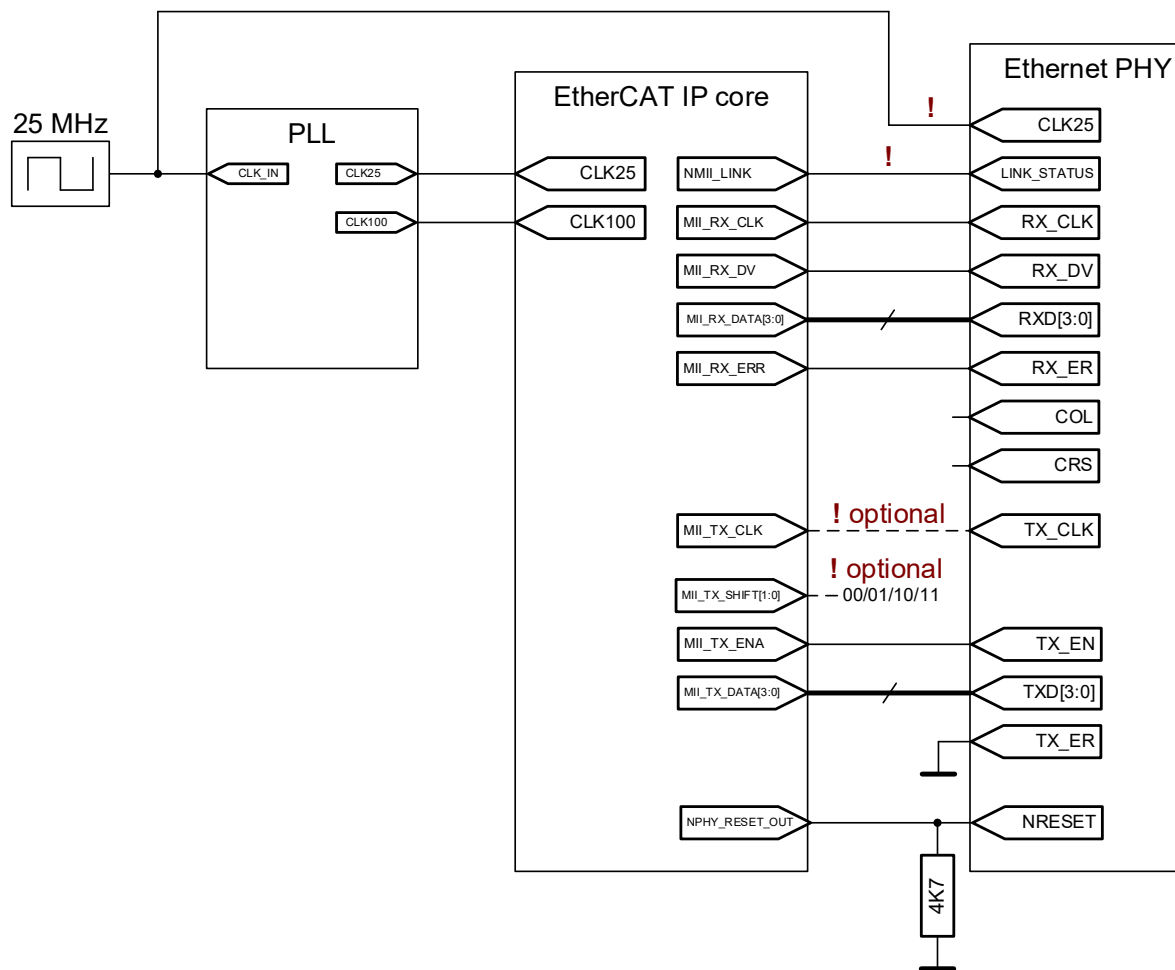


Figure 27: MII example schematic

7.4 RMII interface

The IP core supports RMII with 1-4 communication ports. Nevertheless, MII is recommended since the PHY delay (and delay jitter) is smaller in comparison to RMII.

The Beckhoff ESCs have additional requirements to Ethernet PHYs using RMII, which are easily accomplished by several PHY vendors.



Refer to “section I – technology” for Ethernet PHY requirements.

Additional information regarding the IP core:

- The clock source of the PHYs is the same as for the FPGA (25 MHz quartz oscillator)
- The signal polarity of NRMII_LINK is not configurable inside the IP core, NRMII_LINK is active low. If necessary, the signal polarity must be swapped outside the IP core.
- The IP core can be configured to use the MII management interface for link detection and link configuration.
- The IP core supports arbitrary PHY addresses.

For details about the ESC RMII interface refer to section I.

7.4.1 RMII interface signals

The RMII interface of the IP core has the following signals:

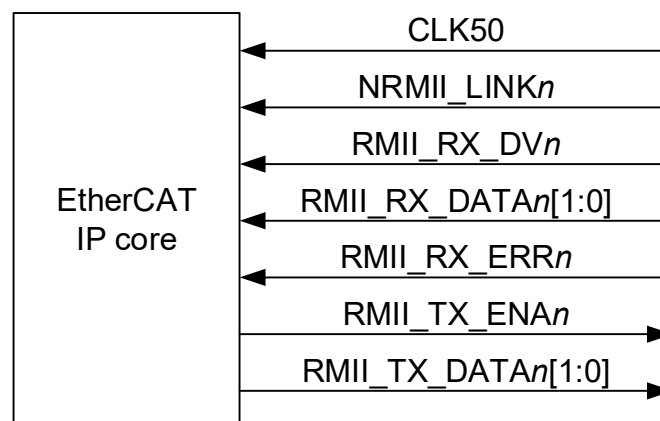


Figure 28: RMII interface signals for port n

Table 43: RMII interface signals for port n

Signal	Direction	Description
CLK50	IN	RMII RX/TX reference clock (50 MHz)
NRMII_LINK n	IN	Input signal provided by the PHY if a 100 Mbit/s (Full Duplex) link is established (alias LINK_MII)
RMII_RX_DV n	IN	Carrier sense/receive data valid
RMII_RX_DATA n [1:0]	IN	Receive data (alias RXD)
RMII_RX_ERR n	IN	Receive error (alias RX_ER)
RMII_TX_ENA n	OUT	Transmit enable (alias TX_EN)
RMII_TX_DATA n [1:0]	OUT	Transmit data (alias TXD)

7.4.2 RMII example schematic

Refer to chapter 6.5 for more information on special markings (!). Take care of proper PHY address configuration.

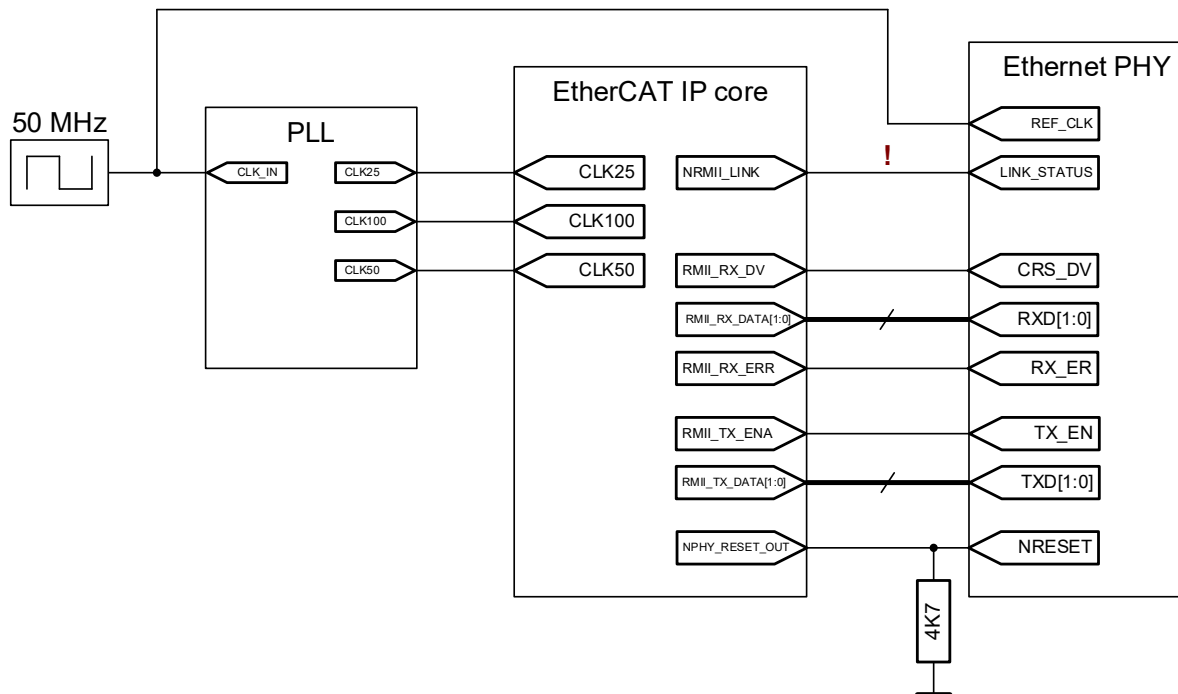


Figure 29: RMII example schematic

7.5 RGMII interface

The IP core supports RGMII with 1-4 communication ports at 100 Mbit/s. Nevertheless, MII is recommended since the PHY delay (and delay jitter) is smaller in comparison to RGMII.

The Beckhoff ESCs have additional requirements to Ethernet PHYs using RGMII, which are easily accomplished by several PHY vendors.



Refer to “section I – technology” for Ethernet PHY requirements.

Additional information regarding the IP core:

- The signal polarity of NRGMII_LINK is not configurable inside the IP core, NRGMII_LINK is active low. If necessary, the signal polarity must be swapped outside the IP core.
- The IP core can be configured to use the MII management interface for link detection and link configuration.
- The IP core supports arbitrary PHY addresses.
- A Gigabit Ethernet PHY has to be restricted to establish only 100 Mbit/s links (e.g. by using MI link detection and configuration).
- If RGMII in-band status is available, set NRGMII_LINK to “no link”. Link detection is performed based on RGMII in-band status signalling (recommended).

For details about the ESC RGMII interface refer to section I.

7.5.1 RGMII interface signals

The RGMII interface of the EtherCAT IP core offers signals for connecting DDR input and output cells, which have to be added by the IP core user.

The RGMII interface of the IP core has the following signals:

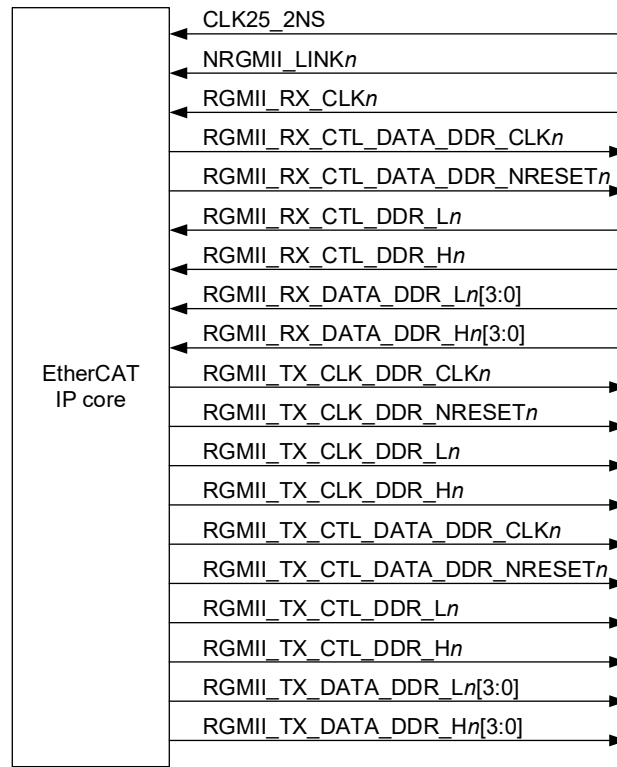


Figure 30: RGMII interface signals for port n

Table 44: RGMII interface signals for port n

Signal	Direction	Description
CLK25_2NS	IN	25 MHz clock signal from PLL (rising edge 2 ns after rising edge of CLK25), used for RGMII TXC
NRGMII_LINK n	IN	Input signal provided by the PHY if a 100 Mbit/s (full Duplex) link is established (alias LINK_MII). Set to high if RGMII in-band status is used (recommended).
RGMII_RX_CLK	IN	Receive clock
RGMII_RX_CTL_DATA_DDR_CLK	OUT	Receive control/data DDR input clock
RGMII_RX_CTL_DATA_DDR_NRESET n	OUT	Receive control/data DDR input reset (act. low)
RGMII_RX_CTL_DDR_L n	IN	Receive control DDR input low
RGMII_RX_CTL_DDR_H n	IN	Receive control DDR input high
RGMII_RX_DATA_DDR_L n [3:0]	IN	Receive data DDR input low
RGMII_RX_DATA_DDR_H n [3:0]	IN	Receive data DDR input high
RGMII_TX_CLK_DDR_CLK n	OUT	Transmit clock DDR output clock (equals CLK25_2NS)
RGMII_TX_CLK_DDR_NRESET n	OUT	Transmit clock DDR output reset (act. low)
RGMII_TX_CLK_DDR_L n	OUT	Transmit clock DDR output low (static 0)
RGMII_TX_CLK_DDR_H n	OUT	Transmit clock DDR output high (static 1)
RGMII_TX_CTL_DATA_DDR_CLK n	OUT	Transmit control/data DDR output clock (equals CLK25)

Signal	Direction	Description
RGMII_TX_CTL_DATA_DDR_NRESET n	OUT	Transmit control/data DDR output reset (act. low)
RGMII_TX_CTL_DDR_L n	OUT	Transmit control DDR output low
RGMII_TX_CTL_DDR_H n	OUT	Transmit control DDR output high
RGMII_TX_DATA_DDR_L n [3:0]	OUT	Transmit data DDR output low
RGMII_TX_DATA_DDR_H n [3:0]	OUT	Transmit data DDR output high

7.5.2 RGMII example schematic

Refer to chapter 6.5 for more information on special markings (!). Take care of proper PHY address configuration.

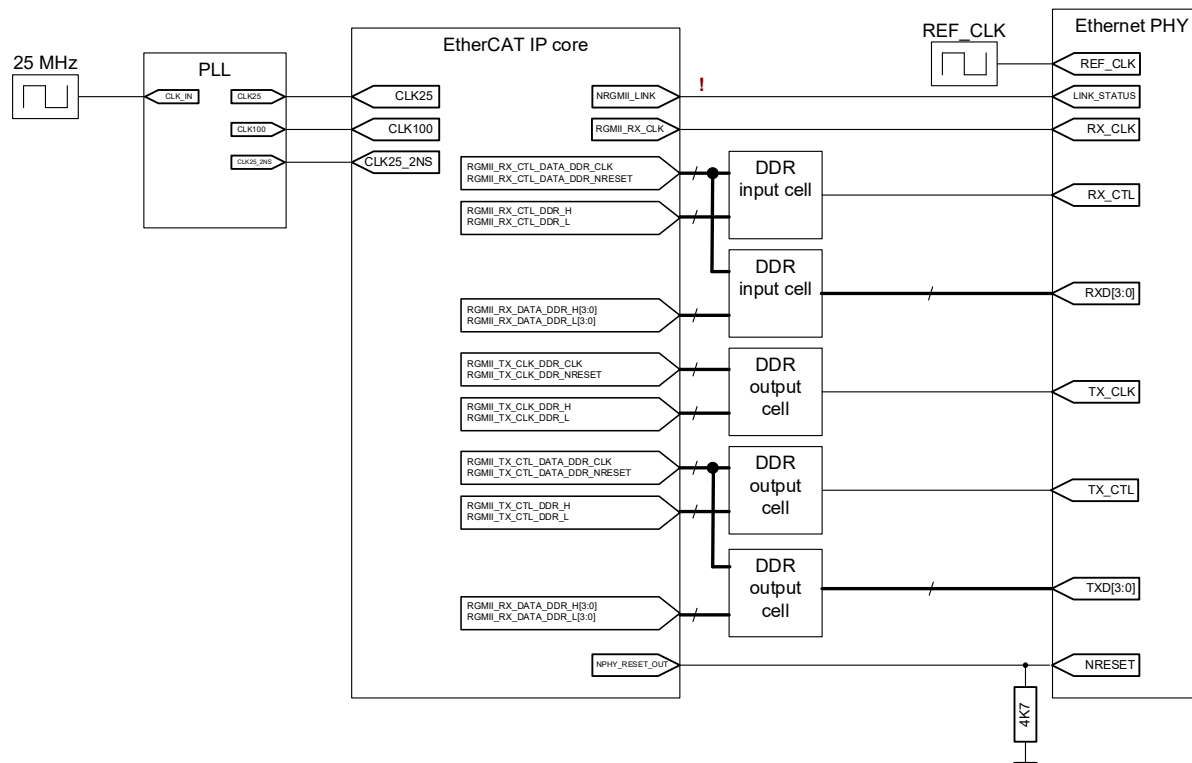


Figure 31: RGMII example schematic

7.5.3 RGMII RX timing options

RGMII uses a source synchronous interface for receive signals. Originally, RX_CLK and RX_CTL/RX_DATA are edge-aligned at the PHY side. RX_CLK needs to be delayed to maintain setup/hold timing at the FPGA side. There are several options for delaying RX_CLK:

- **RX_CLK delay in PHY:**
Some PHYs offer RGMII-ID, which means, the RX_CLK is delayed internally in the PHY. The EtherCAT IP core itself cannot enable this feature using the MII management interface if the PHY requires this. It is up to the IP core user to enable this feature, if required.
- **RX_CLK delay on PCB:**
One option is to delay RX_CLK on the PCB.
- **RX_CLK delay in FPGA with PLL:**
The delay of RX_CLK can be realized with a PLL at each RGMII port, configured for clock phase shift.
- **RX_CLK delay in FPGA without PLL:**
The delay of RX_CLK can be realized with routing delay inside the FPGA.

7.5.4 RGMII TX timing options

RGMII uses a source synchronous interface for receive signals. Originally, TX_CLK and TX_CTL/RX_DATA are edge-aligned at the FPGA side. TX_CLK needs to be delayed to maintain setup/hold timing at the PHY side. There are several options for delaying TX_CLK:

- **TX_CLK delay in PHY:**
Some PHYs offer RGMII-ID, which means, the TX_CLK is delayed internally in the PHY. The EtherCAT IP core itself cannot enable this feature using the MII management interface if the PHY requires this. It is up to the IP core user to enable this feature.
- **TX_CLK delay on PCB:**
One option is to delay TX_CLK on the PCB.
- **TX_CLK delay in FPGA with PLL:**
The delay of TX_CLK can be realized with a PLL providing a delayed CLK25 attached to the CLK25_2NS input of the IP core. This clock is used for the TX_CLK DDR output cell, while CLK25 is used for the TX_CTL/TX_DATA DDR output cells.
- **TX_CLK delay in FPGA without PLL:**
The delay of TX_CLK can be realized with routing delay inside the FPGA.

7.5.5 RGMII timing specifications

Table 45: RGMII timing characteristics IP core

Parameter	Min	Max	Comment
t_{RX_CLK}	8 ns	400 ns	RGMII receive clock period (depending on link parter 10/100/1000 Mbit/s, EtherCAT only supports 100 Mbit/s)
t_{TX_CLK}	40 ns	40 ns	RGMII transmit clock period (always 100 Mbit/s)

Received RGMII DDR signals are evaluated at the rising clock edge. The presented t_H value must have been received before the t_L value, as shown in the RGMII receive timing diagram. The total delay of the DDR cell may be higher if necessary, i.e., RXDV A etc. are presented to the IP core one processing edge later (when RXDV B is shown in the diagram).

Transmitted RGMII DDR signals are updated at the rising clock edge. The presented t_H value must be transmitted before the t_L value, as shown in the RGMII transmit timing diagram. The total delay of the DDR cell may be higher if necessary, i.e., TXEN A etc. are presented to the PHY one processing edge later (when TXEN B is shown in the diagram).

Since the EtherCAT IP core does not generate TXER, it will output TXEN twice. Standard RGMII timing with TX error is also shown for your reference.

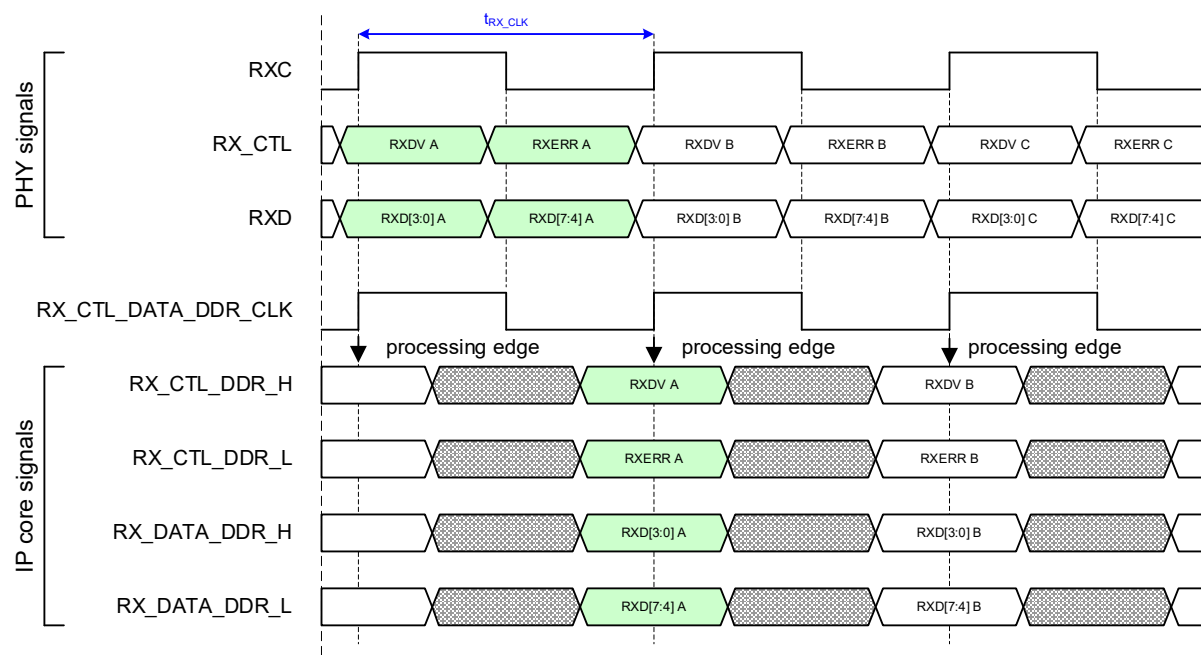


Figure 32: RGMII receive timing

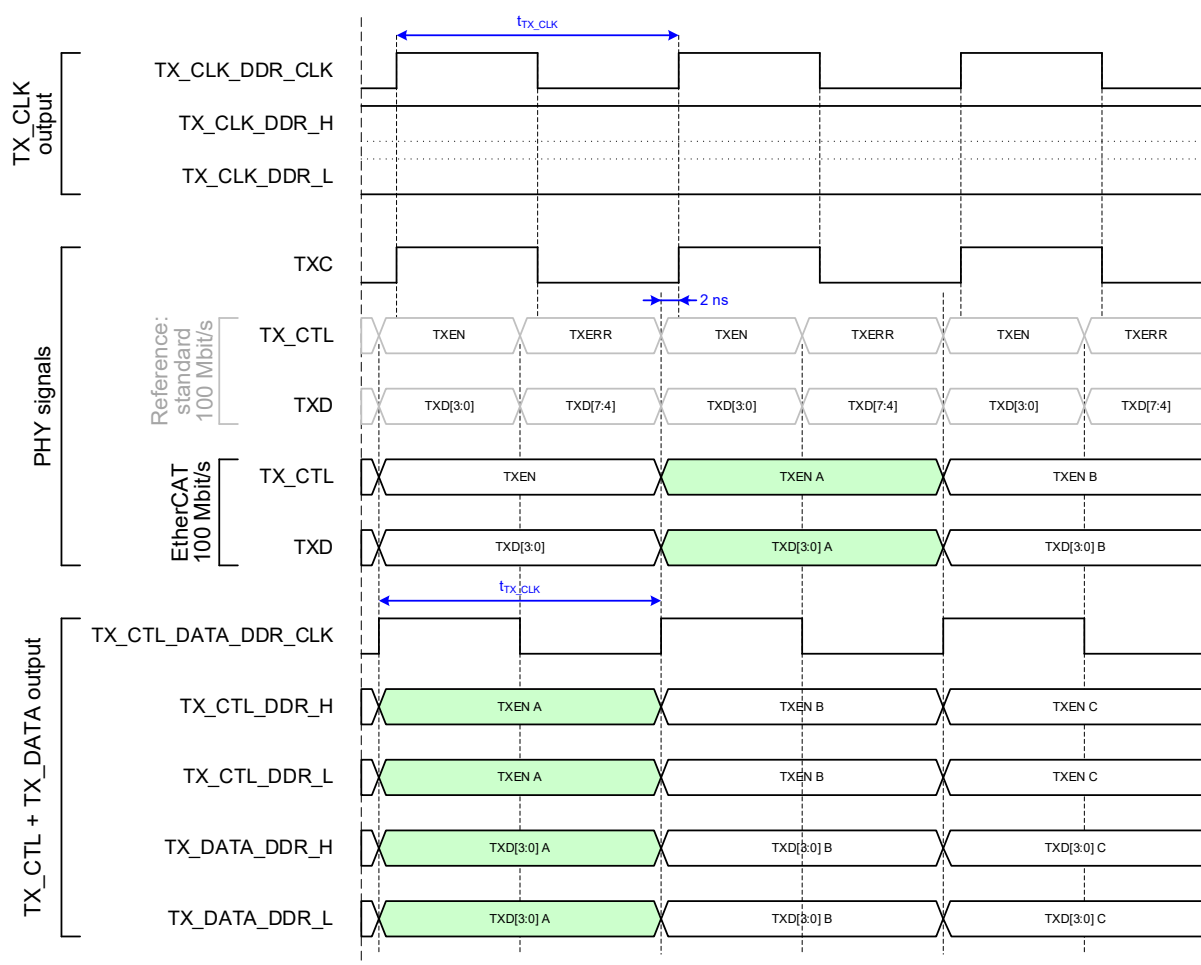


Figure 33: RGMII transmit timing

8 PDI description

The EtherCAT IP core supports up to two PDI interfaces (PDI0 and PDI1), each supporting several interface types.

Please refer to section I of the data sheet for details on how the two PDIs are used.

Table 46: Available PDIs for EtherCAT IP core

PDI type	PDI name	IP core
	PDI0 PDI control register 0x0140[7:0]	
0x00	Interface deactivated	x
0x01	4 digital input	
0x02	4 digital output	
0x03	2 digital input and 2 digital output	
0x04	Digital I/O	x
0x05	SPI slave	x
0x06	Oversampling I/O	
0x07	EtherCAT bridge (port 3)	
0x08	16/64 bit asynchronous microcontroller interface	16
0x09	8/32 bit asynchronous microcontroller interface	8/32
0x0A	16/64 bit synchronous microcontroller interface	
0x0B	8/32 bit synchronous microcontroller interface	
0x0C	16/64 bit multiplexed asynchronous microcontroller interface	
0x0D	8/32 bit multiplexed asynchronous microcontroller interface	
0x0E	16/64 bit multiplexed synchronous microcontroller interface	
0x0F	8/32 bit multiplexed synchronous microcontroller interface	
0x10	32 digital input/0 digital output	
0x11	24 digital input/8 digital output	
0x12	16 digital input/16 digital output	
0x13	8 digital input/24 digital output	
0x14	0 digital input/32 digital output	
0x15	SPI master	
0x80	On-chip bus Avalon: 0x0150[7:5]=000	x
	On-chip bus AXI3: 0x0150[7:5]=001, 0x0152[10:8]=000	
	On-chip bus AXI4: 0x0150[7:5]=001, 0x0152[10:8]=010	x
	On-chip bus AXI4LITE: 0x0150[7:5]=001, 0x0152[10:8]=010	x
	On-chip bus PLB v4.6: 0x0150[7:5]=010	
	On-chip bus OPB: 0x0150[7:5]=100	
Others	Reserved	

PDI type	PDI name	IP core
	PDI1 PDI control register 0x0180[7:0]	
0x00	Interface deactivated	x
0x01	4 digital input	
0x02	4 digital output	
0x03	2 digital input and 2 digital output	
0x04	Digital I/O	x
0x05	SPI slave	x
0x06	Oversampling I/O	
0x07	EtherCAT bridge (port 3)	
0x08	16/64 bit asynchronous microcontroller interface	
0x09	8/32 bit asynchronous microcontroller interface	
0x0A	16/64 bit synchronous microcontroller interface	
0x0B	8/32 bit synchronous microcontroller interface	
0x0C	16/64 bit multiplexed asynchronous microcontroller interface	
0x0D	8/32 bit multiplexed asynchronous microcontroller interface	
0x0E	16/64 bit multiplexed synchronous microcontroller interface	
0x0F	8/32 bit multiplexed synchronous microcontroller interface	
0x10	32 digital input/0 digital output	
0x11	24 digital input/8 digital output	
0x12	16 digital input/16 digital output	
0x13	8 digital input/24 digital output	
0x14	0 digital input/32 digital output	
0x15	SPI master	
0x80	On-chip bus Avalon: 0x0150[7:5]=000	x
	On-chip bus AXI3: 0x0150[7:5]=001, 0x0152[10:8]=000	
	On-chip bus AXI4: 0x0150[7:5]=001, 0x0152[10:8]=010	x
	On-chip bus AXI4LITE: 0x0150[7:5]=001, 0x0152[10:8]=010	x
	On-chip bus PLB v4.6: 0x0150[7:5]=010	
	On-chip bus OPB: 0x0150[7:5]=100	
Others	Reserved	

8.1 PDI deactivated

The PDI is deactivated with PDI type 0x00.

8.2 Digital I/O interface

8.2.1 Interface

The digital I/O PDI is selected with PDI type 0x04. The signals of the digital I/O interface are shown below. The IP core signals have a prefix “PDI_DIGI_” for PDI0, and “PDI1_DIGI_” for PDI1, they are omitted in this chapter.

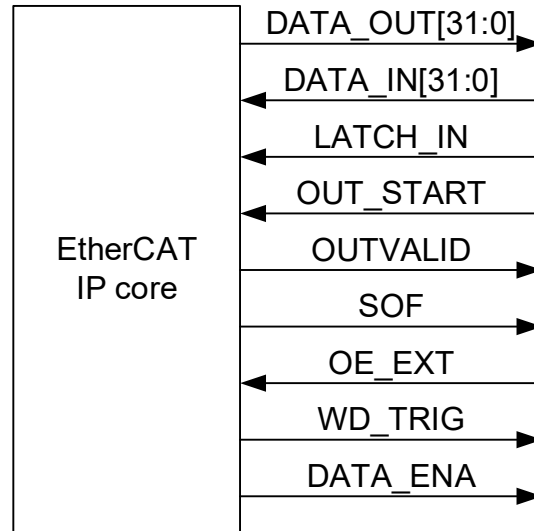


Figure 34: IP core digital I/O signals

Table 47: IP core digital I/O signals

Signal	Direction	Description	Signal polarity
DATA_OUT[31:0]	OUT	Output data	
DATA_IN[31:0]	IN	Input data	
LATCH_IN	IN	External data latch signal	act. high
OUT_START	IN	External data output start signal	act. high
OUTVALID	OUT	Output data is valid/output event	act. high
SOF	OUT	Start of frame	act. high
OE_EXT	IN	Output enable	act. high
WD_TRIG	OUT	Watchdog trigger	act. high
DATA_ENA	OUT	Enable external output data driver	act. high

NOTE: The digital I/O control signal OE_CONF is not supported, and assumed to be low.

The digital I/O PDI supports 1-4 byte of digital I/O signals, with each byte individually configurable as either input or output. At the IP core interface, the I/O signals are separated in input signals (DATA_IN) and output signals (DATA_OUT). The corresponding I/O signals and ESC addresses are listed below.

Table 48: Input/output register reference

I/O byte	Output signal	Output address	Input signal	Input address
PDI0				
0	DATA_OUT[7:0]	0x0F00	DATA_IN[7:0]	0x1000
1	DATA_OUT[15:8]	0x0F01	DATA_IN[15:8]	0x1001
2	DATA_OUT[23:16]	0x0F02	DATA_IN[23:16]	0x1002
3	DATA_OUT[31:24]	0x0F03	DATA_IN[31:24]	0x1003
PDI1				
0	DATA_OUT[7:0]	0x0F00	DATA_IN[7:0]	0x0F10
1	DATA_OUT[15:8]	0x0F01	DATA_IN[15:8]	0x0F11
2	DATA_OUT[23:16]	0x0F02	DATA_IN[23:16]	0x0F12
3	DATA_OUT[31:24]	0x0F03	DATA_IN[31:24]	0x0F13

8.2.2 Configuration

The digital I/O interface is selected with PDI type 0x04 in the PDI0 control register 0x0140, or PDI1 control register 0x0180. It supports different configurations, which are located in registers 0x0150 – 0x0153 / 0x0190 – 0x0193.

8.2.3 Digital inputs

Digital input values of PDI0 appear in the process memory at address 0x1000:0x1003. EtherCAT devices use Little Endian byte ordering, so I/O[7:0] can be read at 0x1000 etc. Digital inputs are written to the process memory by the digital I/O PDI using standard PDI write operations.

Digital inputs of PDI1 are transferred to the digital I/O input data register 0x0F08:0x0F0B.

Digital inputs can be configured to be sampled by the ESC in four ways:

- Digital inputs are sampled at the start of each Ethernet frame, so that EtherCAT read commands to address 0x1000:0x1003 will present digital input values sampled at the start of the same frame. The SOF signal can be used externally to update the input data, because the SOF is signaled before input data is sampled.
- The sample time can be controlled externally by using the LATCH_IN signal. The input data is sampled by the ESC each time a rising edge of LATCH_IN is recognized.
- Digital inputs are sampled at distributed clocks SyncSignal 0 events.
- Digital inputs are sampled at distributed clocks SyncSignal 1 events.

For inputs based on distributed clock SyncSignals, SyncSignal generation must be activated (register 0x0981). SYNC output is not necessary (register 0x0151). SyncSignal pulse length (registers 0x0982:0x0983) should not be set to 0, because acknowledging of SyncSignal events is not possible with digital I/O PDI. Sample time is the beginning of the SyncSignal event.

8.2.4 Digital outputs

Digital output values have to be written to register 0x0F00:0x0F03 (register 0x0F00 controls DATA_OUT[7:0] etc.). Digital output values are not read by the digital I/O PDI using standard read commands, instead, there is a direct connection for faster response times.

The process data watchdog (register 0x0440) has to be either active or disabled; otherwise digital outputs will not be updated. Digital outputs can be configured to be updated in 4 ways:

- Digital outputs are updated at the end of each EtherCAT frame (EOF mode).
- Digital outputs are updated at a rising edge of OUT_START signal (OUT_START mode)
- Digital outputs are updated with distributed clocks SyncSignal 0 events.
- Digital outputs are updated with distributed clocks SyncSignal 1 events.

For outputs based on distributed clock SyncSignals, SyncSignal generation must be activated (register 0x0981). SYNC output is not necessary (register 0x0151). SyncSignal pulse length (registers 0x0982:0x0983) should not be set to 0, because acknowledging of SyncSignal events is not possible with digital I/O PDI. Output time is the beginning of the SyncSignal event.

An output event is always signaled by a pulse on OUTVALID even if the digital outputs remain unchanged.

For output data to be visible on the I/O signals, the following conditions have to be met:

- SyncManager watchdog must be either active (triggered) or disabled.
- OE_EXT (output enable) must be high.
- Output values have to be written to the registers 0x0F00:0x0F03 within a valid EtherCAT frame.
- The configured output update event must have occurred.

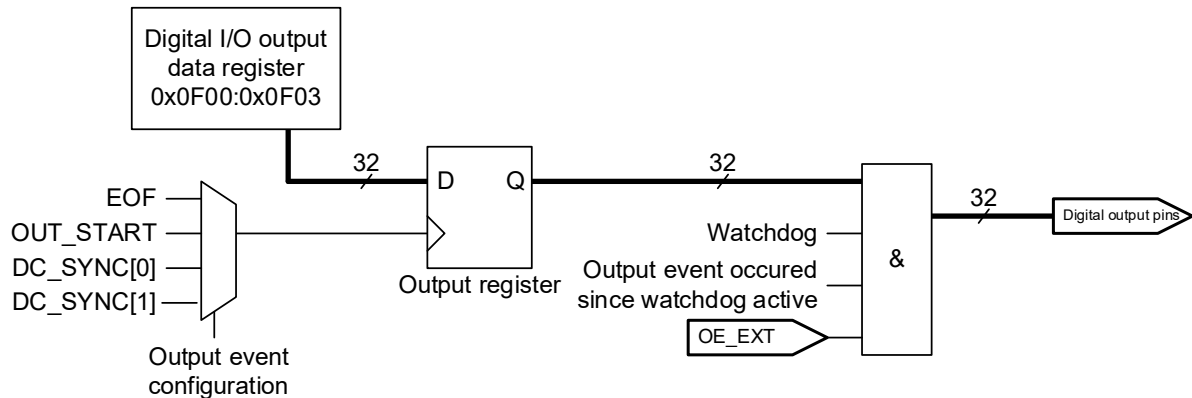


Figure 35: Digital output principle schematic

NOTE: The digital outputs are not driven (high impedance) until the FPGA is loaded. Depending on the FPGA configuration, digital outputs (like all other FPGA user pins) might have pull-up resistors until the FPGA has loaded its configuration. This behaviour has to be taken into account when using digital output signals.

8.2.5 Output enable

The IP core has an output enable signal OE_EXT. With the OE_EXT signal, the I/O signals can be cleared. The I/O signals will be driven low after the output enable signal OE_EXT is set to low or the SyncManager watchdog is expired (and not disabled).

8.2.6 SyncManager watchdog

The SyncManager watchdog (registers 0x0440:0x0441) must be either active (triggered) or disabled for output values to appear on the I/O signals. The SyncManager watchdog is triggered by an EtherCAT write access to the output data registers.

If the output data bytes are written independently, a SyncManager with a length of 1 byte is used for each byte of 0x0F00:0x0F03 containing output bits (SyncManager N configuration: buffered mode, EtherCAT write/PDI read, and watchdog trigger enabled: 0x44 in register 0x0804+N*8). Alternatively, if all output data bits are written together in one EtherCAT command, one SyncManager with a length of 1 byte is sufficient (SyncManager N configuration: buffered mode, EtherCAT write/PDI read, and watchdog trigger enabled: 0x44 in register 0x0804+N*8). The start address of the SyncManager should be one of the 0x0F00:0x0F03 bytes containing output bits, e.g., the last byte containing output bits.

The SyncManager watchdog can also be disabled by writing 0 into registers 0x0420:0x0421.

The watchdog mode configuration bit is used to configure if the expiration of the SyncManager watchdog will have an immediate effect on the I/O signals (output reset immediately after watchdog timeout) or if the effect is delayed until the next output event (output reset with next output event). The latter case is especially relevant for distributed clock SYNC output events, because any output change will occur at the configured SYNC event.

Immediate output reset after watchdog timeout is not available if OUTVALID mode set to watchdog trigger (0x0150[1]=1/0x0190[1]=1).

8.2.7 SOF

SOF indicates the start of an Ethernet/EtherCAT frame. It is asserted shortly after RX_DV=1 or EBUS SOF. Input data is sampled in the time interval between $t_{\text{SOF_to_DATA_setup}}$ and $t_{\text{SOF_to_DATA_setup}}$ after the SOF signal is asserted.

8.2.8 OUTVALID

A pulse on the OUTVALID signal indicates an output event. If the output event is configured to be the end of a frame, OUTVALID is issued shortly after RX_DV=0 or EBUS EOF, right after the CRC has been checked and the internal registers have taken their new values. OUTVALID is issued independent of actual output data values, i.e., it is issued even if the output data does not change.

8.2.9 Timing specifications

Table 49: Digital I/O timing characteristics IP core

Parameter	Min	Max	Comment
PRELIMINARY TIMING			
$t_{\text{DATA_setup}}$	x^2		Input data valid before LATCH_IN
$t_{\text{DATA_hold}}$	x^2		Input data valid after LATCH_IN
$t_{\text{LATCH_IN}}$	x^2		LATCH_IN high time
t_{SOF}	$40 \text{ ns} - x^2$	$40 \text{ ns} + x^2$	SOF high time
$t_{\text{SOF_to_DATA_setup}}$	0 ns	$1,2 \mu\text{s} - x^2$	Input data valid after SOF, so that inputs can be read in the same frame
$t_{\text{SOF_to_DATA_hold}}$	$1,6 \mu\text{s} + x^2$		Input data invalid after SOF
$t_{\text{input_event_delay}}$	440 ns		Time between consecutive input events
t_{OUTVALID}	$80 \text{ ns} - x^2$	$80 \text{ ns} + x^2$	OUTVALID high time
$t_{\text{DATA_to_OUTVALID}}$	$80 \text{ ns} - x^2$		Output data valid before OUTVALID
$t_{\text{WD_TRIG}}$	$40 \text{ ns} - x^2$	$40 \text{ ns} + x^2$	WD_TRIG high time
$t_{\text{DATA_to_WD_TRIG}}$		$20 \text{ ns} + x^2$	Output data valid after WD_TRIG
$t_{\text{DATA_to_SYNC}}$		$10 \text{ ns} + x^2$	Output data valid after SYNC0/1
$t_{\text{OE_EXT_to_DATA_invalid}}$	0 ns	x^2	Outputs zero or outputs high impedance after OE_EXT set to low
$t_{\text{output_event_delay}}$	320 ns		Time between consecutive output events
$t_{\text{OUT_ENA_valid}}$	$80 \text{ ns} - x^2$		OUT_ENA valid before OUTVALID
$t_{\text{OUT_ENA_invalid}}$	$80 \text{ ns} - x^2$		OUT_ENA invalid after OUTVALID

² EtherCAT IP core: time depends on synthesis results

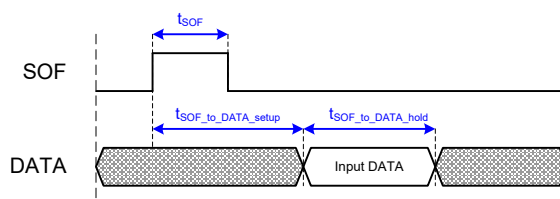


Figure 36: Digital input: input data sampled at SOF, input data can be read in the same frame

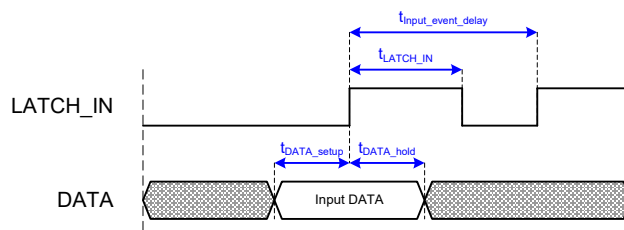


Figure 37: Digital input: input data sampled with LATCH_IN

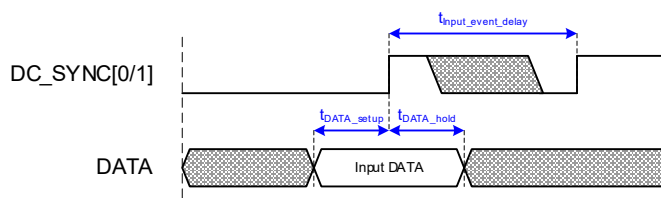


Figure 38: Digital input: input data sampled with DC_SYNC

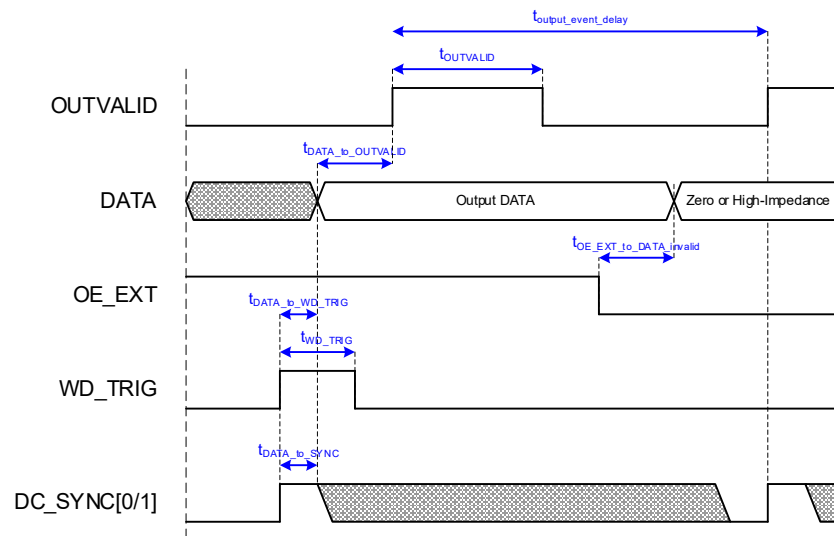


Figure 39: Digital output timing

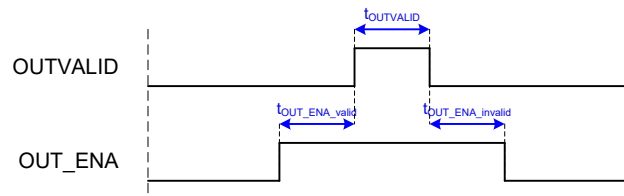


Figure 40: OUT_ENA timing

8.3 SPI slave interface

8.3.1 Interface

The SPI slave PDI is selected with PDI type 0x05. The signals of the digital I/O interface are shown below. The IP core signals have a prefix “PDI_” for PDI0, and “PDI1_” for PDI1, they are omitted in this chapter.

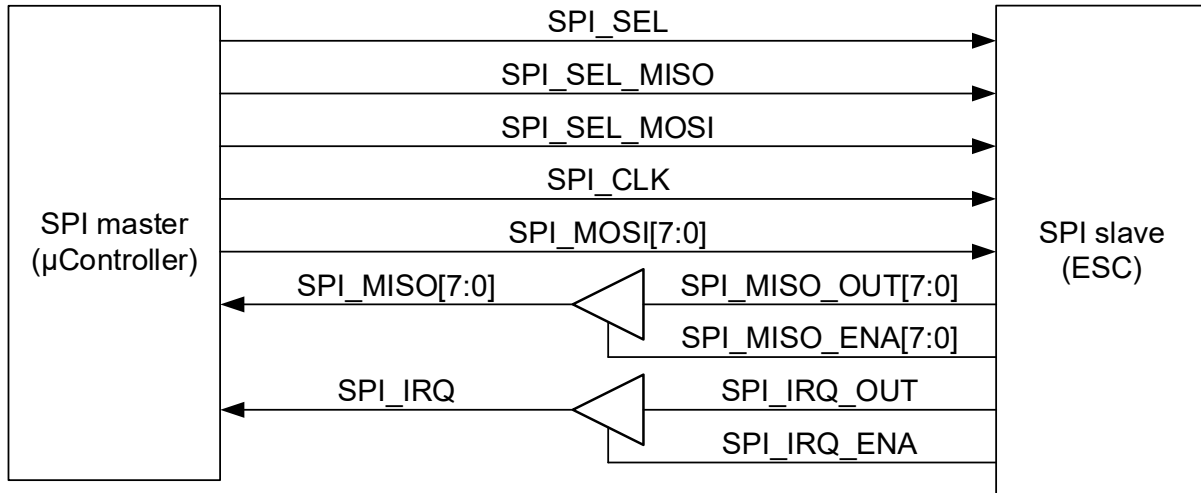


Figure 41: SPI master and slave interconnection (unidirectional)

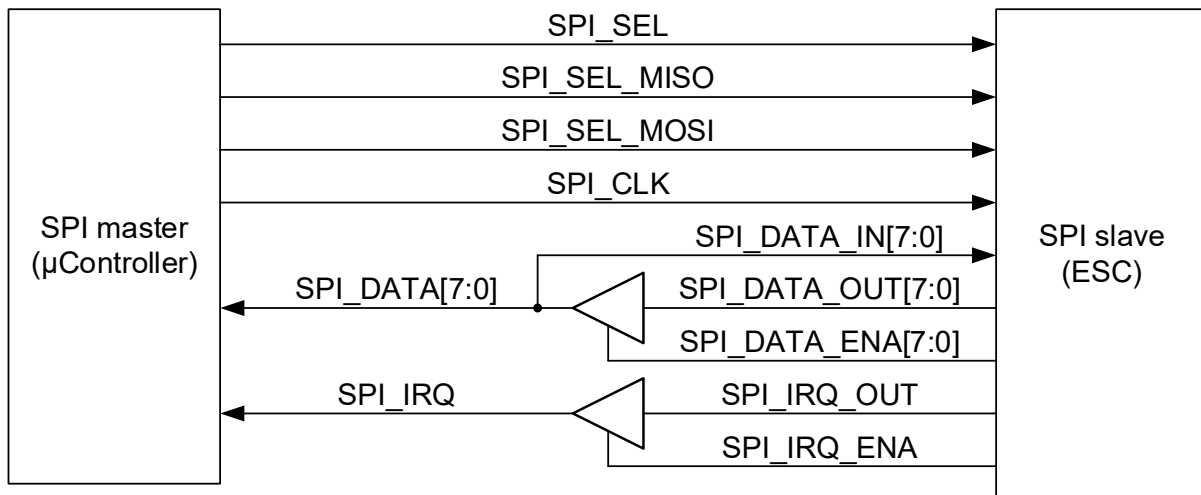


Figure 42: SPI master and slave interconnection (bidirectional)

Table 50: SPI signals

Signal	Direction	Description	Signal polarity
SPI_CLK	IN (master → slave)	SPI clock	
SPI_SEL	IN (master → slave)	SPI random access chip select	Typical: act. low
SPI_SEL_MISO	IN (master → slave)	MISO chip select	Same as SPI_SEL
SPI_SEL_MOSI	IN (master → slave)	MOSI chip select	
SPI_MOSI[7:0]	IN (master → slave)	MOSI data (unidir)	
SPI_MISO_OUT[7:0]	OUT (slave → master)	MISO data output value (unidir)	
SPI_MISO_ENA[7:0]	OUT (slave → master)	MISO data output driver enable (unidir)	act. high
SPI_DATA_IN[7:0]	IN (master → slave)	MISO/MOSI data (bidirectional)	act. high
SPI_DATA_OUT[7:0]	OUT (slave → master)	MISO/MOSI data (bidirectional)	act. high
SPI_DATA_ENA[7:0]	OUT (slave → master)	MISO/MOSI data (bidirectional)	act. high
SPI_IRQ_OUT	OUT (slave → master)	SPI interrupt output value	
SPI_IRQ_ENA	OUT (slave → master)	SPI interrupt output driver enable	act. high

8.3.2 Configuration

The SPI slave interface is selected with PDI type 0x05 in the PDI0 control register 0x0140, or PDI1 control register 0x0180. It supports different data widths, operation modes, and signal polarities. The SPI slave configuration is located in register 0x0150:0x0153 / 0x0190:0x0193.

8.3.3 Access types

A regular SPI access to any ESC register or ESC RAM uses SPI_SEL as select signal. This select signal indicates that a address/command phase will be used before data is exchanged.

The EtherCAT IP core also supports direct SyncManager access using SPI_SEL_MISO and SPI_SEL_MOSI select signals. When these signals are used, no address phase or command phase is required, instead, MISO or MOSI data from a specific SyncManager per direction is immediately accessible.

SPI_SEL_MISO and SPI_SEL_MOSI might be activated simultaneously (within a defined period of time), indicating bidirectional transfer.

But, SPI_SEL must never be activated overlapping with either SPI_SEL_MISO or SPI_SEL_MOSI.

8.3.4 SPI_SEL random access

Each SPI_SEL random access is separated into an address phase and a data phase. In the address phase, the SPI master transmits the first address to be accessed and the command. In the data phase, MISO data is presented by the SPI slave (read command) or MOSI data is transmitted by the master (write command). The address phase consists of 2 or 3 bytes depending on the address mode. The number of data bytes for each access may range from 0 to N bytes. The slave internally increments the address for the following bytes after reading or writing the start address. The bits of both address/command and data are transmitted in byte groups.

The master starts an SPI access by asserting SPI_SEL and terminates it by de-asserting SPI_SEL (polarity determined by configuration). While SPI_SEL is asserted, the master has to cycle SPI_CLK eight times for each byte transfer in x1 data width, or only once for x8 data width. In each clock cycle, both master and slave transmit 1/2/4/8 bit to the other side (full duplex). The relevant edges of SPI_CLK for master and slave can be configured by selecting SPI mode and data out sample mode.

The most significant bits of a byte are transmitted first, the least significant bits last, the byte order is low byte first. EtherCAT devices use little endian byte ordering.

8.3.4.1 Commands

The command CMD0 in the second address/command byte may be read, read with following wait state bytes, write, NOP, or address extension. The command CMD1 in the third address/command byte may have the same values:

Table 51: SPI commands CMD0 and CMD1

CMD[2]	CMD[1]	CMD[0]	Command
0	0	0	NOP (no operation)
0	0	1	reserved
0	1	0	Read (not recommended)
0	1	1	Read with following wait state bytes
1	0	0	Write
1	0	1	reserved
1	1	0	Address extension (3 address/command bytes)
1	1	1	reserved

8.3.4.2 Address modes

The SPI slave interface supports two address modes, 2 byte addressing and 3 byte addressing. With two byte addressing, the lower 13 address bits A[12:0] are selected by the SPI master, while the upper 3 bits A[15:13] are assumed to be 000b inside the SPI slave, thus only the first 8 Kbyte in the EtherCAT slave address space can be accessed. Three byte addressing is used for accessing the whole 64 Kbyte address space of an EtherCAT slave.

For SPI masters which do only support consecutive transfers of more than one byte, additional address extension commands can be inserted.

Table 52: Address modes without wait state byte (read access without wait state byte)

Byte	2 byte address mode		3 byte address mode	
0	A[12:5]	address bits [12:5]	A[12:5]	address bits [12:5]
1	A[4:0] CMD0[2:0]	address bits [4:0] read/write command	A[4:0] CMD0[2:0]	address bits [4:0] 3 byte addressing: 110b
2	D0[7:0]	data byte 0	A[15:13] CMD1[2:0] res[1:0]	address bits [15:13] read/write command two reserved bits, set to 00b
3	D1[7:0]	data byte 1	D0[7:0]	data byte 0
4 ff.	D2[7:0]	data byte 2	D1[7:0]	data byte 1

Table 53: Address modes for read access with wait state byte

Byte	2 byte address mode		3 byte address mode	
0	A[12:5]	address bits [12:5]	A[12:5]	address bits [12:5]
1	A[4:0] CMD0[2:0]	address bits [4:0] read command: 011b	A[4:0] CMD0[2:0]	address bits [4:0] 3 byte addressing: 110b
2	0xFF	wait state byte	A[15:13] CMD1[2:0] res[1:0]	address bits [15:13] read command: 011b two reserved bits, set to 00b
3	D0[7:0]	MISO data byte 0	0xFF	wait state byte
4	D1[7:0]	MISO data byte 1	D0[7:0]	MISO data byte 0
5 ff.	D2[7:0]	MISO data byte 2	D1[7:0]	MISO data byte 1

8.3.4.3 Interrupt request register (AL event register)

During the address phase, the SPI slave transmits the PDI interrupt request registers 0x0220-0x0221 (2 byte address mode), and additionally register 0x0222 for 3 byte addressing on SPI_MISO:

Table 54: Interrupt request register transmission

Byte	2 byte address mode			3 byte address mode		
	SPI_MOSI	SPI_MISO		SPI_MOSI	SPI_MISO	
0	A[12:5]	I0[7:0]	interrupt request register 0x0220	A[12:5]	I0[7:0]	interrupt request register 0x0220
1	A[4:0] CMD0[2:0]	I1[7:0]	interrupt request register 0x0221	A[4:0] CMD0[2:0]	I1[7:0]	interrupt request register 0x0221
2	(data phase)			A[15:13] CMD1[2:0]	I2[7:0]	interrupt request register 0x0222

The order of PDI interrupt request registers 0x0220 and 0x0221 can be swapped by SPI slave configuration 0x0150[6], so SyncManager interrupts can be polled by a short one byte SPI access (terminated after the first byte).

8.3.4.4 Write access

In the data phase of a write access, the SPI master sends the write data bytes to the SPI slave (SPI_MOSI). The write access is terminated by taking back SPI_SEL after the last byte. The SPI_MISO signal is undetermined during the data phase of write accesses.

8.3.4.5 Read access

In the data phase of a read access, the SPI slave sends the read data bytes to the SPI master (SPI_MISO).

Read wait state

Between the last address phase byte and the first data byte of a read access, the SPI master has to wait for the SPI slave to fetch the read data internally. Subsequent read data bytes are prefetched automatically, so no further wait states are necessary.

The SPI master can choose between these possibilities, using different read commands:

- The SPI master waits for the specified worst case internal read time t_{read} after the last address/command byte and before the first clock cycle of the data phase.
- The SPI master inserts one wait state byte after the last address/command byte. The wait state byte must have a value of 0xFF transferred on SPI_MOSI. Using the wait state byte is recommended.

Read termination

The SPI_MOSI signal is used for termination of the read access by the SPI master. For the last data byte, the SPI master has to set SPI_MOSI to high (read termination byte = 0xFF), so the slave will not prefetch the next read data internally. If SPI_MOSI is low during a data byte transfer, at least one more byte will be read by the master afterwards.

8.3.4.6 Bidirectional operation with SPI_SEL

The SPI slave also supports bidirectional operation using SPI_SEL. The protocol is again the same, just with a few restrictions:

- The interrupt request register values are not available as SPI_MISO data during address phase.
- Every read command requires a wait state byte, the normal read without wait state byte is not supported. The wait state byte is required for bus driver change (SPI master drives during address phase, SPI slave drives during data phase).
- Read termination is not supported/available. The SPI slave internally fetches the next data bytes, even if they are not transferred later. This has to be taken into account e.g. for adjacent SyncManagers, which have to be avoided.

The SPI_MISO_ENA signal might be used for drivers in the data path between master and slave, it is active while the SPI slave drives SPI_DATA.

8.3.4.7 SPI masters with 2 byte and 4 byte minimum access

The address phase can be extended by an arbitrary number of address extension bytes (command 0b110) to achieve a total access length that is supported by the SPI master. The address portion of the last address extension byte is used for the access.

The length of the data phase is in control of the master and can be set to the appropriate length, the length of the address phase has to be extended. The address phase of a read access can be set to a multiple of 2/4 by using the 3 byte address mode and a wait state byte. The address phase of a write access can be enhanced to 4 bytes using 3 byte address mode and an additional address extension byte (byte 2) according to Table 55.

Table 55: Write access for 2 and 4 byte SPI masters

Byte	2 byte SPI master		4 byte SPI master	
0	A[12:5]	address bits [12:5]	A[12:5]	address bits [12:5]
1	A[4:0]	address bits [4:0]	A[4:0]	address bits [4:0]
	CMD0[2:0]	write command: 100b	CMD0[2:0]	3 byte addressing: 110b
2	D0[7:0]	data byte 0	A[15:13]	address bits [15:13]
			CMD1[2:0]	3 byte addressing: 110b
			res[1:0]	two reserved bits, set to 00b
3	D1[7:0]	data byte 1	A[15:13]	address bits [15:13]
			CMD2[2:0]	write command: 100b
			res[1:0]	two reserved bits, set to 00b
4	D2[7:0]	data byte 2	D0[7:0]	data byte 0
5	D3[7:0]	data byte 3	D1[7:0]	data byte 1
6	D4[7:0]	data byte 4	D2[7:0]	data byte 2
7	D5[7:0]	data byte 5	D3[7:0]	data byte 3

NOTE: The address phase of a write access can be further extended by an arbitrary number of address extension bytes containing 110b as the command. The address phase of a read access can also be enhanced with additional address extension bytes (the read wait state has to be maintained anyway).

8.3.5 SPI_SEL_MISO/SPI_SEL_MOSI configured SyncManager access

Two additional select signals become available when SPI slave configuration register 0x0150[7]=1. In contrast to the random access protocol used with SPI_SEL, the new select signals do not require an address phase/command phase. Instead, these signals allow immediately reading/writing SyncManager data, i.e., they are ideal for cyclic process data access.

Typically, a μ Controller will use both SPI_SEL, and one or two of the additional select signals.

The corresponding SyncManagers for SPI_SEL_MISO/SPI_SEL_MOSI depend on the PDI (instance PDI0 or instance PDI1), and on the select signal:

Table 56: SyncManager used by SPI slave for SPI_SEL_MISO/SPI_SEL_MOSI

Access	PDI0	PDI1	SyncManager direction
SPI_SEL_MISO	SyncManager 15	SyncManager 13	ECAT write, PDI read
SPI_SEL_MOSI	SyncManager 14	SyncManager 12	ECAT read, PDI write

Each of the two select signals can be used from the moment when the corresponding SyncManager is activated by the master (there is no dependency on the AL status register 0x0130).

The SyncManager configuration does not only define the start address of an SPI access, it also defines the length. MISO data will be zero for any byte beyond the configured SyncManager buffer length. MOSI data will not be written beyond the SyncManager buffer length.

The SPI slave PDI will check SyncManager direction and activation before using it.

It is allowed and potentially beneficial to assert SPI_SEL_MISO and SPI_SEL_MOSI simultaneously. In unidirectional mode, MISO and MOSI data exchange happens during the same SPI_CLK cycles, maximizing throughput on the interface. In bidirectional mode, MISO and MOSI data exchange happens one after the other. Both select signals have to be active during the whole access (before first SPI_CLK edge until after last SPI_CLK edge), and they need to have the active edge within a short time window, which allows the ESC to align the required accesses. The length of the SyncManager buffers does not have to be identical for simultaneous assertion of SPI_SEL_MISO/SPI_SEL_MOSI.

8.3.5.1 SPI_SEL_MISO access

Between the active edge of SPI_SEL_MISO and the first data byte, the SPI master has to wait for the SPI slave to fetch the read data internally. Subsequent read data bytes are prefetched automatically, so no further wait states are necessary.

The SPI master can choose between these possibilities:

- The SPI master waits for the specified worst case internal read time t_{read} before the first clock edge. Depending on the SPI master, and the SPI_CLK frequency, the time between SPI_SEL and first clock edge might be sufficient for the ESC to fetch the data. In this case, this setting is preferred.
- The SPI master inserts one wait state byte before the first actual data byte. The wait state byte has to be configured using SPI slave extended configuration register 0x0152[2] / 0x0192[2]. The data presented during the wait state byte has to be ignored.
Using the wait state byte is usually recommended.

8.3.5.2 Bidirectional operation with SPI_SEL_MISO/SPI_SEL_MOSI

Bidirectional operation can be selected when using SPI_SEL_MISO/SPI_SEL_MOSI. If only one of these two select signals is used, there is no difference to unidirectional mode (except for the pinout of course).

Simultaneous assertion of SPI_SEL_MISO and SPI_SEL_MOSI signals is also supported, as long as the active edge is within a short time window, which allows the ESC to detect the simultaneous access. First, the MISO data exchange will be executed. A turnaround byte without data exchange will then allow the bus driver to change, followed by the transfer of the MOSI data bytes. Optionally, the MISO data exchange starts with a wait state byte (configured by 0x0152[2]/0x192[2]).

This example timing diagram shows the byte ordering, when both SPI_SEL_MISO and SPI_SEL_MOSI are asserted. SPI_CLK is using x4 operation (2 clocks per byte), clock edge reference is not accurate.

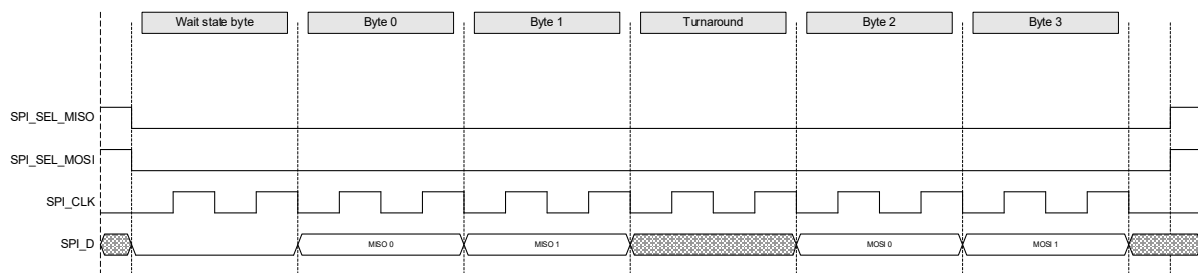


Figure 43: SPI slave bidirectional mode with simultaneous SEL_MISO/SEL_MOSI access

The SPI_MISO_ENA signal might be used for drivers in the data path between master and slave, it is active while the SPI slave drives SPI_D.

8.3.6 MISO/MOSI data width x1, x2, x4, x8

The SPI slave built in the ET11?? supports different data width modes: 1, 2, 4, or 8 bit. The data width is selected by SPI slave configuration, it cannot be selected during operation, and it does not change within an access.

The protocol used for 1 bit operation is also used for other data widths, only the number of SPI_CLK cycles is reduced.

8.3.7 Access errors and SPI status flag

The following reasons for SPI access errors are detected by the SPI slave:

- The number of clock cycles recognized while SPI_SEL/SPI_SEL_MISO/SPI_SEL_MOSI is asserted is not a multiple of 8 (incomplete bytes were transferred).
- For a read access, a clock cycle occurred while the slave was busy fetching the first data byte.
- For an SPI_SEL read access, the data phase was not terminated by setting SPI_MOSI to high for the last byte.
- For an SPI_SEL read access, additional bytes were read after termination of the access.
- Invalid combination of select signals SPI_SEL, SPI_SEL_MISO, SPI_SEL_MOSI
- SPI_SEL_MISO and SPI_SEL_MOSI active at the same time, but not simultaneously (too much skew)
- For SPI_SEL_MISO/SPI_SEL_MOSI access: SyncManager configuration is not valid

A wrong SPI access will have these consequences:

- The PDI error counter 0x030D/0x0340 will be incremented. For the first error, an SPI PDI error code will be stored in the PDI error code register 0x030E:0x030F/0x0341:0x0342, indicating details of the error reason.
- A status flag will indicate the error until the next access (not for SPI mode 0/2 with normal data out sample)
- For debugging, ESC configuration B4[11] can be used to enable a PDI error signal on the LED_RUN/LED_ERR pins, for PDI0/PDI1 respectively.

A wrong SPI access will also have these consequences, if PDI configuration 0x0150[7]=0 and PDI1 is not used:

- Registers will not take-over write data (nevertheless, RAM will be written).
- Special functions are not executed (e.g., SyncManager buffer switching).

A status flag, which indicates if the last access had an error, is available in any mode except for SPI mode 0/2 with normal data out sample. The status flag is presented on SPI_MISO after the slave is selected (SPI_SEL or SPI_SEL_MISO) and until the first clock cycle occurs. So the status can be read either between two accesses by assertion of SPI_SEL without clocking, or at the beginning of an SPI_SEL/SPI_SEL_MISO access just before the first clock cycle. The status flag will be high for a good access, and low for a wrong access. If you only want to get the status flag, prefer using SPI_SEL because SPI_SEL_MISO starts fetching the data internally (even without any SPI clock).

8.3.8 Timing specifications

Table 57: SPI slave timing characteristics IP core

Parameter	Min	Max	Comment
PRELIMINARY TIMING			
W	8 / 16 / 32 bit		Internal PDI bus width
P	10 / 20 / 40 ns		Internal PDI bus period
M	1 / 2		Number of used PDI interfaces (PDI1 used → M=2)
t _{SEL_to_CLK}	x ³		First SPI_CLK cycle after SPI_SEL asserted
t _{CLK_to_SEL}	a) x ³ b) t _{CLK} /2 + x ³		De-assertion of SPI_SEL after last SPI_CLK cycle a) SPI mode 0/2, SPI mode 1/3 with normal data out sample b) SPI mode 1/3 with late data out sample
t _{read}	4.5 * P * M		Only for read access between address/command and first data byte. Can be ignored if BUSY or wait state bytes are used.
t _{SEL_to_MISO_valid}		x ³	Status/interrupt byte 0 bit 7 valid after SPI_SEL asserted
t _{SEL_to_MISO_invalid}	0 ns	x ³	Status/interrupt byte 0 bit 7 invalid after SPI_SEL de-asserted
t _{STATUS_valid}	x ³		Time until status of last access is valid. Can be ignored if status is not used.
t _{access_delay}	x ³		Delay between SPI accesses
t _{MOSI_setup}	x ³		SPI_MOSI valid before SPI_CLK edge
t _{MOSI_hold}	x ³		SPI_MOSI valid after SPI_CLK edge
t _{CLK_to_MISO_valid}		x ³	SPI_MISO valid after SPI_CLK edge
t _{CLK_to_MISO_invalid}	0 ns		SPI_MISO invalid after SPI_CLK edge
t _{IRQ_delay}	4 * P		Internal delay between AL event and SPI_IRQ output to enable correct reading of the interrupt registers.

Table 58: SPI slave max. SPI_CLK frequency f_{CLK}=1/t_{CLK}

Number of PDIs	One PDI (PDI0)			Two PDIs (PDI0+PDI1)		
Internal PDI bus frequency	25 MHz	50 MHz	100 MHz	25 MHz	50 MHz	100 MHz
PRELIMINARY TIMING						
1x MISO/MOSI line	30 MHz	40 MHz	40 MHz	23 MHz	40 MHz	40 MHz
2x MISO/MOSI lines	12 MHz	25 MHz	40 MHz	11 MHz	21 MHz	39 MHz
4x MISO/MOSI lines	5 MHz	10 MHz	20 MHz	4 MHz	9 MHz	16 MHz
8x MISO/MOSI lines	3 MHz	7 MHz	13 MHz	3 MHz	6 MHz	11 MHz

NOTE: The values are additionally reduced by actual synthesis results

³ EtherCAT IP core: time depends on synthesis results

Table 59: Read/write timing diagram symbols

Symbol	Comment
A15..A0	Address bits [15:0]
D0_7..D0_0	Data bits byte 0 [7:0]
D1_7..D1_0	Data bits byte 1 [7:0]
I0_7..I0_0	Interrupt request register 0x0220 [7:0]
I1_7..I1_0	Interrupt request register 0x0221 [7:0]
I2_7..I2_0	Interrupt request register 0x0222 [7:0]
C0_2..C0_0	Command 0 [2:0]
C1_2..C1_0	Command 1 [2:0] (3 byte addressing)
Status	0: last SPI access had errors 1: last SPI access was correct

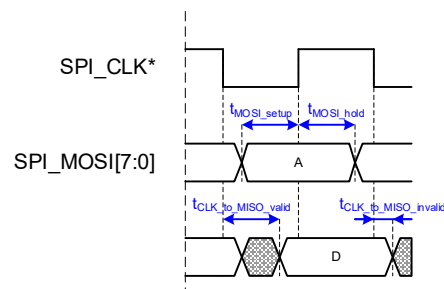


Figure 44: Basic SPI_MOSI/SPI_MISO timing (*refer to access diagrams for relevant edges of SPI_CLK)

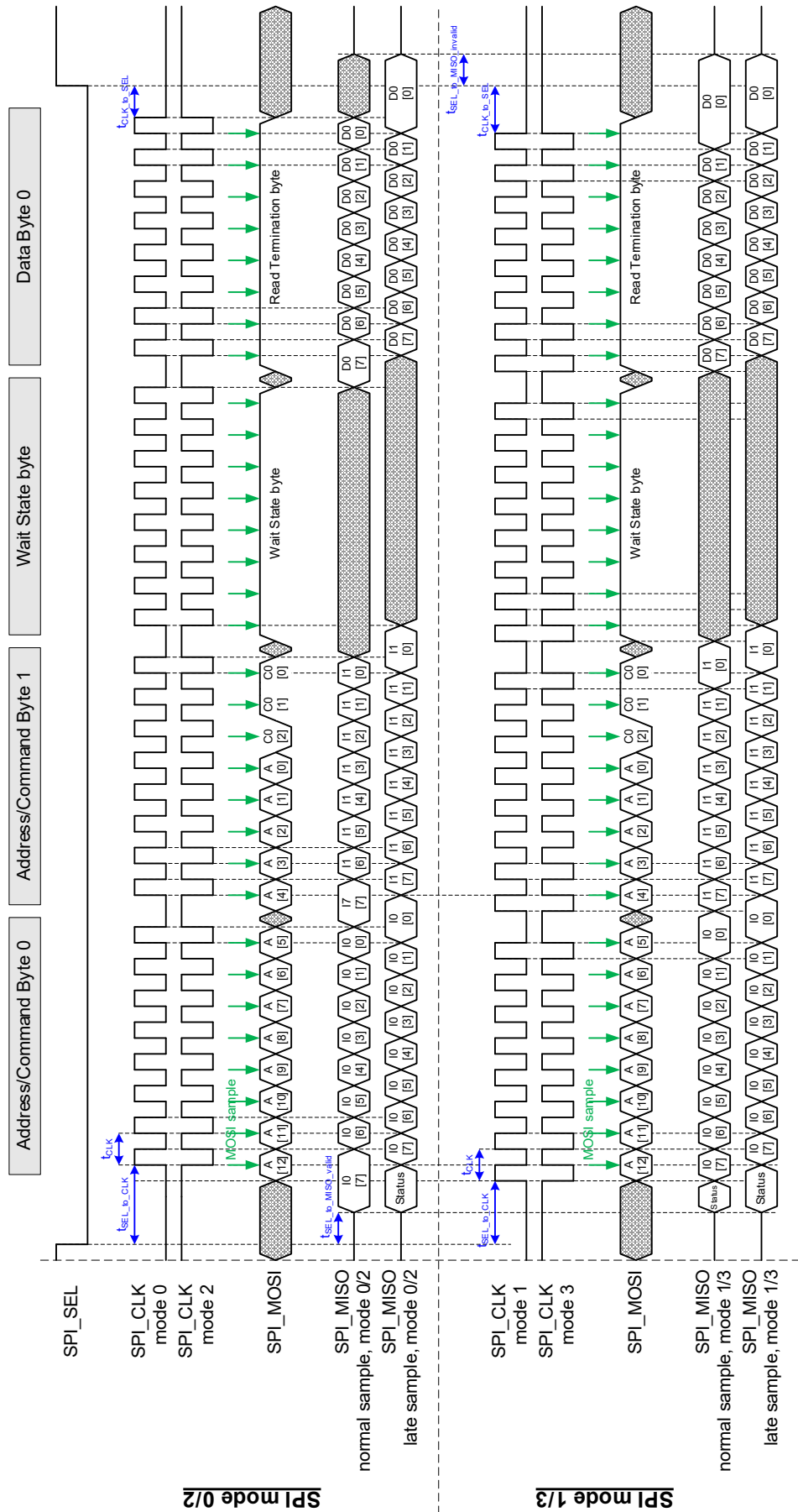


Figure 45: SPI_SEL random read access (2 byte addressing, 1 byte read data) with wait state byte

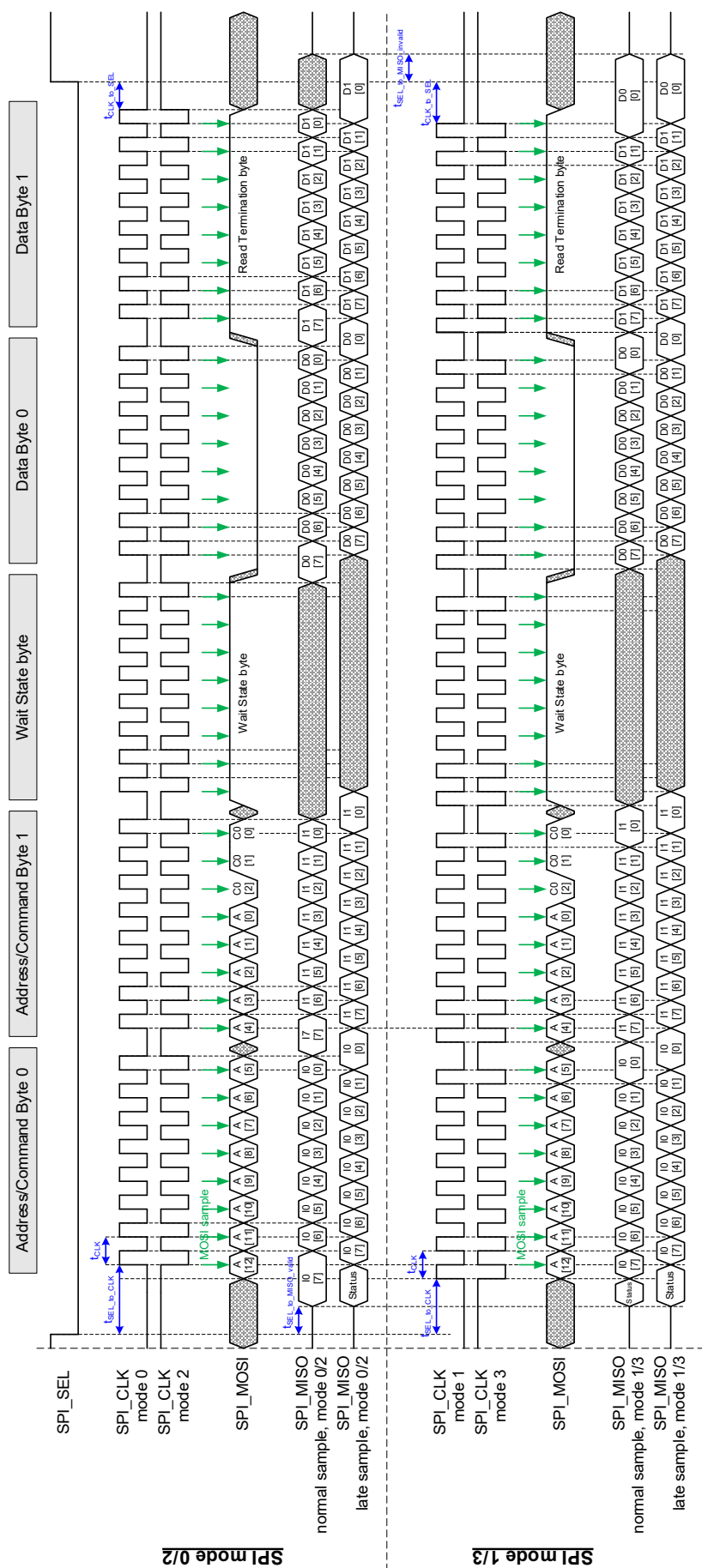


Figure 46: SPI_SEL random read access (2 byte addressing, 2 byte read data) with wait state byte

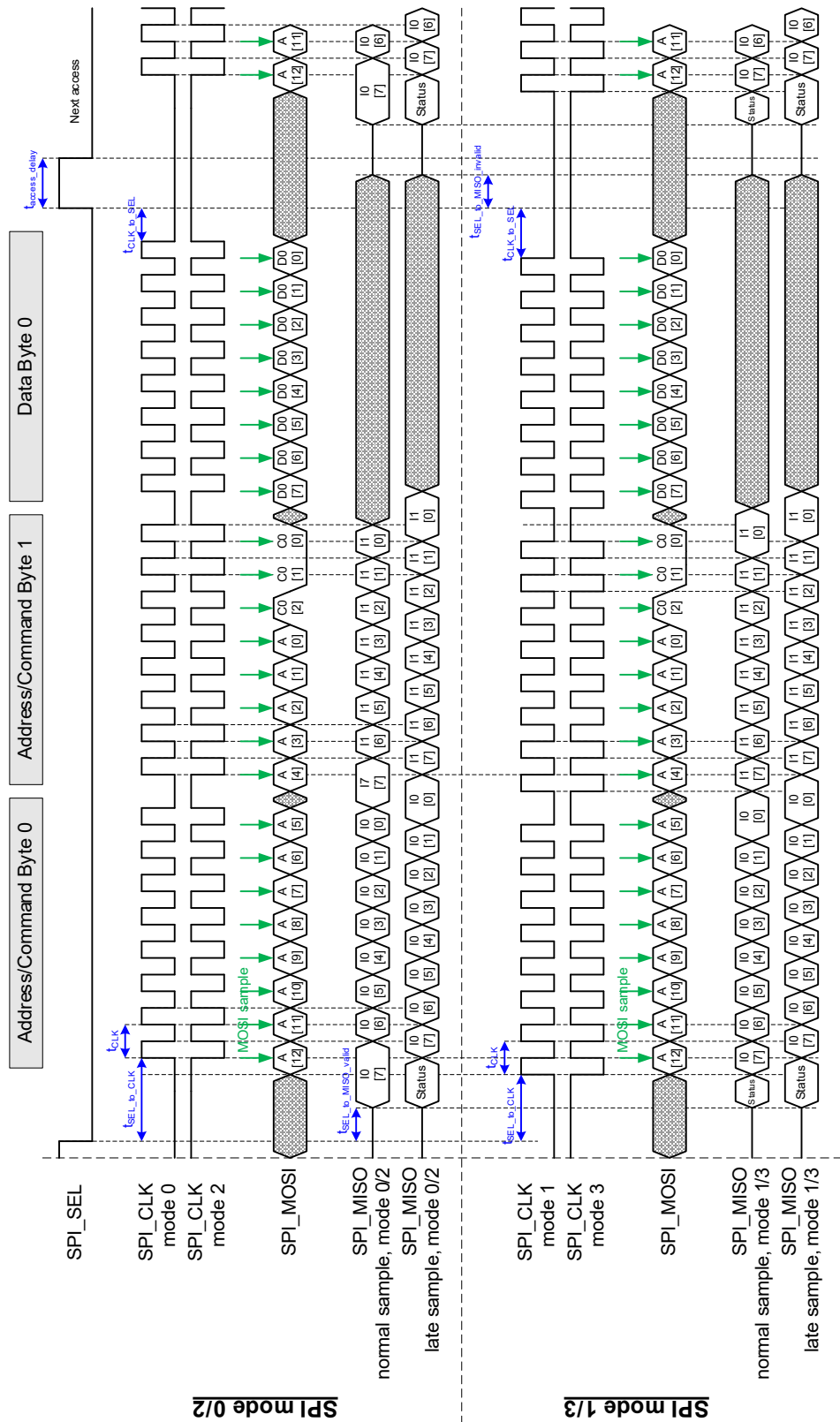


Figure 47: SPI_SEL random write access (2 byte addressing, 1 byte write data)

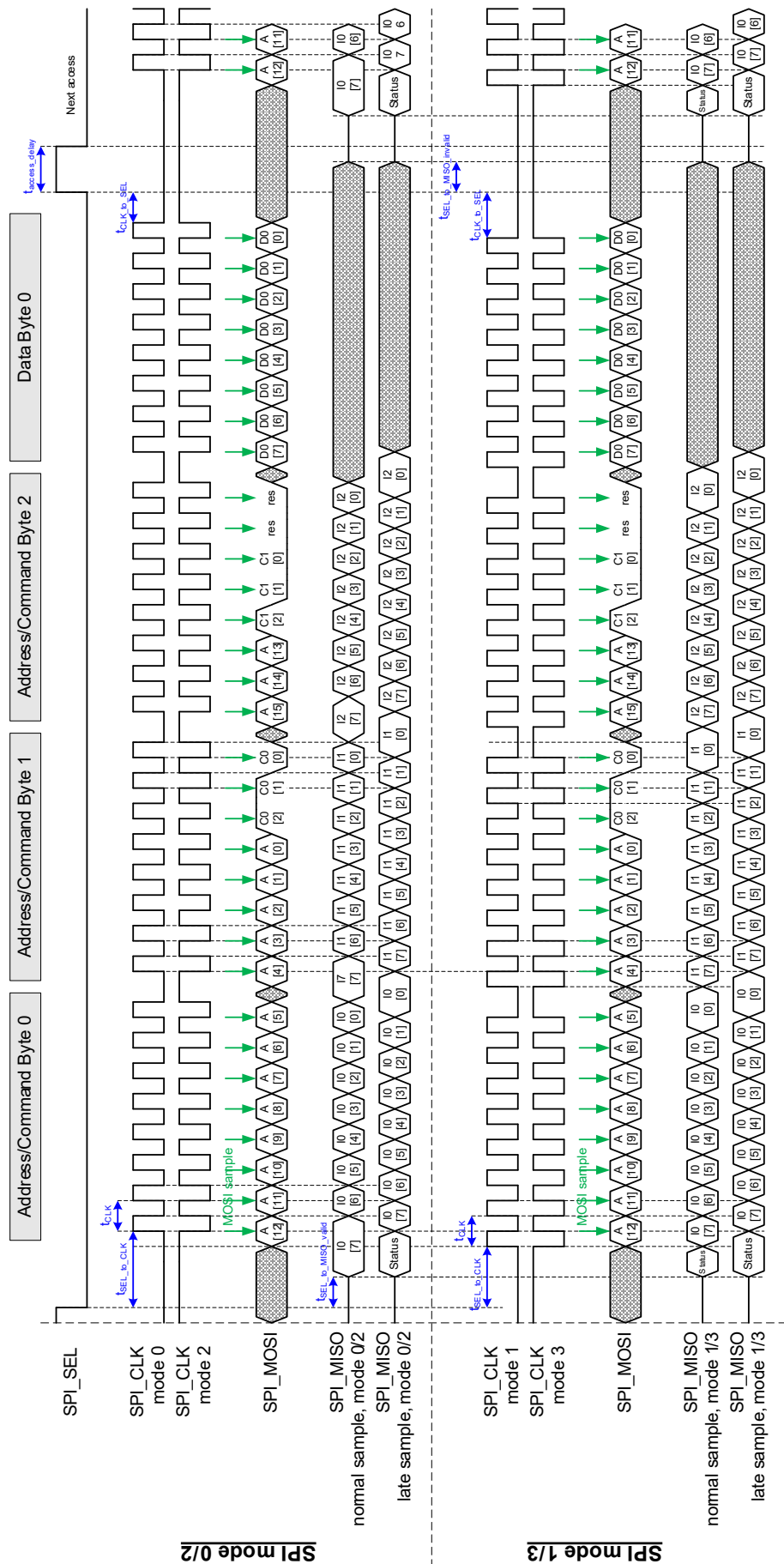


Figure 48: SPI_SEL random write access (3 byte addressing, 1 byte write data)

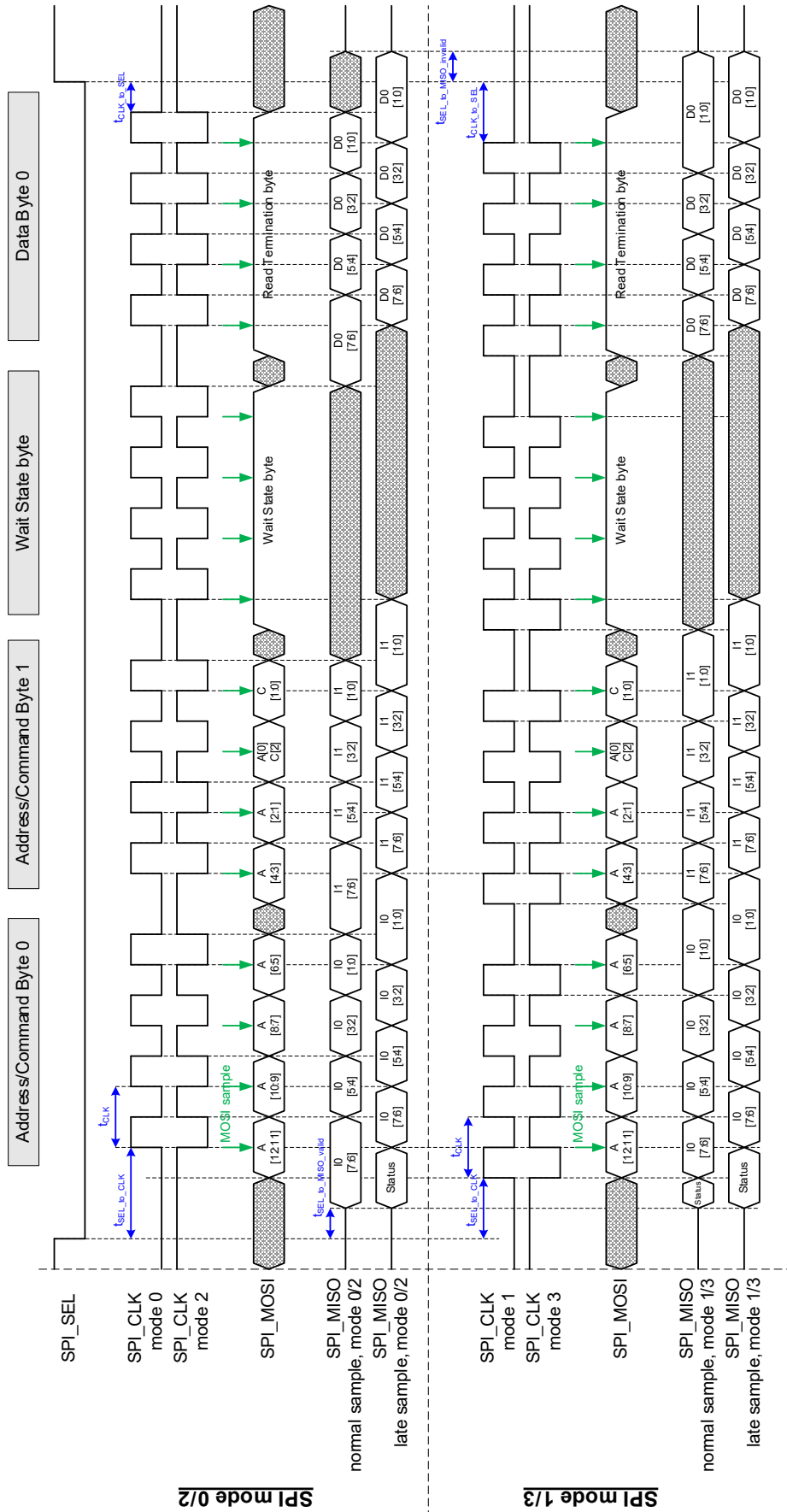


Figure 49: SPI_SEL random read access (2 bit data width, with wait state byte)

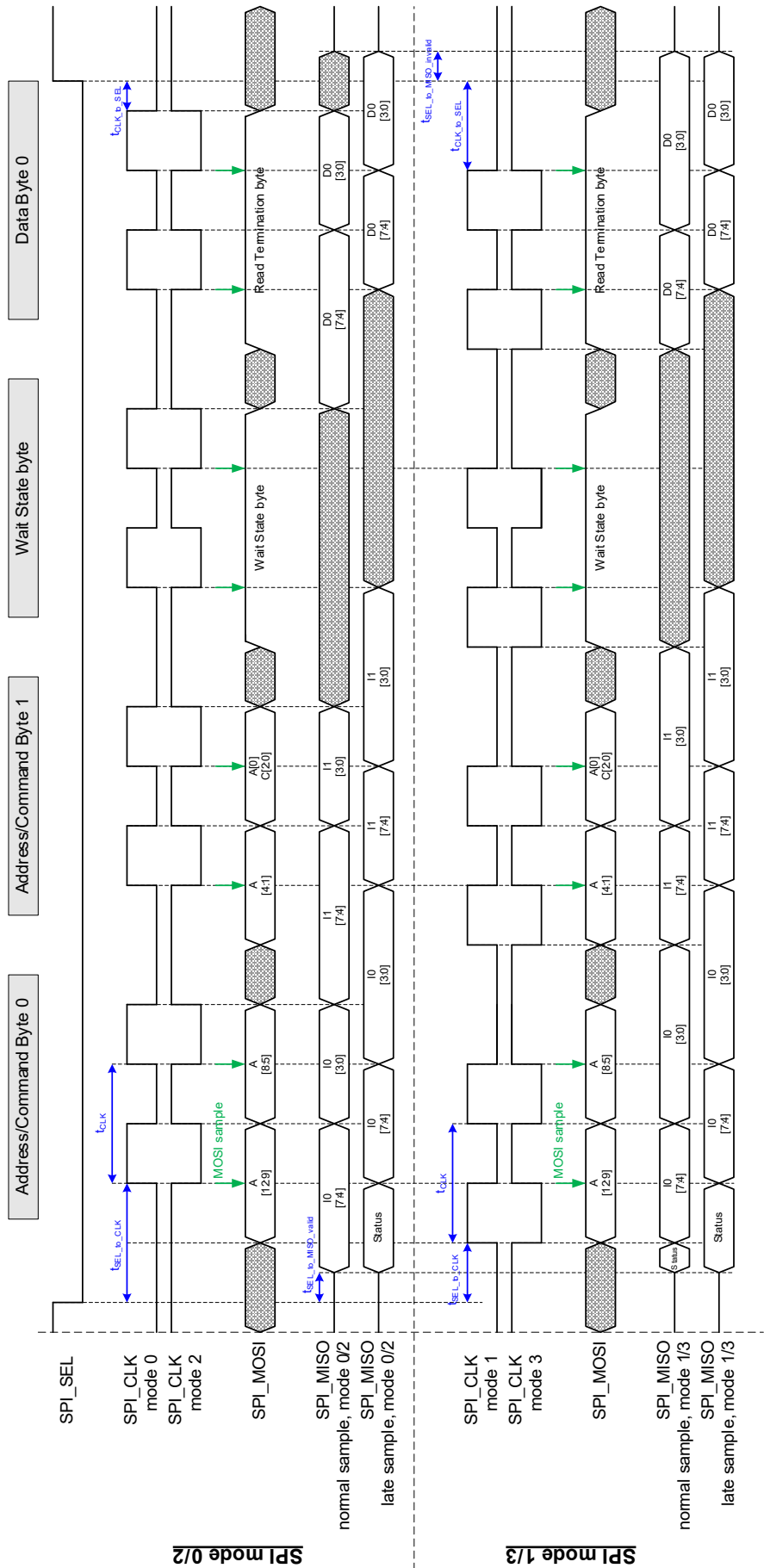


Figure 50: SPI_SEL random read access (4 bit data width, with wait state byte)

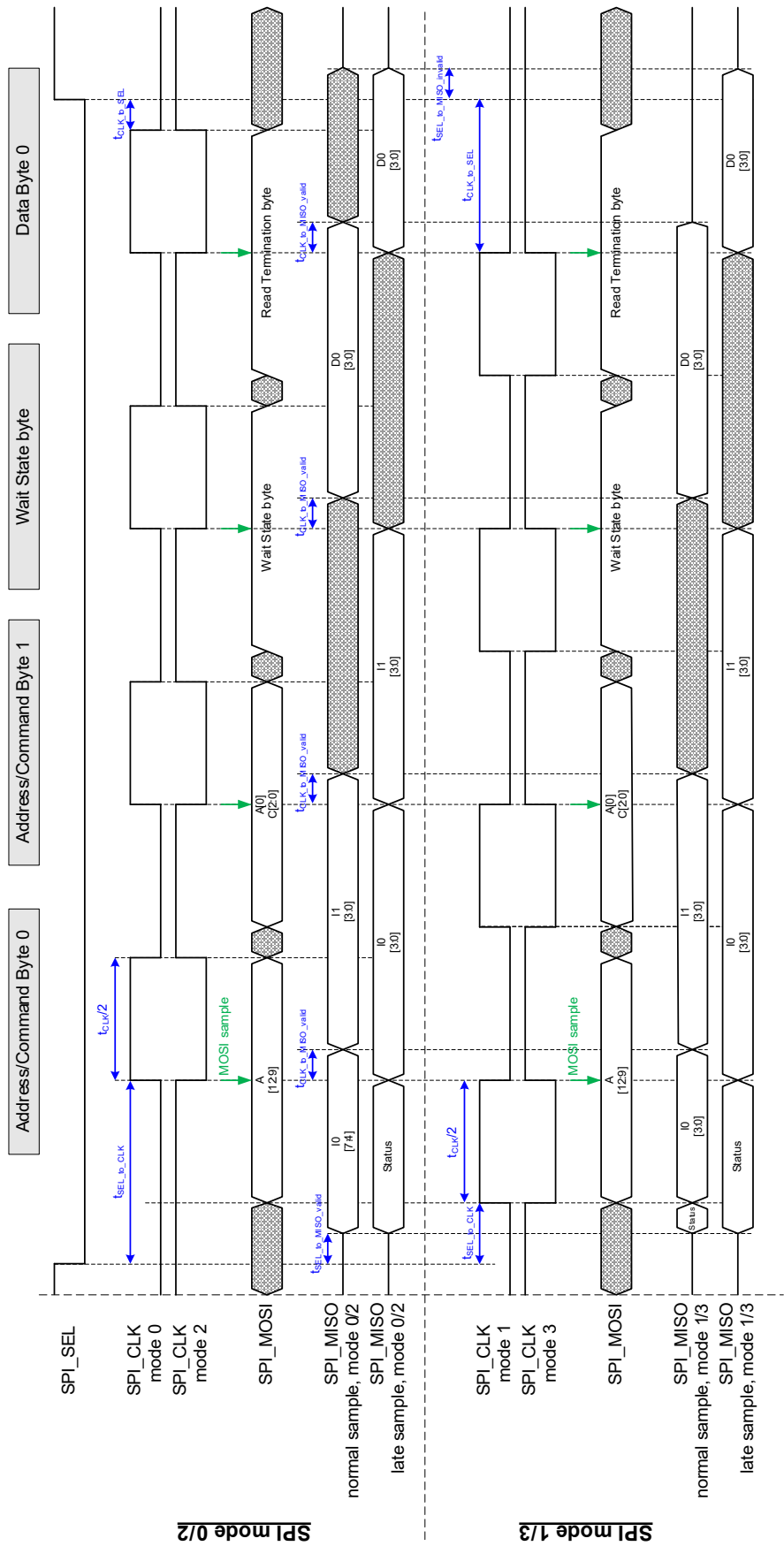


Figure 51: SPI_SEL random read access (8 bit data width, with wait state byte)

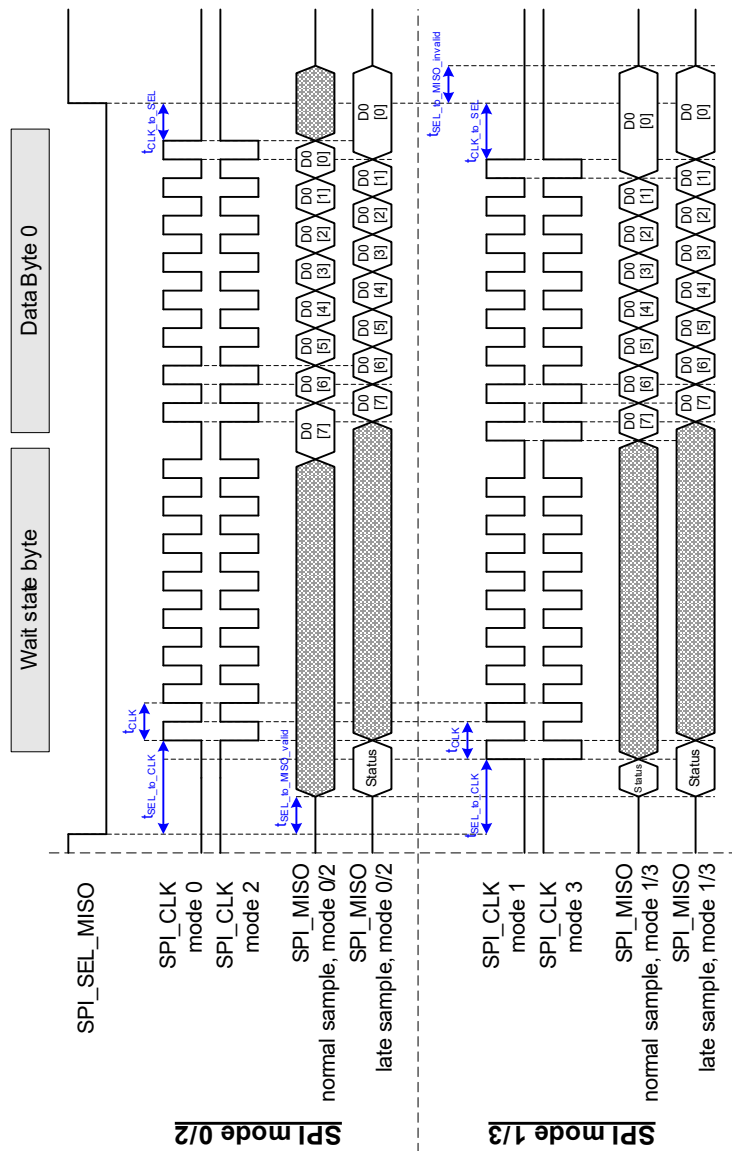


Figure 52: SPI_SEL_MISO access (wait state byte)

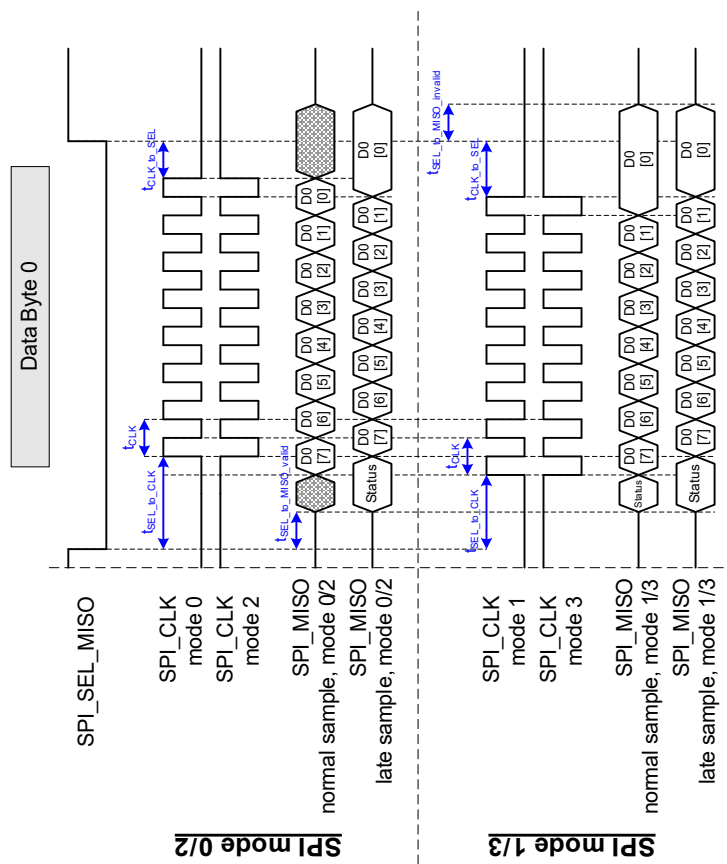


Figure 53: SPI_SEL_MISO access (no wait state byte)

8.4 Asynchronous 8/16/32 bit μ Controller interface

8.4.1 Interface

The asynchronous μ Controller interface uses demultiplexed address and data busses. The bidirectional data bus can be either 8, 16, or 32 bit wide. The signals of the asynchronous μ Controller interface are shown below. The IP core signals have a prefix "PDI_UC_" for PDI0, which is omitted in this chapter. All signals are denoted with typical polarity configuration.

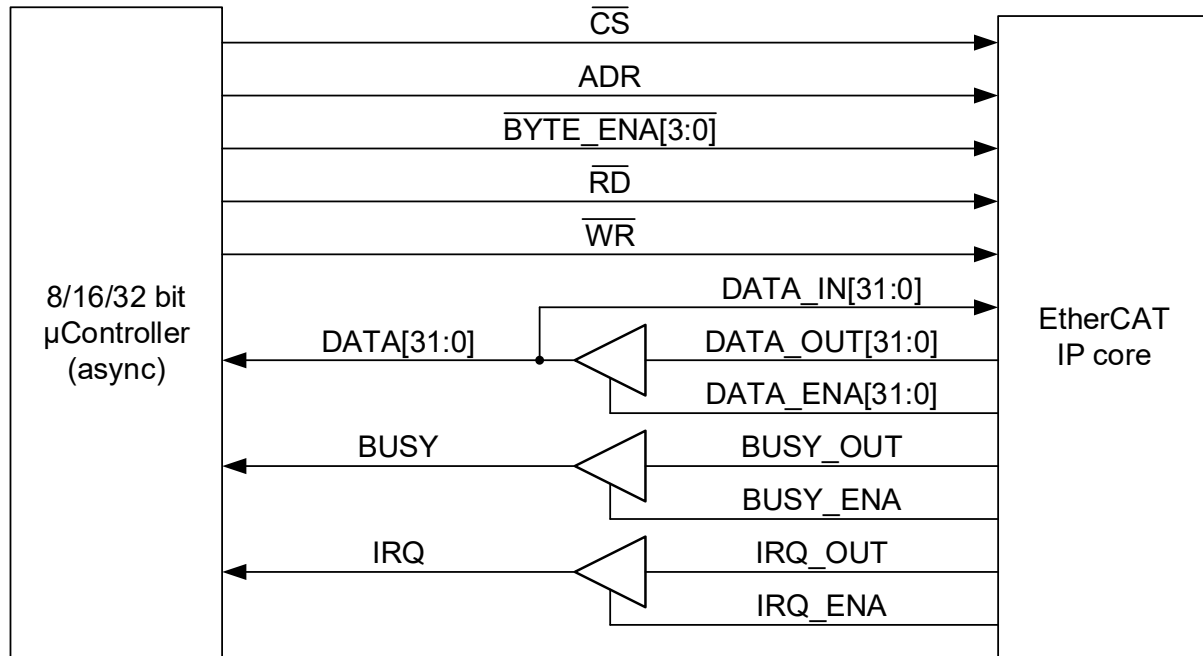


Figure 54: μ Controller interface interconnection

Table 60: μ Controller signals

Signal async	Direction	Description	Signal polarity
CS	IN (μ C $\rightarrow$ ESC)	Chip select	Typical: act. low
ADR[15:0]	IN (μ C $\rightarrow$ ESC)	Address bus	
BYTE_ENA[3:0]	IN (μ C $\rightarrow$ ESC)	Byte enable (for 16/32 bit μ C). BYTE_ENA[1] is used as BHE for 16 bit μ C if configured.	Typical: act. low
RD	IN (μ C $\rightarrow$ ESC)	Read command	Typical: act. low
WR	IN (μ C $\rightarrow$ ESC)	Write command	Typical: act. low
DATA_IN[31:0]	BD (μ C $\leftrightarrow$ ESC)	Data bus	act. high
DATA_OUT[31:0]	BD (μ C $\leftrightarrow$ ESC)	Data bus	act. high
DATA_ENA[31:0]	BD (μ C $\leftrightarrow$ ESC)	Data bus	act. high
BUSY_OUT	OUT (ESC $\rightarrow$ μ C)	Busy output driver value	
BUSY_ENA	OUT (ESC $\rightarrow$ μ C)	Busy output driver enable	Act. high
IRQ_OUT	OUT (ESC $\rightarrow$ μ C)	Interrupt output driver value	
IRQ_ENA	OUT (ESC $\rightarrow$ μ C)	Interrupt output driver enable	Act. high

The interfaces with 8/16 bit data bus width only use DATA[7:0]/DATA[15:0], and BYTE_ENA[1:0].

Some μ Controllers have a READY signal, this is the same as the BUSY signal, just with inverted polarity.

8.4.2 Configuration

The 8/16/32 bit asynchronous μ Controller interface is selected with PDI type 0x08/0x09 in the PDI0 control register 0x0140, or PDI1 control register 0x0180. It supports different configurations, which are located in registers 0x0150:0x0153 / 0x0190:0x0193.

8.4.3 μ Controller access

The 8 bit μ Controller interface reads or writes 8 bit per access, the 16 bit μ Controller interface supports both 8 bit and 16 bit read/write accesses, and the 32 bit μ Controller supports 8/16/32 bit read/write accesses.

When the μ Controller interface is configured for a larger data bus width than the internal PDI bus width, the access will be serialized by the EtherCAT IP core. The lower addresses are accessed first. Byte enable signals are taken into account.

EtherCAT devices use Little Endian byte ordering.

8.4.3.1 μ Controller 16 bit interface access types

The byte enable signals BYTE_ENA[1:0], or alternatively, the least significant address bit ADR[0] together with BYTE_ENA[1] (byte high enable, BHE) are used to distinguish between 8 bit low byte access, 8 bit high byte access and 16 bit access.

Table 61: 8 bit high/low byte and 16 bit access distinction

ADR[0]	BYTE_ENA[1] (act. low). 0x0152[6]=0 / 0x0192[6]=0	BYTE_ENA[1:0] (act. low). 0x0152[6]=1 / 0x0192[6]=1	Access
0	0	00	16 bit access to ADR[15:0] and ADR[15:0]+1 (low and high byte)
0	1	10	8 bit access to ADR[15:0] (low byte, even address)
1	0	01	8 bit access to ADR[15:0] (high byte, odd address)
1	1	11	invalid access

8.4.3.2 μ Controller 32 bit interface access types

The byte enable signals BYTE_ENA[3:0] are used to distinguish between 8/16/32 bit access.

Table 62: 32 bit access distinction

ADR[1:0]	BYTE_ENA[3:0] (act. low).	Access
00	1110	8 bit access to address 0
00	1101	8 bit access to address 1
00	1011	8 bit access to address 2
00	0111	8 bit access to address 3
00	1100	16 bit access to address 0
00	0011	16 bit access to address 2
00	1111	32 bit access to address 0
other	other	Invalid access

8.4.4 Write access

CS and/or WR can be asserted in any order and the access will start when both are asserted. Either CS or WR may be permanently asserted. Address, byte high enable and write data are asserted on the falling edge of WR (active low). Once the μ Controller interface is not BUSY, a rising edge on WR completes the μ Controller access. At the end of a write access, CS and/or WR can be de-asserted in any order. Shortly after the rising edge of WR, the access can be finished by de-asserting ADR, BYTE_ENABLE and DATA. The μ Controller interface indicates its internal operation with the BUSY signal. Since the BUSY signal is only driven while CS is asserted, the BUSY driver will be released after CS de-assertion.

Depending on the configuration "Write on falling edge", the internal write access is either performed after the falling edge of WR, or after the rising edge of WR.

Write at falling edge of WR

If the falling edge is selected, the internal write operation begins with the falling edge of WR, and BUSY indicates when the write operation is finished. The internal write operation is performed during the external write access.

Write at rising edge of WR

If the rising edge of WR is selected, the internal operation begins with the rising edge of WR, i.e., after the external write access. Thus, the external write access is very fast, but an access immediately following will be delayed by the preceding write access. The worst-case access time is higher in this case.

8.4.5 Read access

CS and/or RD can be asserted in any order and the access will start when both are asserted. Either CS or RD may be permanently asserted. Address and BHE have to be valid before the falling edge of RD, which signals the start of the access. The μ Controller interface will show its BUSY state afterwards – if it is not already busy executing a preceding write access – and release BUSY when the read data are valid. The read data will remain valid until either ADR, ENABLE, RD or CS change. The data bus will be driven while CS and RD are asserted. BUSY will be driven while CS is asserted. At the end of a read access, CS and/or RD can be de-asserted in any order.

With "Read BUSY delayed" configuration, BUSY de-assertion for read accesses can be additionally delayed by 20 ns, so external DATA setup requirements in respect to BUSY can be met.

8.4.6 μ Controller access errors

These reasons for μ Controller access errors are detected by the μ Controller interface:

- Read or write access to the 16 bit interface with A[0]=1 and BYTE_ENABLE[1](act. low)=1, i.e. an access to an odd address without byte high enable.
- De-assertion of WR (or de-assertion of CS while WR remains asserted) while the μ Controller interface is BUSY.
- De-assertion of RD (or de-assertion of CS while RD remains asserted) while the μ Controller interface is BUSY (read has not finished).

A wrong μ Controller access will have these consequences:

- The PDI error counter 0x030D/0x0340 will be incremented.
- For A[0]=1 and ENABLE[1] (act. low)=1 accesses, no access will be performed internally.
- De-assertion of WR (or CS) while the μ Controller interface is BUSY might corrupt the current and the preceding transfer (if it is not completed internally). Registers might accept write data and special functions (e.g., SyncManager buffer switching) might be performed.
- If RD (or CS) is de-asserted while the μ Controller interface is BUSY (read has not finished), the access will be terminated internally. Internal byte transfers might still be completed, so special functions (e.g., SyncManager buffer switching) might be performed.

The reason for the access error can be found in the PDI error code register 0x030E:0x030F/0x0341:0x0342.

8.4.7 Default BUSY state

In normal mode, the ESC gets busy when an access is detected (CS and WR asserted/de-asserted, or CS and RD asserted). The BUSY signal remains active until the access is finished, or CS gets de-asserted.

In default busy state operation, the ESC indicates busy directly after CS is asserted – regardless of WR and RD. It remains busy until the access is finished (read data valid, write access finished). If the μ Controller does not assert RD or WR, the ESC indicates busy until CS is de-asserted.

8.4.8 Connection with 16 bit μ Controllers without byte addressing

If the ESC is connected to 16 bit μ Controllers/DSPs which only support 16 bit addressing, ADR[0] and BYTE_ENABLE[1:0] of the EtherCAT device have to be tied to GND, so the ESC will always perform 16 bit accesses. All other signals are connected as usual.

8.4.9 Connection with 8 bit μ Controllers

If the ESC is connected to 8 bit μ Controllers, the DATA[31:8] signals are not used.

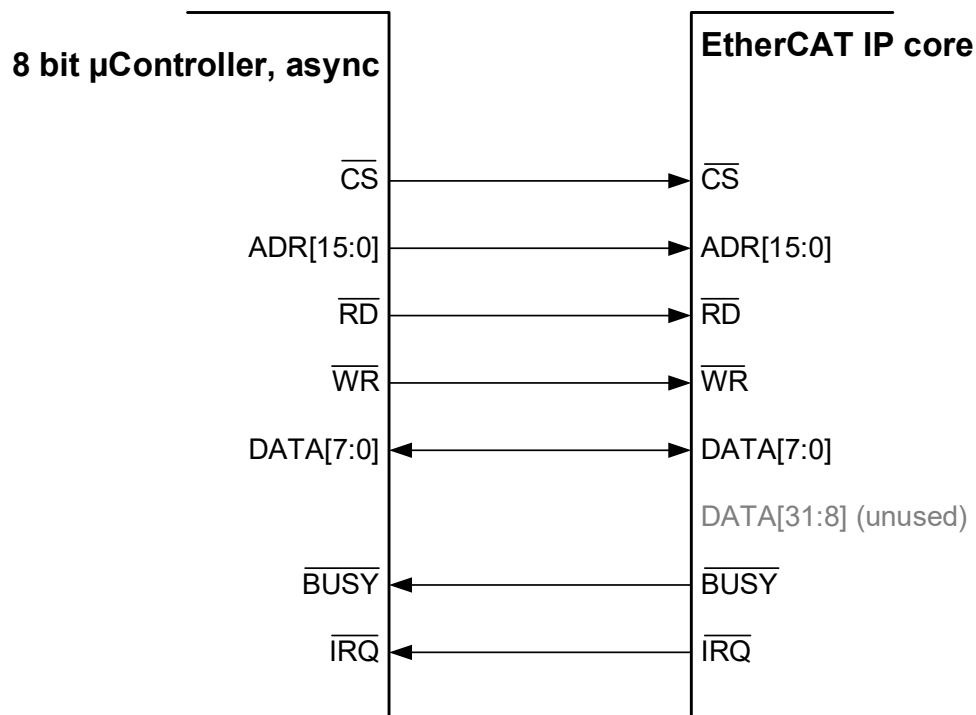


Figure 55: Connection with 8 bit μ Controllers

8.4.10 Timing specification

Table 63: µController timing characteristics IP core

Parameter	Min	Max	Comment
PRELIMINARY TIMING			
W		8 / 16 / 32 bit	Internal PDI bus width
P		10 / 20 / 40 ns	
M		1 / 2	Number of used PDI interfaces (PDI1 used → M=2)
D		8 / 16 / 32 bit	µController PDI data bus width
t _{CS_to_BUSY}		x ⁴	BUSY driven and valid after CS assertion
t _{ADR_BE_setup}	x ⁴		ADR and BYTE_ENA valid before RD assertion
t _{RD_to_DATA_driven}	0 ns ⁵		DATA bus driven after RD assertion
t _{RD_to_BUSY}	0 ns ⁵	x ⁴	BUSY asserted after RD assertion
t _{read}			External read time (RD assertion to BUSY de-assertion) with normal read busy output (0x0152[0]), all BYTE_ENA enabled. Additional 20 ns with delayed read busy output.
		a) 25 ns $+ \left(2 + \left\lceil \frac{D}{W \times M} \right\rceil \right) \times P$ + x ⁴	a) write after falling edge of WR, or without preceding write access, or t _{WR_to_RD} ≥ t _{prec_write}
		b) 45 ns $+ \left(2 + \left\lceil \frac{D}{W \times M} \right\rceil \right) \times P$ + x ⁴	b) worst case with preceding write access
t _{BUSY_to_DATA_valid}		a) x ⁴ -5 ns b) x ⁴ -20 ns	DATA bus valid after device BUSY is de-asserted a) normal read busy output b) delayed read busy output
t _{ADR_BE_to_DATA_invalid}	0 ns ⁵		DATA invalid after ADR or BYTE_ENA change
t _{CS_RD_to_DATA_release}	0 ns ⁵	x ⁴	DATA bus released after CS de-assertion or RD de-assertion
t _{CS_to_BUSY_release}	0 ns ⁵	x ⁴	BUSY released after CS de-assertion
t _{CS_delay}	0 ns ⁵		Delay between CS de-assertion and assertion
t _{RD_delay}	x ⁴		Delay between RD de-assertion and assertion
t _{ADR_BE_DATA_setup}	x ⁴		ADR, BYTE_ENA and write DATA valid before WR de-assertion
t _{ADR_BE_DATA_hold}	x ⁴		ADR, BYTE_ENA and write DATA valid after WR de-assertion
t _{WR_active}	x ⁴		WR assertion time
t _{BUSY_to_WR_CS}	0 ns ⁵		WR or CS de-assertion after BUSY de-assertion

⁴ EtherCAT IP core: time depends on synthesis results⁵ EtherCAT IP core: time depends on synthesis results, specified value has to be met anyway

Parameter	Min	Max	Comment
$t_{WR_to_BUSY}$		x^4	BUSY assertion after WR de-assertion
t_{write}	0 ns	$15\text{ ns} + \left(1 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^4$	External write time (WR assertion to BUSY de-assertion)
t_{WR_delay}	x^4		Delay between WR deassertion and assertion
$t_{WR_to_RD}$	0 ns		Delay between WR deassertion and RD assertion
$t_{CS_WR_overlap}$	x^4		Time both CS and WR have to be deasserted simultaneously (only if CS is deasserted at all)
$t_{CS_RD_overlap}$	x^4		Time both CS and RD have to be deasserted simultaneously (only if CS is deasserted at all)

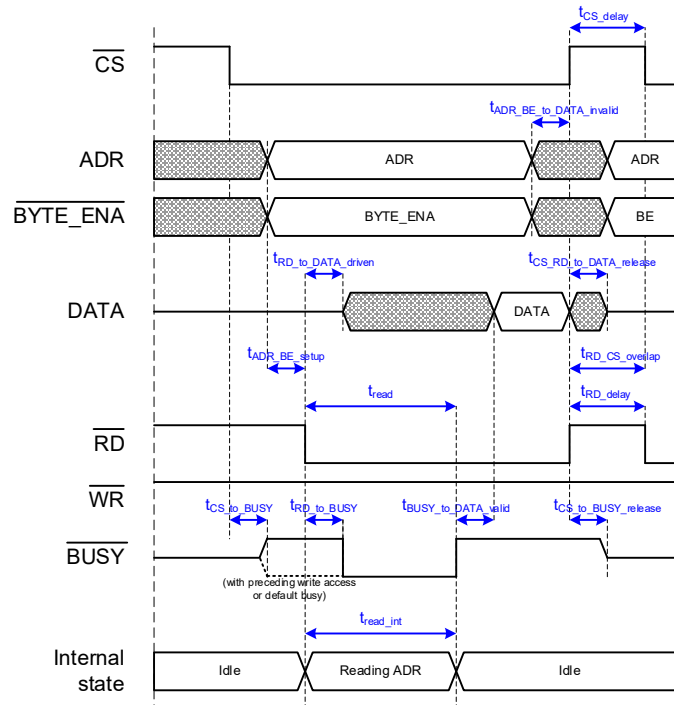


Figure 56: Read access (without preceding write access)

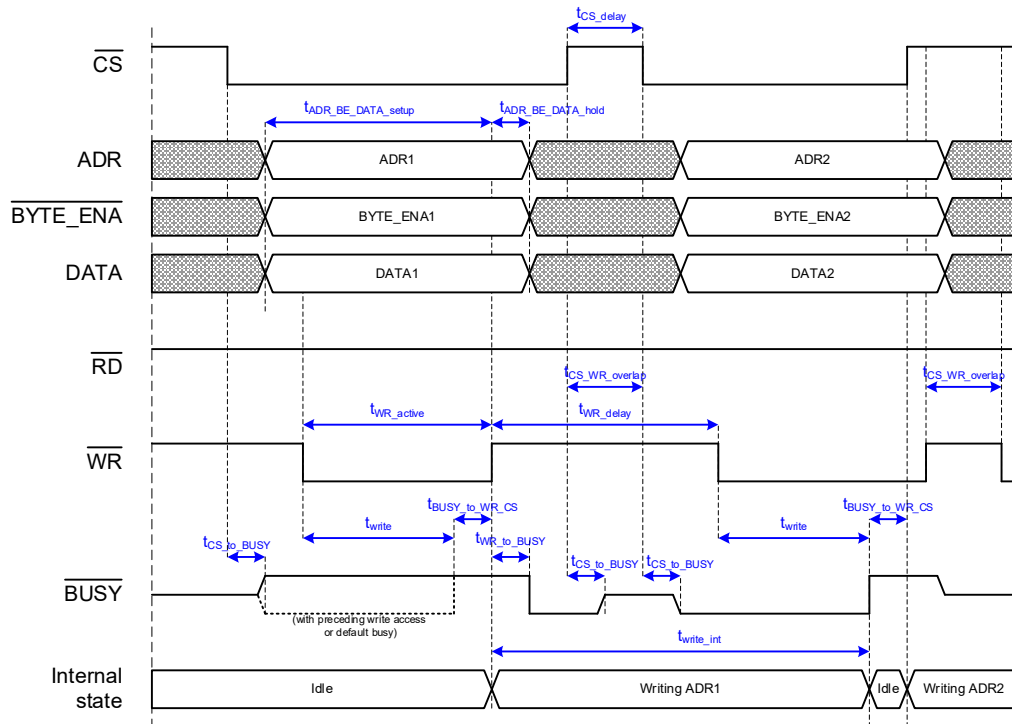


Figure 57: Write access (write after rising edge nWR, without preceding write access)

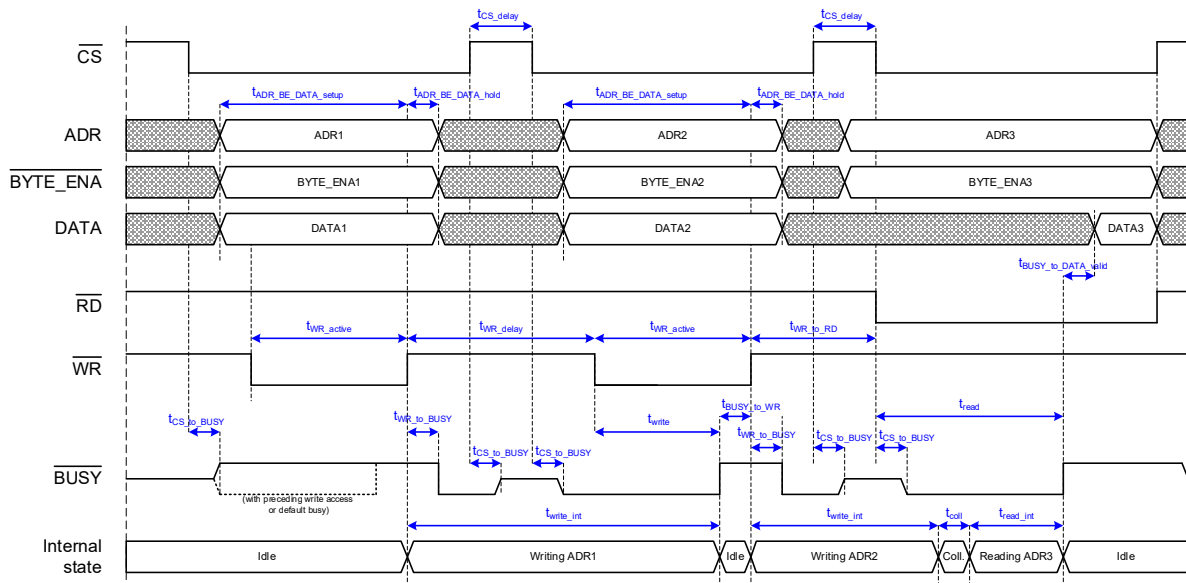


Figure 58: Sequence of two write accesses and a read access

Note: The first write access to ADR1 is performed after the first rising edge of WR. After that, the ESC is internally busy writing to ADR1. After CS is de-asserted, BUSY is not driven anymore, nevertheless, the ESC is still writing to ADR1.

Hence, the second write access to ADR2 is delayed because the write access to ADR1 has to be completed first. So, the second rising edge of WR must not occur before BUSY is gone. After the second rising edge of WR, the ESC is busy writing to ADR2. This is reflected with the BUSY signal as long as CS is asserted.

The third access in this example is a read access. The ESC is still busy writing to ADR2 while the falling edge of RD occurs. In this case, the write access to ADR2 is finished first, and afterwards, the read access to ADR3 is performed. The ESC signals BUSY during both write and read access.

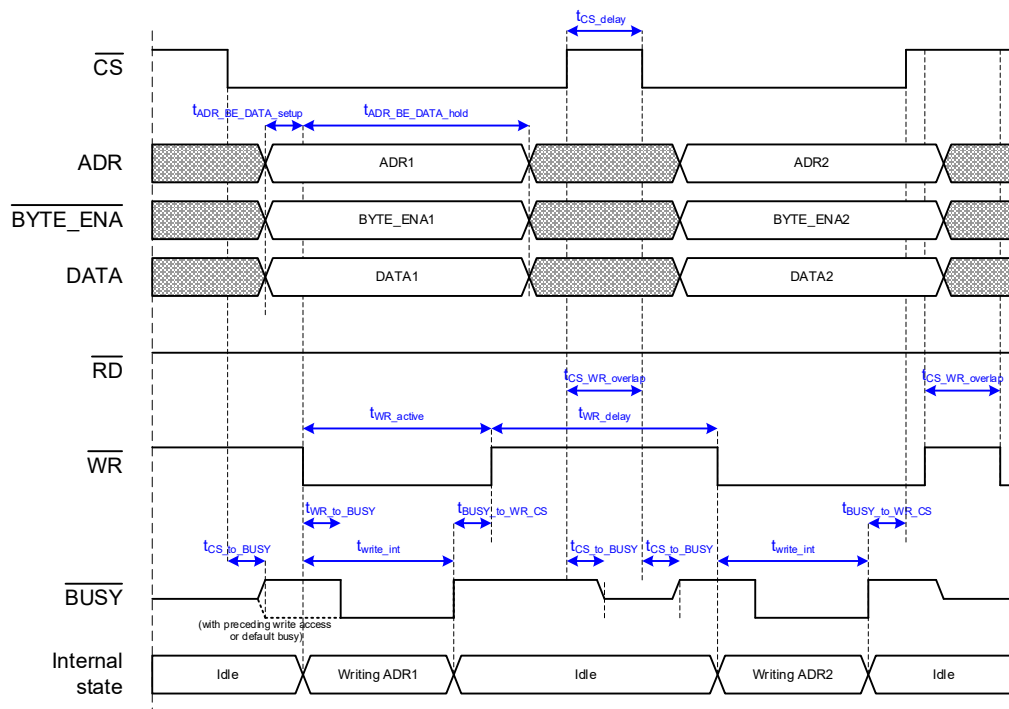


Figure 59: Write access (write after falling edge nWR)

8.5 Avalon on-chip bus

8.5.1 Interface

The Avalon slave PDI is selected with PDI type 0x80. The signals of the Avalon interface are shown below. The IP core signals have a prefix “PDI_AVALON_” for PDI0, and “PDI1_AVALON_” for PDI1, they are omitted in this chapter. The generics CLK_PDI_EXT, CLK_PDI1_EXT, PDI_EXT_BUS_WIDTH, PDI1_EXT_BUS_WIDTH have no prefix. The Avalon PDI uses memory addressing/dynamic bus sizing.

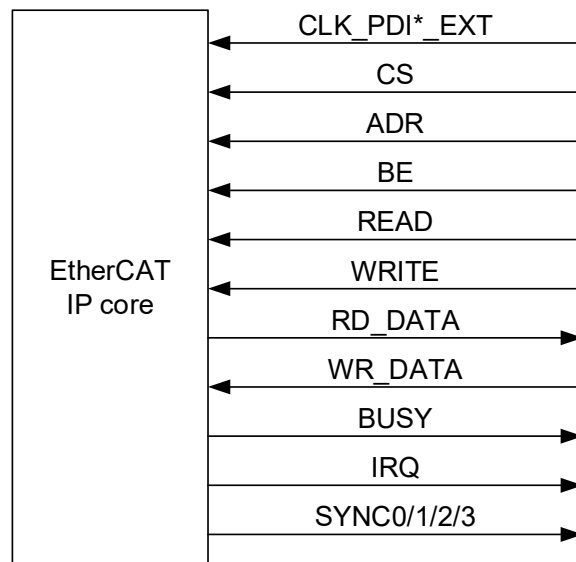


Figure 60: Avalon signals

Table 64: Avalon signals

Signal	Direction	Description	Signal polarity
Avalon generic parameters			
CLK_PDI_EXT	GENERIC	Avalon bus clock PDI0	
CLK_PDI1_EXT	GENERIC	Avalon bus clock PDI1	
PDI_EXT_BUS_WIDTH	GENERIC	Avalon data bus width PDI0	
PDI1_EXT_BUS_WIDTH	GENERIC	Avalon data bus width PDI1	
Avalon signals			
CS	IN	Chip select	act. high
ADR [18-Id(<Avalon data bus width>):0]	IN	Address	
BE [<Avalon data bus width>/8-1:0]	IN	Byte enable	act. high
READ	IN	Read request	act. high
WRITE	IN	Write request	act. high
RD_DATA [<Avalon data bus width>-1:0]	OUT	Read data	
WR_DATA [<Avalon data bus width>-1:0]	IN	Write data	
BUSY	OUT	Busy / wait request	act. high
IRQ	OUT	Interrupt	act. high
SYNC0/1/2/3	OUT	DC SyncSignal 0/1/2/3 used as interrupts	act. high

Please refer to the Avalon memory-mapped interface specification from Altera for details about the Avalon bus (<http://www.altera.com>).

8.5.2 Configuration

The Avalon interface has PDI type 0x80 in the PDI control register 0x0140. The Avalon clock speed and the Avalon data bus width are configurable in the Avalon PDI configuration dialog.

On-chip bus CLK

The Avalon bus clock period can be selected from a wide range. The maximum clock speed depends on the FPGA and the synthesis. When the EtherCAT IP is connected to a bus in the platform designer, this configuration is automatically determined.

On-chip bus CLK is synchronous and a multiple of the internal PDI clock

Configuring the bus clock to be a multiple of the internal PDI clock and to be synchronous to it will result in best performance:

- Internal PDI clock = 25 MHz: Avalon bus clock frequency = $N * 25 \text{ MHz}$ ($N=1\dots31$)
- Internal PDI clock = 50 MHz: Avalon bus clock frequency = $N * 50 \text{ MHz}$ ($N=1\dots31$)
- Internal PDI clock = 100 MHz: Avalon bus clock frequency = $N * 100 \text{ MHz}$ ($N=1\dots31$)

The rising edge of Avalon clock has to be synchronous with the rising edge of CLK25/CLK50/CLK100 of the EtherCAT IP core (otherwise the Avalon bus clock is considered as asynchronous).

On-chip bus CLK is asynchronous or a fraction of the internal PDI clock

Select this option if the bus clock is asynchronous and synchronization is required (results in extra delay), or if the Avalon bus clock is slower than the internal PDI clock (not recommended, reduce internal PDI clock).

8.5.3 Interrupts

The Avalon slave interface supports up to 5 interrupts for easy connection in the platform designer:

- the main/global PDI interrupt (IRQ)
- Distributed clocks SyncSignals 1-4. These interrupts are available if DC are selected, and the option “enable separate DC SyncSignal interrupt signals” is enabled. The DC SyncSignals are still available as standard DC_SYNC[3:0] signals.

8.5.4 On-chip bus access and data bus width

When the Avalon interface is configured for a larger data bus width than the internal PDI bus width, the access will be serialized by the EtherCAT IP core. The lower addresses are accessed first. Byte enable signals are taken into account.

Since an Avalon master always performs read accesses with the whole data bus width (e.g. 32 bit for a NIOS II processor) regardless of the actually issued read command (8/16/32 bit) without use of byte enable signals, care has to be taken especially for SyncManager configuration. SyncManagers should be configured with a length and alignment of multiples of this data width. Alternatively, PDI function acknowledge by write can be used. For write accesses, byte enable signals are used to identify bytes which have to be written.

8.5.5 Timing specifications

Table 65: Avalon timing characteristics

Parameter	Min	Max	Comment
PRELIMINARY TIMING			
W	8 / 16 / 32 bit		Internal PDI bus width
P	10 / 20 / 40 ns		Internal PDI bus period
M	1 / 2		Number of used PDI interfaces (PDI1 used → M=2)
D	8 / 16 / 32 / 64 bit		Avalon data bus width (PDI_EXT_BUS_WIDTH/ PDI1_EXT_BUS_WIDTH)
t _{Clk}	x ⁶	40 ns	Avalon bus clock period (CLK_PDI_EXT, CLK_PDI1_EXT). All bus signals are synchronous to bus clock.
t _{Read}	a) $4 \times t_{CLK} + \left(1 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^6$	a) $3 \times t_{CLK} + \left(2 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^6$	Aligned read access time a) synchronous
	b) $6.5 \times t_{CLK} + \left(3.5 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^6$	b) $6.5 \times t_{CLK} + \left(5.5 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^6$	b) asynchronous
t _{Write}	a) $3 \times t_{CLK}$	a) $\left\lceil \frac{D}{W \times M} \right\rceil \times P + x^6$	Aligned write access time a) synchronous
	b) $5.5 \times t_{CLK} + 2.5 \times P + x^6$	b) $\left(3.5 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^6$	b) asynchronous
t _{First}	1 μs		Time between reset release and first access

⁶ EtherCAT IP core: time depends on synthesis results

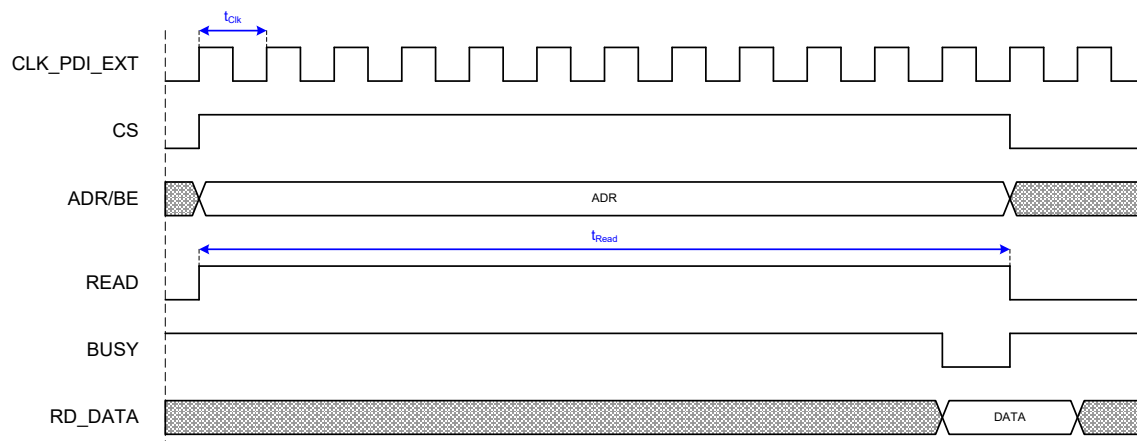


Figure 61: Avalon read access (single access)

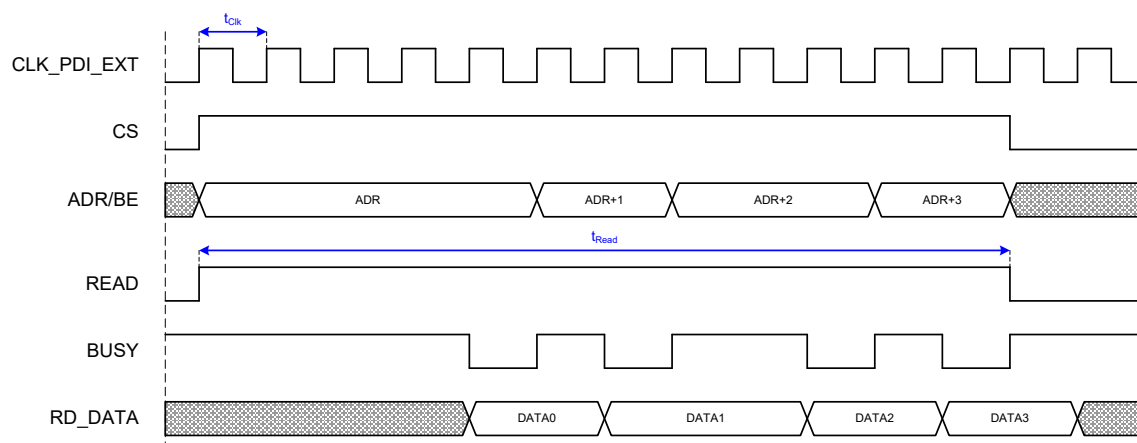


Figure 62: Avalon read access (multiple access)

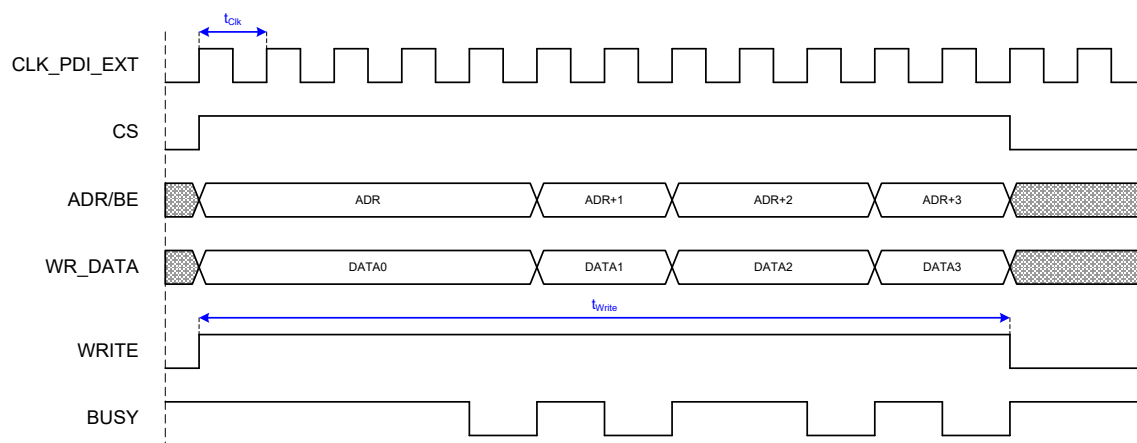


Figure 63: Avalon write access (multiple access)

8.6 AXI4/AXI4-Lite on-chip bus

8.6.1 Interface

The AXI slave PDI is selected with PDI type 0x80. The signals of the AXI interface are shown below. The IP core signals have a prefix “PDI_AXI_” for PDI0, and “PDI1_AXI_” for PDI1, they are omitted in this chapter. The generics CLK_PDI_EXT, CLK_PDI1_EXT, PDI_EXT_BUS_WIDTH, PDI1_EXT_BUS_WIDTH, PDI_BUS_ID_WIDTH, PDI1_BUS_ID_WIDTH have no prefix.

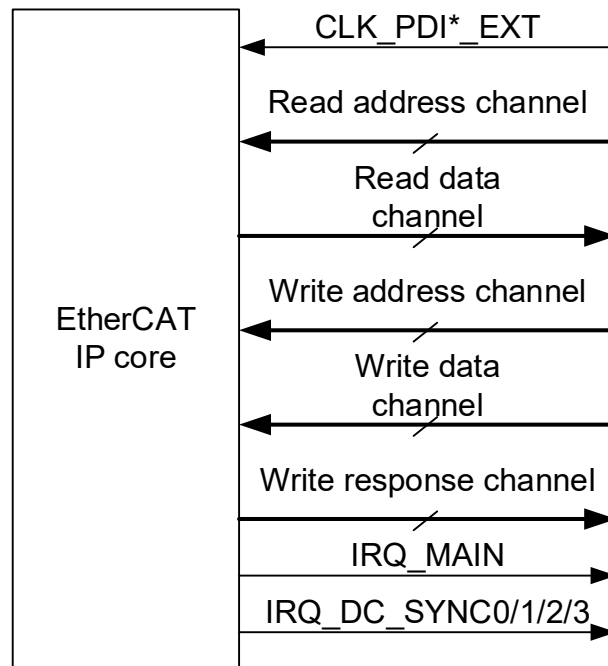


Figure 64: AXI4 signals/channels

Table 66: AXI4-Lite signals

Signal	Direction	Description	Channel	Signal polarity
AXI generic parameters				
CLK_PDI_EXT	GENERIC	AXI bus clock PDI0		
CLK_PDI1_EXT	GENERIC	AXI bus clock PDI1		
PDI_EXT_BUS_WIDTH	GENERIC	AXI data bus width PDI0		
PDI1_EXT_BUS_WIDTH	GENERIC	AXI data bus width PDI1		
AXI signals				
PDI_AXI_AWADDR[15:0]	INPUT	Write address	WR addr.	
PDI_AXI_AWPROT[2:0]	INPUT	Write protection type	WR addr.	
PDI_AXI_AWVALID	INPUT	Write address valid	WR addr.	act. high
PDI_AXI_AWREADY	OUTPUT	Write address ready	WR addr.	act. high
PDI_AXI_WDATA [(<i><AXI data bus width></i> -1:0)]	INPUT	Write data	WR data	
PDI_AXI_WSTRB [<i><AXI data bus width></i> /8-1:0]	INPUT	Write data byte enable	WR data	
PDI_AXI_WVALID	INPUT	Write data valid	WR data	act. high
PDI_AXI_WREADY	OUTPUT	Write data ready	WR data	act. high
PDI_AXI_BRESP[1:0]	OUTPUT	Write response	WR resp.	
PDI_AXI_BVALID	OUTPUT	Write response valid	WR resp.	act. high
PDI_AXI_BREADY	INPUT	Write response ready	WR resp.	act. high
PDI_AXI_ARADDR[15:0]	INPUT	Read address	RD addr.	
PDI_AXI_ARPROT[2:0]	INPUT	Read protection type	RD addr.	

Signal	Direction	Description	Channel	Signal polarity
PDI_AXI_ARVALID	INPUT	Read address valid	RD addr.	act. high
PDI_AXI_ARREADY	OUTPUT	Read address ready	RD addr.	act. high
PDI_AXI_RDATA [<AXI data bus width>-1:0]	OUTPUT	Read data	RD data	
PDI_AXI_RRESP[1:0]	OUTPUT	Read response	RD data	
PDI_AXI_RVALID	OUTPUT	Read data valid	RD data	act. high
PDI_AXI_RREADY	INPUT	Read data ready	RD data	act. high
PDI_AXI_IRQ_MAIN	OUTPUT	Interrupt		act. high
DC_SYNC0/1/2/3	OUT	DC SyncSignal 0/1/2/3 used as interrupts		act. high

Table 67: Additional AXI4 signals

Signal	Direction	Description	Channel	Signal polarity
AXI generic parameters				
PDI_BUS_ID_WIDTH	GENERIC	AXI ID width PDI0		
PDI1_BUS_ID_WIDTH	GENERIC	AXI ID width PDI1		
AXI signals				
PDI_AXI_AWID [<AXI ID width>-1:0]	INPUT	Write address ID	WR addr.	
PDI_AXI_AWLEN[7:0]	INPUT	Write length	WR addr.	
PDI_AXI_AWSIZE[2:0]	INPUT	Write size	WR addr.	
PDI_AXI_AWBURST[1:0]	INPUT	Write burst type	WR addr.	
PDI_AXI_AWLOCK	INPUT	Write lock	WR addr.	
PDI_AXI_AWCACHE[3:0]	INPUT	Write cache type	WR addr.	
PDI_AXI_AWREGION[3:0]	INPUT	Write region identifier	WR addr.	
PDI_AXI_AWQOS[3:0]	INPUT	Write QoS identifier	WR addr.	
PDI_AXI_WLAST	INPUT	Write data last	WR data	act. high
PDI_AXI_BID [<AXI ID width>-1:0]	OUTPUT	Write response ID	WR resp.	
PDI_AXI_ARID [<AXI ID width>-1:0]	INPUT	Read address ID	RD addr.	
PDI_AXI_ARLEN[7:0]	INPUT	Read length	RD addr.	
PDI_AXI_ARSIZE[2:0]	INPUT	Read size	RD addr.	
PDI_AXI_ARBURST[1:0]	INPUT	Read burst type	RD addr.	
PDI_AXI_ARLOCK	INPUT	Read lock	RD addr.	
PDI_AXI_ARCACHE[3:0]	INPUT	Read cache type	RD addr.	
PDI_AXI_ARREGION[3:0]	INPUT	Read region identifier	RD addr.	
PDI_AXI_ARQOS[3:0]	INPUT	Read QoS identifier	RD addr.	
PDI_AXI_RID [<AXI ID width>-1:0]	OUTPUT	Read data ID	RD data	
PDI_AXI_RLAST	OUTPUT	Read data last	RD data	act. high

Please refer to the AMBA® AXI and ACE® protocol specification from ARM® for details about the AXI4/AXI4-Lite interface (<http://www.arm.com>).

8.6.2 Configuration

The AXI4 interface has PDI type 0x80 in the PDI control register 0x0140. The on-chip bus subtype is "001" for AXI4 and "010" for AXI4-Lite in the PDI on-chip bus extended configuration register 0x0152:0x0153. The AXI clock speed, data bus width, and ID width are set by generics.

On-chip bus CLK

The AXI bus clock period can be selected from a wide range. The maximum clock speed depends on the FPGA and the synthesis. When the EtherCAT IP is connected to a bus in the platform designer, this configuration is automatically determined.

On-chip bus CLK is synchronous and a multiple of the internal PDI clock

Configuring the bus clock to be a multiple of the internal PDI clock and to be synchronous to it will result in best performance:

- Internal PDI clock = 25 MHz: AXI bus clock frequency = $N * 25 \text{ MHz}$ ($N=1\dots31$)
- Internal PDI clock = 50 MHz: AXI bus clock frequency = $N * 50 \text{ MHz}$ ($N=1\dots31$)
- Internal PDI clock = 100 MHz: AXI bus clock frequency = $N * 100 \text{ MHz}$ ($N=1\dots31$)

The rising edge of AXI clock has to be synchronous with the rising edge of CLK25/CLK50/CLK100 of the EtherCAT IP core (otherwise the AXI bus clock is considered as asynchronous).

On-chip bus CLK is asynchronous or a fraction of the internal PDI clock

Select this option if the bus clock is asynchronous and synchronization is required (results in extra delay), or if the AXI bus clock is slower than the internal PDI clock (not recommended, reduce internal PDI clock).

8.6.3 Interrupts

The AXI slave interface supports up to 5 interrupts for easy connection in the platform designer:

- the main/global PDI interrupt (IRQ)
- Distributed clocks SyncSignals 1-4. These interrupts are available if DC are selected, and the option "enable separate DC SyncSignal interrupt signals" is enabled. The DC SyncSignals are still available as standard DC_SYNC[3:0] signals.

8.6.4 On-chip bus access and data bus width

When the AXI interface is configured for a larger data bus width than the internal PDI bus width, the access will be serialized by the EtherCAT IP core. The lower addresses are accessed first. Byte enable signals are taken into account.

Since an AXI master always performs read accesses with the whole data bus width (e.g. 32 bit for a Microblaze processor) regardless of the actually issued read command (8/16/32 bit) without use of byte enable signals, care has to be taken especially for SyncManager configuration. SyncManagers should be configured with a length and alignment of multiples of this data width. Alternatively, PDI function acknowledge by write can be used. For write accesses, byte enable signals are used to identify bytes which have to be written.

8.6.5 Timing specifications

Table 68: AXI timing characteristics

Parameter	Min	Max	Comment
PRELIMINARY TIMING			
W		8 / 16 / 32 bit	Internal PDI bus width
P		10 / 20 / 40 ns	Internal PDI bus period
M		1 / 2	Number of used PDI interfaces (PDI1 used → M=2)
D		8, 16, 32, or 64 bit	AXI data bus width (PDI_EXT_BUS_WIDTH/ PDI1_EXT_BUS_WIDTH)
t _{clk}	x ⁷	40 ns	AXI bus clock period (CLK_PDI_EXT, CLK_PDI1_EXT). All bus signals are synchronous to bus clock.
t _{Read} t _{Write}	a) $t_{CLK} + \left(1 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^7$ b) $3.5 \times t_{CLK} + \left(2.5 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^7$	a) $\left(2 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^7$ b) $3.5 \times t_{CLK} + \left(4.5 + \left\lceil \frac{D}{W \times M} \right\rceil\right) \times P + x^7$	Aligned read/write access time a) synchronous, read or write b) asynchronous, read or write
t _{First}	1 μs		Time between reset release and first access
BW _{int}		a) $\frac{W}{8 \times P}$ b) 400 Mbyte/s	a) Internal PDI bandwidth limit [Byte/s] for read or write accesses b) Abs. max internal PDI bandwidth for 32 bit/100 MHz internal PDI

⁷ EtherCAT IP core: time depends on synthesis results

Figure 65: AXI read access

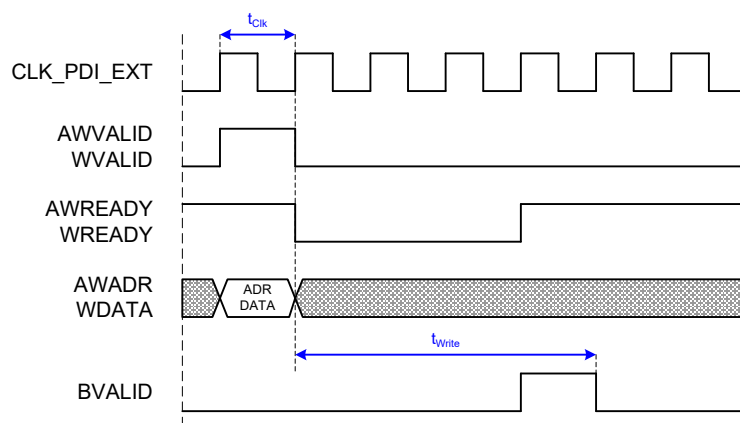


Figure 66: AXI write access

9 Distributed clocks DC_SYNC/DC_LATCH signals

For details about the distributed clocks refer to section I.

9.1 Signals

The distributed clocks unit of the IP core has the following external signals (depending on the ESC configuration):

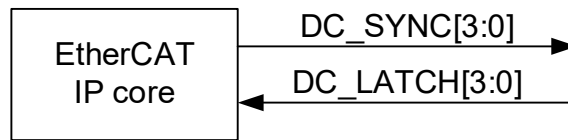


Figure 67: Distributed clocks signals

Table 69: Distributed clocks signals

Signal	Direction	Description	Signal polarity
DC_SYNC[3:0]	OUT	SyncSignals	Act. high
DC_LATCH[3:0]	IN	LatchSignals	Act. high

9.2 Timing specifications

Table 70: DC SYNC/LATCH timing characteristics IP core

Parameter	Min	Max	Comment
t_{DC_LATCH}	$12\text{ ns} + x^8$		Time between DC_LATCH events
$t_{DC_SYNC_Jitter}$		$11\text{ ns} + x^1$	DC_SYNC output jitter
$t_{DC_SYNC_IRQ_pulse_width}$	40 ns		DC_SYNC pulse width if the DC_SYNC signal is used as AL event request (PDI interrupt) signal (masked by register 0x0220 ff.), and if acknowledge mode is not used.

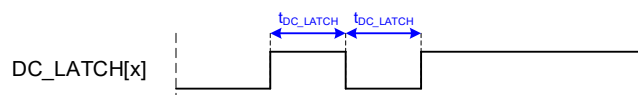


Figure 68: LatchSignal timing

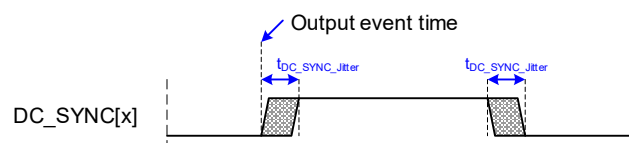


Figure 69: SyncSignal timing

⁸ EtherCAT IP core: time depends on synthesis results

10 SII EEPROM interface (I²C)

For details about the ESC SII EEPROM interface refer to section I. The SII EEPROM interface is intended to be a point-to-point interface between IP core and I²C EEPROM. If other I²C masters are required to access the I²C bus, the IP core must be held in reset state (e.g. for in-circuit-programming of the EEPROM).

10.1 Signals

The EEPROM interface of the IP core has the following signals:

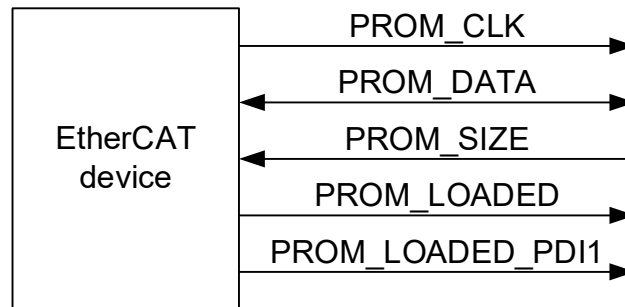


Figure 70: I²C EEPROM signals

Table 71: I²C EEPROM signals

Signal	Direction	Description	Signal polarity
PROM_CLK	OUT	I ² C clock	
PROM_DATA	BIDIR	I ² C data	
PROM_SIZE	IN	EEPROM size configuration	
PROM_LOADED	OUT	EEPROM is loaded (area A)	act. high
PROM_LOADED_PDI1	OUT	EEPROM is loaded (area B)	act. high

Both PROM_CLK and PROM_DATA must have a pull-up resistor (4.7 kΩ recommended for ESCs), either integrated into the ESC or externally.

10.2 Timing specifications

Table 72: EEPROM timing characteristics IP core

Parameter	Typical		Comment
	Up to 16 KBit	32 KBit-4 MBit	
t _{CLK}	~ 6.72 μs		EEPROM clock period (f _{CLK} ≈ 150 kHz)
t _{Write}	~ 250 μs	~ 310 μs	Write access time (without errors)
t _{Read}	a) ~ 440 μs b) ~ 1.23 ms c) ~ 2.41 ms	a) ~ 500 μs b) ~ 1.29 ms c) ~ 2.53 ms	Read access time (without errors): a) 2 words b) ESC configuration area A only (8 words) c) ESC configuration area A+B (16 words)
t _{Delay}	~ 80 μs		Time until configuration loading begins after reset is gone

11 Electrical specifications

Table 73: AC Characteristics

Symbol	Parameter	Min	Typ	Max	Units
f_{CLK25}	Clock source (CLK25) with initial accuracy at room temperature	25 MHz $\pm$ 25 ppm			

Table 74: Forwarding delays

Symbol	Parameter	Min	Average	Max	Units
PRELIMINARY TIMING					
t_{Diff}	Average difference processing delay minus forwarding delay (without RX FIFO jitter)		40		ns
t_{MM}	MII port to MII port delay: a) Through ECAT processing unit (processing) b) Alongside ECAT processing unit (forwarding) Conditions: FIFO size 7, no TX shift compensation or manual TX shift configuration with MII_TX_SHIFT = 00	a) 280+x ⁹ b) 240+x ⁹	a) 300+x ⁹ b) 260+x ⁹	a) 320+x ⁹ b) 280+x ⁹	ns
t_{RR}	RMII port to RMII port delay: a) Through ECAT processing unit (processing) b) Alongside ECAT processing unit (forwarding) Conditions: FIFO size 7 NOTE: Delay is low because receive FIFO is inside PHY	a) 140+x ⁹ b) 100+x ⁹	a) 140+x ⁹ b) 100+x ⁹	a) 140+x ⁹ b) 100+x ⁹	ns
t_{GG}	RGMII port to RGMII port delay: a) Through ECAT processing unit (processing) b) Alongside ECAT processing unit (forwarding) Conditions: FIFO size 7	a) 440+x ⁹ b) 400+x ⁹	a) 460+x ⁹ b) 420+x ⁹	a) 480+x ⁹ b) 440+x ⁹	ns

NOTE: Average timings are used for DC calculations. Delays are taken from the clock edge with the first receive data, until transmit data is presented.

⁹ EtherCAT IP core: time depends on synthesis results

12 Synthesis constraints

The timing constraints required for the EtherCAT IP core result from interface and clock timing requirements. This chapter describes the basic timing requirements and it contains example constraints to fulfill them. Please refer to the example designs and the constraint file in the documentation folder for detailed constraining examples.

To prevent Quartus from adding pass-through logic to the memory which leads to additional resource requirements and timing restrictions, the Analysis & Synthesis option "Add Pass-Through Logic to Inferred RAMs" should be set to "Off".

Turn on Analysis & Synthesis option: auto RAM Replacement, otherwise the RAM inside the IP core will be implemented with individual registers.

12.1 Timing requirements

Table 75: EtherCAT IP core constraints

Signal	Requirement	Value	Clock reference	Description
CLK25	period	40 ns		Reference clock (25 MHz)
CLK25_2NS	a) period b) phase shift	a) 40 ns b) 2 ns	CLK25	Derived clock (25 MHz). Phase shift is rising edge CLK25 to rising edge CLK25_2NS.
CLK50	a) period b) phase shift	a) 20 ns b) 0 ns	CLK25	Derived clock (50 MHz). Phase shift is rising edge to rising edge.
CLK100	a) period b) phase shift	a) 10 ns b) 0 ns	CLK25	Derived clock (100 MHz). Phase shift is rising edge to rising edge.
CLK_PDI_EXT, CLK_PDI1_EXT	a) period b) phase shift c) period d) phase shift	a) 40 ns/N b) 0 ns c) app. specific d) don't care	a,b) CLK25 c, d) none	a, b) CLK_PDI_EXT is synchronous to CLK 25, period is integer fraction of CLK25 period. Phase shift is rising edge to rising edge. c, d) CLK_PDI_EXT is asynchronous to CLK25.
nRESET	Ignore timing			nRESET is asynchronous to any clock
MCLK/MCLKn	min. period	400 ns		IEEE802.3 requirement (2.5 MHz)
MDIO/MDION	a) setup b) hold at PHY input	a) 10 ns b) 10 ns	MCLK (rising edge)	MDIO is changed with falling edge of MCLK, max. output skew of MCLK and MDIO is 190 ns. Constraining is usually not required. IEEE802.3 requirement.
MII_RX_CLK0-3	period	40 ns		MII receive reference clock (25 MHz). IEEE802.3 requirement.
MII_RX_DATA0-3[3:0] MII_RX_DV0-3 MII_RX_ERR0-3	a) setup b) hold	a) 10 ns b) 10 ns	MII_RX_CLK0-2 (rising edge)	IEEE802.3 requirement
MII_TX_CLK0-3	period	40 ns		MII transmit reference clock (25 MHz). Only used for automatic TX shift compensation. IEEE802.3 requirement.
MII_TX_DATA0-3[3:0] MII_TX_ENA0-3	Clock-to-Pin a) min b) max	a) 0 ns b) 25 ns	TX_CLK0-2 from PHY (rising edge)	IEEE802.3 requirement
PROM_CLK	period	App. dep.		I ² C clock. Actual ESC output clock is 6.72 μ s ($\approx$ 150 kHz). Min. 2.5 μ s (400 Khz) for example I ² C EEPROM chip.
PROM_DATA	a) setup b) hold	a) 250 ns b) 0 ns	PROM_CLK a) rising edge b) falling edge	PROM_DATA is changed in the middle of the low phase of PROM_CLOCK, i.e., max. output skew of PROM_CLK/PROM_DATA is 1.43 μ s. Constraining is usually not required. Example I ² C EEPROM chip requirement.
RMII_RX_DATA0-3[1:0] RMII_RX_DV0-3 RMII_RX_ERR0-3 RMII_TX_DATA0-3[1:0] RMII_TX_ENA0-3	a) setup b) hold	a) 4 ns b) 2 ns	CLK50 (rising edge)	RMII specification requirement
RGMII_RX_CLK0-3	period	a) 8 ns b) 40 ns c) 400 ns		RGMII receive reference clock. RGMII spec. requirement. Link detection (in-band signaling) supports 10/100/1000 Mbit/s, but EtherCAT supports only 100 Mbit/s. Needs to be constrained for 1000 Mbit/s a) 1000 Mbit/s b) 100 Mbit/s c) 10 Mbit/s

Signal	Requirement	Value	Clock reference	Description
RGMII_RX_CTL0-3 RGMII_RX_DATA0-3[3:0]	a) setup b) hold	a) b)	RGMII_RX_CLK 0-2 (both edges)	Depending on RX_CLK delay option, RGMII spec. requirement
RGMII_TX_CLK0-3	a) period b) phase shift	a) 40 ns b) 2 ns	CLK25_2NS	RGMII transmit reference clock (25 MHz), derived from CLK25_2NS. RGMII spec. requirement.
RGMII_TX_CTL0-3 RGMII_TX_DATA0-3[3:0]	Clock-to-Pin a) min b) max	a) b)	RGMII_TX_CLK 0-2 (both edges)	Depending on TX_CLK delay option, RGMII spec. requirement
Other signals, especially PDI signals	application dependent			

12.2 Automatic TX shift compensation

The requirement for transmit signals MII_TX_ENA/MII_TX_DATA is that they must be updated within 0 ns to 25 ns after the rising edge of MII_TX_CLK (according to IEEE802.3). Some PHYs have relaxed requirements, so you may change the values accordingly.

As the transmit signals are not generated from a transmit FIFO, they have no direct relationship to MII_TX_CLK, but they have a fixed phase relation to MII_TX_CLK (because ESC and PHY use the same clock source). The auto TX shift feature compensates this phase shift by measuring the TX_CLK phase relation to CLK100 and selecting an appropriate output delay for the TX signals between 0-3 register delays at CLK100. Without delays, the automatic TX shift function (with MII_TX_SHIFT="00") makes sure that the TX signals change between 0 ns and 10 ns after the rising edge of MII_TX_CLK. Figure 71 shows a principle diagram of the paths used for constraining MII TX signals.

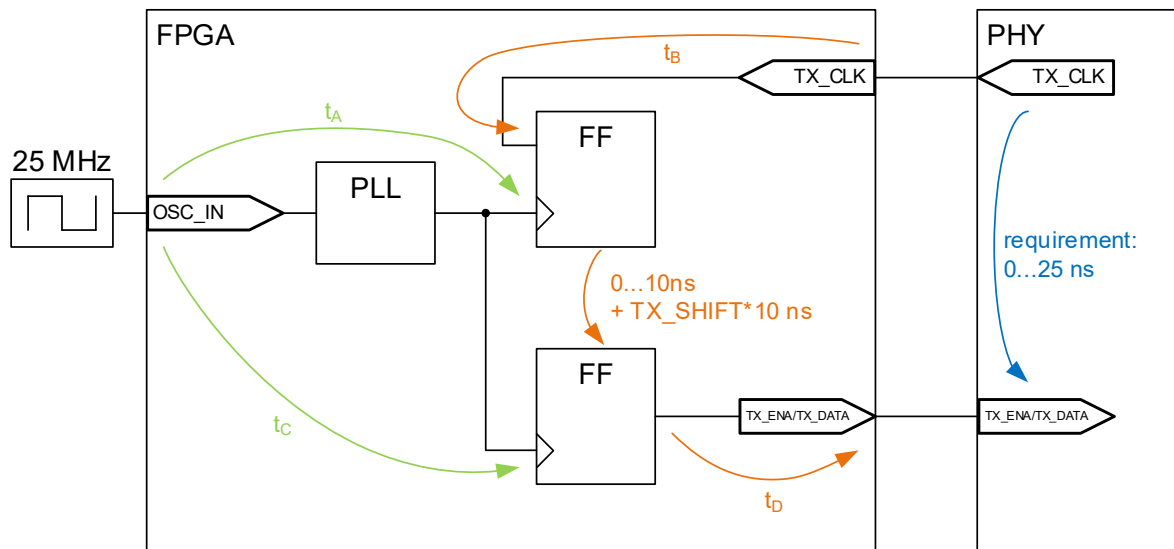


Figure 71: MII auto TX shift constraining (principle)

The delay inside the FPGA between MII_TX_CLK input and MII_TX_ENA/MII_TX_DATA outputs consists of

- physical delay t_B delay along the input paths (MII_TX_CLK input to IP core flip-flops)
- physical delay t_D delay along the output paths (IP core flip-flops to MII_TX_DATA and MII_TX_ENA ports)
- design jitter caused by the auto TX shift compensation (0...10 ns)
- additional configured design delay (MII_TX_SHIFT value * 10 ns)
- physical clock skew between the flip-flops for measurement and output ($t_C - t_A$)

The total delay inside the FPGA must stay within the timing window of 0...25 ns for any combination of input path and output path:

$$0 \text{ ns} < (t_B - t_A) + (t_C + t_D) + 0 \dots 10 \text{ ns} + \text{MII_TX_SHIFT} * 10 \text{ ns} < 25 \text{ ns}$$

As far as we know, this cannot be constrained directly with current FPGA tools. We suggest constraining input delay and output delay independently. In the above equation, the input/output paths were already grouped with the corresponding clock delays, so they can be used in constraints easily:

$$t_{\text{Input}} = t_B - t_A$$

$$t_{\text{Output}} = t_C + t_D$$

Using the default MII_TX_SHIFT value of “00”, a timing window of 0 ns to 15 ns remains for any physical delays in the FPGA (other MII_TX_SHIFT values are not recommended):

$$0 \text{ ns} < t_{\text{Input}} + t_{\text{Output}} < 15 \text{ ns}$$

You should reduce this window to allow for some skew in traces on the PCB and clock skew inside the FPGA. The following example will use a window of 1 ns to 14 ns. Finally, we want to achieve the following timing constraints:

Maximum delay: $t_{\text{Input_max}} + t_{\text{Output_max}} < 14 \text{ ns}$

Minimum delay: $t_{\text{Input_min}} + t_{\text{Output_min}} > 1 \text{ ns}$

Allotting the total delay budget to input and output paths is the difficult part of this approach, and it may have to be redone after changes to the design. The tool will subtract clock delay for input paths and add it for output paths. This influence of clock delay is cancelled out in the sum of input and output delays, but it affects how much of the timing window must be allotted to the input and output paths.

In the following, we focus on the maximum delay constraint for two reasons:

- the clock skew between the input and output flip-flops is minimal in FPGA designs, so the minimum delay is positive anyhow, due to pad and routing delays
- the FPGA tools have significant problems with minimum delay constraints, they often result in worse solutions due to inappropriate buffering of signals

The minimum delay constraints are shown in the examples, but they can be commented out.

We suggest this process:

1. Synthesize the design without input/output constraints for MII_TX_CLK, MII_TX_DATA[*], and MII_TX_ENA (but with constraints for all other signal if possible).
2. Add max delay constraints for input and output paths of MII_TX_CLK, MII_TX_DATA[*], and MII_TX_EN, but do not synthesize again, just check the timing:
3. Report the max delays along all input paths (MII_TX_CLK port to IP core) and along all output paths (IP core to MII_TX_ENA and MII_TX_DATA[*] ports). A TCL function *report_mii_tx_timing* is part of the constraints setting file, and may assist in reporting the timing.
4. Change max delay constraints for input and output paths to values which are about 1 ns larger than the actual timing. It is ok if the sum is violating the 14 ns limit at this point.
5. Synthesize the design with these max delay constraints and report the delays again. Tighten the values iteratively until the max delay sum is below 14 ns.

Additional steps for minimum timing (not recommended):

6. Add min delay constraints, but do not synthesize again, just check the timing:
7. Report the min delays along all input/output paths.
8. Change min delay constraints for input and output paths to values which are about 1 ns smaller than the actual timing. It is ok if the sum is violating the 1 ns limit at this point. Please note that the delays can be negative (e.g. -1.5 ns for input paths and 2.5 ns for output paths)
9. Synthesize the design with min and max delay constraints and report the delays again. Tighten the values iteratively until the min delay sum is above 1 ns.

Finding a working split of the timing window can be a lot of work for some designs. If the delay along the longest input path plus the longest output path is greater than 14 ns, you have to optimize the design. This is sometimes caused by unnecessary clock buffers on the MII_TX_CLK signal, by delay in the PLL because of a wrong clock compensation mode, or by configured delay in the I/O cells.

In such cases, it has been found beneficial to either use fast input registers for MII_TX_CLK, or to add an additional register for MII_TX_ENA/MII_TX_DATA clocked by CLK100 (increases latency), which can be placed near/inside the output pads (set MII_TX_SHIFT value to 3 or "11" to compensate the delay).

Theoretically, without adding an additional register, MII_TX_SHIFT could be used to move the TX signals completely into the following TX_CLK period, if $t_{\text{Input}} + t_{\text{Output}}$ would exceed 15 ns. This equation must be met in such cases:

$$0 \text{ ns} < t_{\text{Input}} + t_{\text{Output}} + 0 \dots 10 \text{ ns} + \text{MII_TX_SHIFT} * 10 \text{ ns} - 40 \text{ ns} < 25 \text{ ns}$$

This means that a timing window of 10 ns to 25 ns remains for any delays in the FPGA ($t_b + t_d$), if a MII_TX_SHIFT value of 3 (or "11") is used. Unfortunately, FPGA tools will use significant buffering to achieve the minimum timing, which causes large timing variations over temperature/power/process variations. So changing MII_TX_SHIFT without adding a register is not a solution.

12.3 Manual TX shift compensation

Manual TX shift compensation consists of these steps:

1. constraining the MII_TX_ENA/MII_TX_DATA output paths
2. measure the timing at the PHY
3. set MII_TX_SHIFT to an appropriate value which fulfills the PHY requirements.
4. Check the timing at the PHY again

13 Appendix

13.1 Third party trademarks

Altera, Quartus, Avalon, Nios, Agilex, Arria, Cyclone, Stratix, MAX® are trademarks of Altera Corporation or its parent corporation.

Intel, Intel Atom are trademarks of Intel Corporation or its subsidiaries.

Arm, AMBA, ACE are registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere.

13.2 Support and service

Beckhoff and their partners around the world offer comprehensive support and service, making available fast and competent assistance with all questions related to Beckhoff products and system solutions.

13.2.1 Beckhoff's branch offices and representatives

Please contact your Beckhoff branch office or representative for local support and service on Beckhoff products!

The addresses of Beckhoff's branch offices and representatives round the world can be found on her internet pages: <http://www.beckhoff.com>

You will also find further documentation for Beckhoff components there.

13.3 Beckhoff headquarters

Beckhoff Automation GmbH & Co. KG
Huelshorstweg 20
33415 Verl
Germany

phone: + 49 (0) 5246/963-0

fax: + 49 (0) 5246/963-198

e-mail: info@beckhoff.com

web: www.beckhoff.com

Beckhoff support

Support offers you comprehensive technical assistance, helping you not only with the application of individual Beckhoff products, but also with other, wide-ranging services:

- world-wide support
- design, programming and commissioning of complex automation systems
- and extensive training program for Beckhoff system components

hotline: + 49 (0) 5246/963-157

fax: + 49 (0) 5246/963-9157

e-mail: support@beckhoff.com

Beckhoff service

The Beckhoff service center supports you in all matters of after-sales service:

- on-site service
- repair service
- spare parts service
- hotline service

hotline: + 49 (0) 5246/963-460

fax: + 49 (0) 5246/963-479

e-mail: service@beckhoff.com